

11/01/22

Introduction

Assessment Plan

- 10%: Quiz-1
- 10%: Quiz-2
- 20%: Mid-term Exam (individual)
- 25%: Programming Assignment (group solutions)
- 35%: End-term Exam (individual)

Definitions of Cloud Computing

Data center hardware and software that provides services.

- Armbrust, et al., 2009

[Preliminary definition]

A style of computing in which massively scalable IT-related capabilities are provided **as a service** using Internet technologies to multiple external customers

- Gartner, 2008

[Expanded definition]

A **pay-per-use model** for enabling available, convenient, on-demand network access to a **shared pool of configurable computing resources** (e.g. networks, servers, storage, applications, services) that can be rapidly provisioned and released with **minimal management effort** or service provider interaction.

- National Institute of Standards and Technology (NIST), 2009

[Important definition]

Cloud Computing is using the Internet to access someone else's software running on someone else's hardware in someone else's data center

- Lewis Cunningham, 2010

[Chad definition]

Basically, the user/client doesn't need to worry about who is running the software, where it is running, whose software is in use, etc.

Cloud is a parallel and distributed computing system consisting of a collection of interconnected and virtualized computers that are dynamically provisioned and presented as one or more unified computing resources based on service-level

agreements (SLA) established through negotiation between the service provider and consumers.

- Buyya et al., 2009

[Refined definition and the most important one]

History of Cloud Computing

Centralized vs Distributed Systems

Centralized Systems: State stored on a single computer.

Advantages:

- Simpler to set up
- Easier to understand
- Can be faster for a single user

The database and the DBMS are located on the same machine which can be used by one or more machines.

Distributed Systems

A distributed system can be characterized as a collection of mostly autonomous processors communicating over a communication network and having the following features,

- No common physical clock
- No shared memory
- Geographical separation
- Autonomy and heterogeneity (amongst the machines/processors).

State is divided over multiple computers but presented as a single unified unit (virtualization).

Advantages:

- More robust (more fault tolerance)
- More scalable (supports more traffic and users)
- More complex

The database and the DBMS software are distributed and are connected by a computer network. Data is often replicated and stored at different servers as copies in order to ensure fault tolerance.

So if a server fails and its data is lost, then a different server which stored the same data can be used for recovery.

Distributed DBMS provides,

- Distribution and Network transparency
- Replication and Fragmentation transparency
- Increased reliability and availability
- Easier expansion / scalability

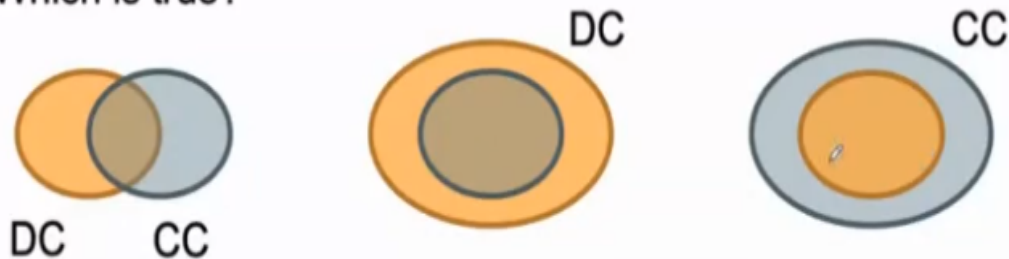
Transparency refers to the fact that the user doesn't need to be concerned with that aspect, i.e. Replication transparency means that the user doesn't need to know that the data being used is replicated in order to aid fault tolerance.

Same goes for any other transparencies.

Even though we get all these benefits, distributed query processing very challenging and it is very difficult to implement Transactional Semantics, i.e. ACID properties.

Distributed Computing (DC) vs Cloud Computing (CC)

Which is true?



This debate is pointless and very subjective and all 3 options can be proven right on their own merit.

But what is undeniable is that Distributed Computing (started in the 1980s) led to Cloud Computing (started in the 2010s).

The pay-per-use model for Cloud Computing evolved from something called **Utility Computing** which usually talked about paying for usage of services like gas, electricity, etc.

Grid Computing which was introduced in the 1990s put forward the notion of sharing under-utilized resources to a community that needs them.

These resources were not being allotted on an on-demand basis, one can request but it is subject to availability.

Grid Computing systems link computer resources together in a way that lets someone use one computer to access and leverage the collected power of all the computers in the system.

The need of Grid Computing makes a lot of sense for tackling tasks like genome sequencing, protein folding, etc. which requires a lot of computation power in an era where computation resources were scarce.

Cloud Computing ofcourse borrows the notion of sharing resources for sure but it is more business and service oriented, no longer concerned about underutilization of resources and users that are willing to pay should receive the service.

Characteristics of Cloud Computing

The illusion of infinite dedicated computing resources.

The elimination of an up-front commitment by cloud users.

The ability to pay for use.

Cloud Architecture

The following points should be understood to define cloud architecture,

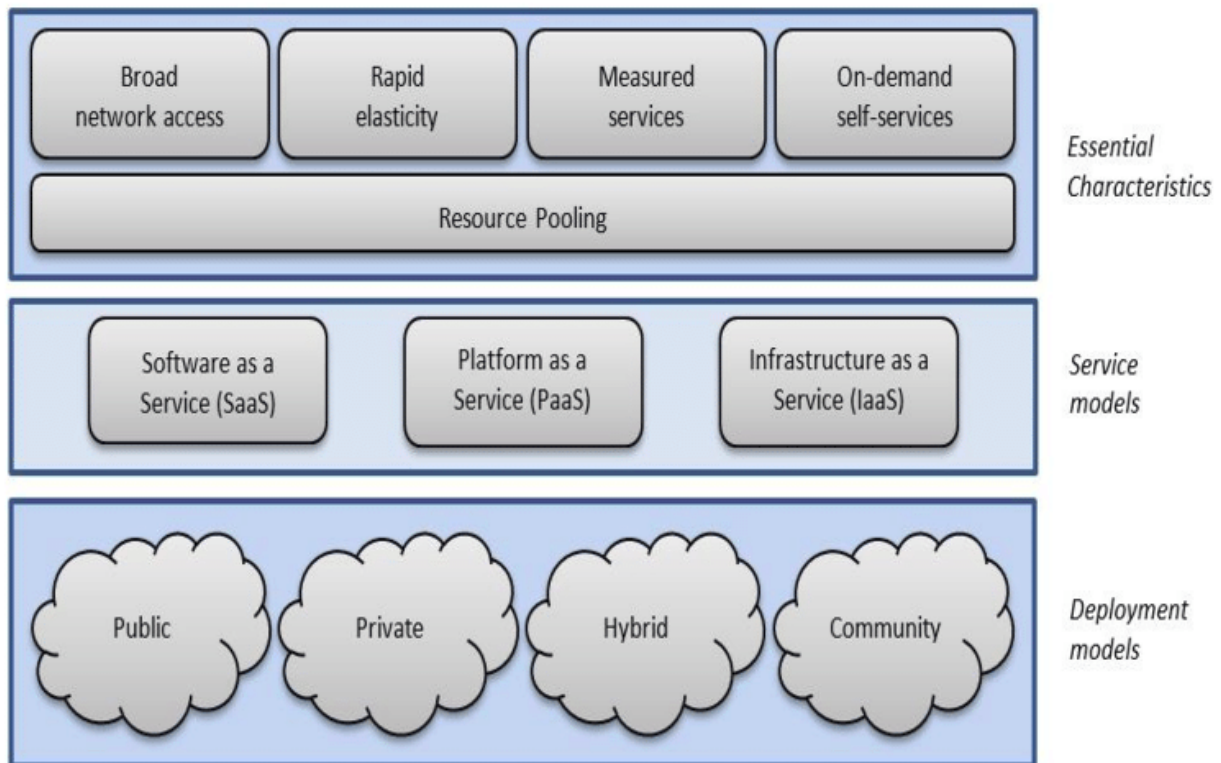
Service Models: What are the types of services it is going to offer?

Deployment Models: How can we deploy the cloud infrastructure that provides you with the services?

Essential Characteristics: What are the essential characteristics we expect from a cloud?

NIST Visual Model of Cloud Computing Definition

[This defines all the 3 components mentioned above]



Essential Characteristics

On-Demand Self Service

- A consumer should be provisioned computing capabilities such as server time and network storage as needed automatically without requiring human interactions with a service provider.
- That is, the service provider should be able to handle requests from a subscriber without any human interaction.

Broad Network Access

- Capabilities must be available over the network.
- These capabilities are accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (like mobile phones, laptops, etc) as well as other traditional or cloud-based software services.
- There should be a sense of universality while accessing the cloud services.

Resource Pooling

- The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model with different physical and virtual resources which are dynamically assigned and reassigned as per consumer demand.

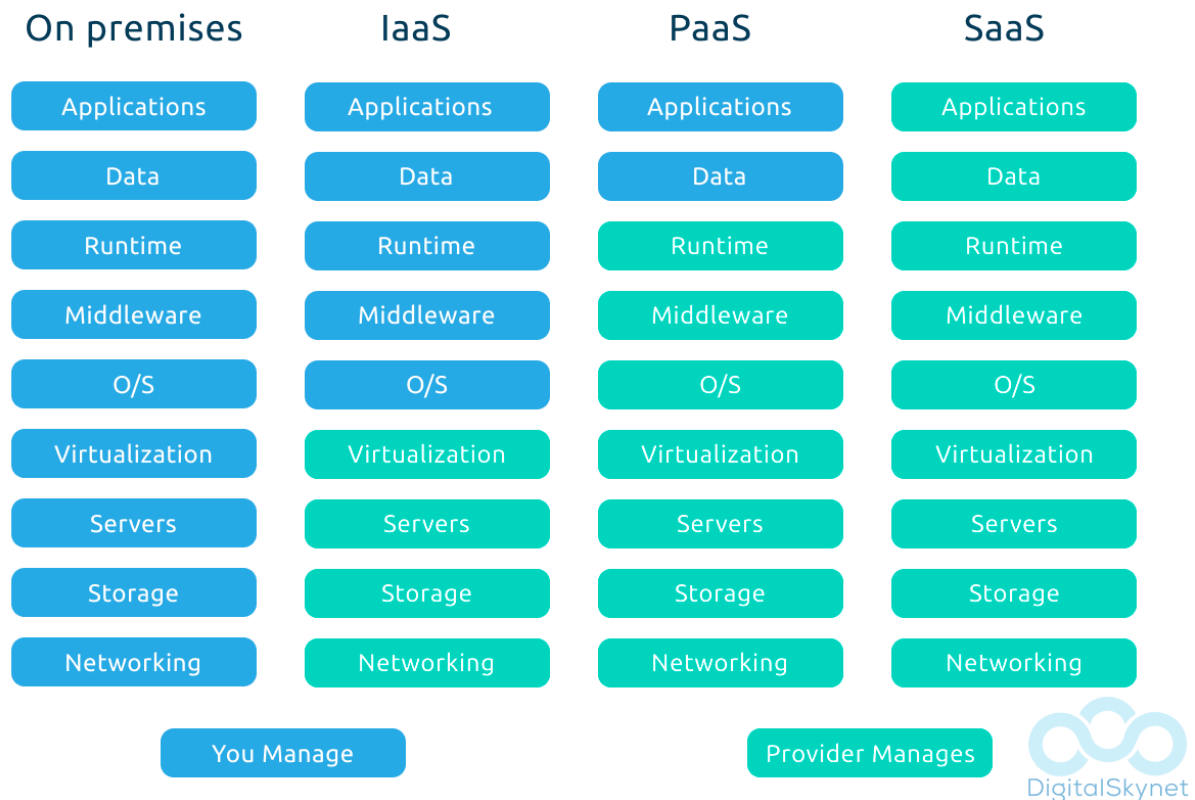
Rapid Elasticity

- Capabilities can be rapidly, elastically and in some cases automatically provisioned to quickly scale out and rapidly released to quickly scale in.
- That is, it should allow enterprises to SPIN-OUT and SPIN-DOWN servers rapidly on-demand.
- To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
- The term "rapid" is used as the user should never feel that there is a shortage of resources, i.e. an illusion of infinite, dedicated computing resources should be maintained.

Measured Service

- Cloud systems automatically control and optimize resource usage by leveraging a metering capability at some level of abstraction appropriate to the type of service.
- In simple words, access to the cloud should not be free and there should be a metering capability to measure services used by a user according to which billing must be done.
- Resource (CPU, disk, memory, network) usage can be monitored, controlled and reported providing transparency for both the provider and the consumer of the service.

Cloud Service Models



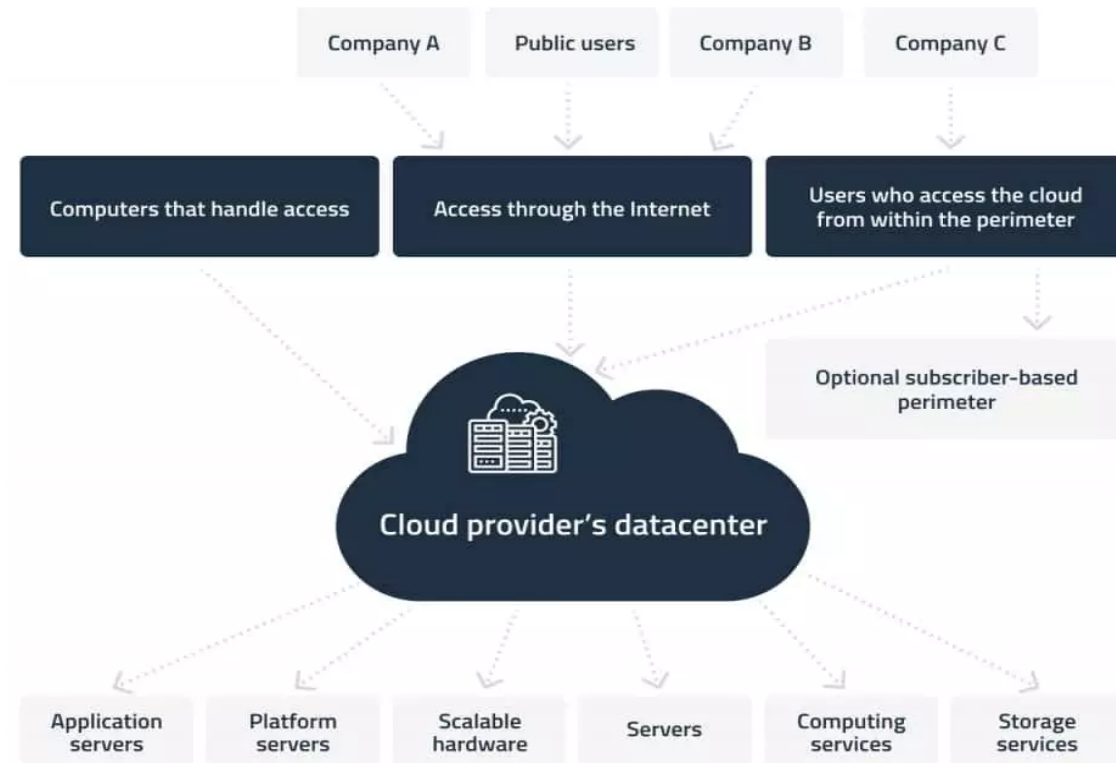
This should basically tell you everything there is to know about the Cloud Service model. In the case of On-Prem the cloud is hosted in your premises itself, so there is no role of a cloud provider as you have to manage everything.

13/01/22

Cloud Deployment Models

1. Public Cloud

The cloud infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.

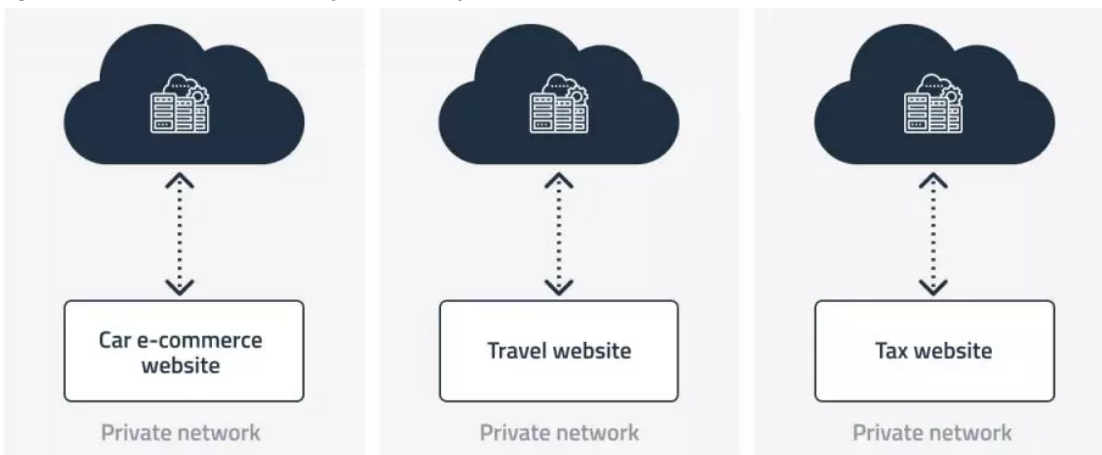


Advantages:

- Minimal investment
- No setup cost
- Infrastructure management is not required
- No maintenance
- Dynamic scalability

2. Private Cloud

The cloud infrastructure is operated solely for a single organization. It may be managed by the organization or a third party, and may exist on-premises or off-premises.

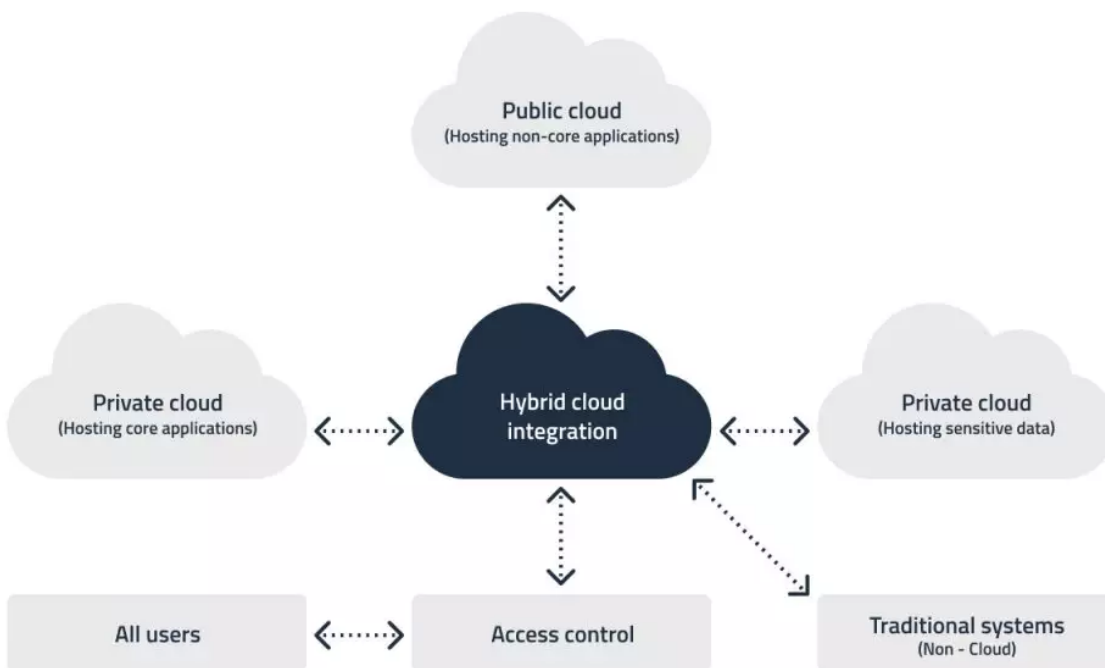


Advantages:

- Better control
- Data security & privacy
- Support legacy systems (supporting some legacy softwares on public cloud can be cumbersome or undesired)
- Customization

3. Hybrid Cloud

The cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain as unique entities but are bound together by standardized or proprietary technology that enables data and application portabilities (e.g., cloud bursting for load-balancing between clouds).

**Advantages:**

- Flexibility & control
- Cost effective
- Better security

4. Community Cloud

The cloud infrastructure is shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, policy, or compliance considerations).



Advantages:

- Cost effective
- Security
- Shared resources
- Collaboration and data sharing

Module 2: Distributed File System Principles

Terminologies associated with File Systems

File Service

- A specification of what the file system offers to its clients

File server

- An implementation of a file service
- Typically, run on one or more systems [more than one system = distributed system]

File access types

1. **Explicit access**

- Client initiates a connection and accesses remote resources by their host name and file location.
- Typical examples: FTP, SSH, Telnet
- Used in the early days of UNIX, doesn't have any particularly special setup required..
- Horribly inflexible and slow and something better is required.

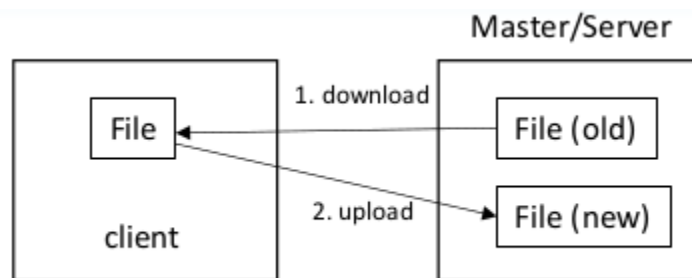
2. Transparent access

- Client accesses remote resources as if they were local resources.
- So the client doesn't need to know where the server/file/resources are located which is why this access type is called transparent access.
- Feature of a true distributed file system and it is the distributed file system that acts as a middleware for helping the client get access to the file that they were searching for which may or may not be remote (transparency babyee :P).

File access models

1. Upload/download model

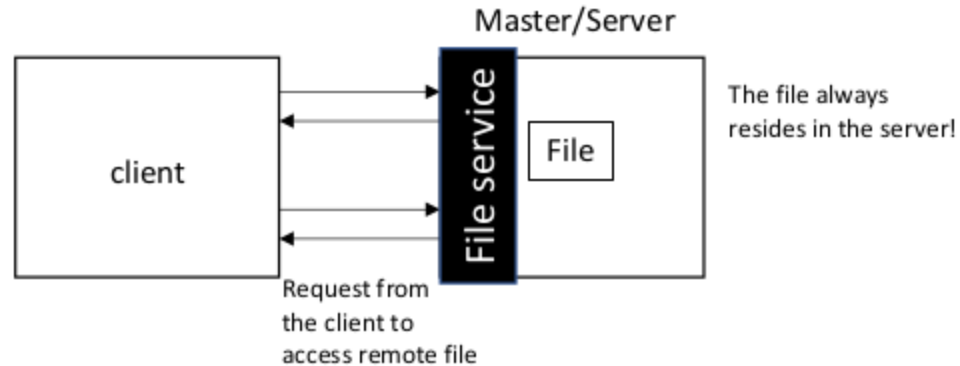
- Download: copy the file from server to client so that client can read/modify it.
- Upload: copy the file from client to server in order to update the server's copy of the file to the latest version.



- Issues with this model,
 - What if the client does not want to modify the whole file?
 - What if the client does not have enough space?
 - Sometimes the entire file to be modified can be very huge but only small modifications are required to be done by the client.
 - In such cases, such a model can be very tedious and inflexible to deal with.
 - What if other clients want to access the same file?

2. Remote access model

- File service provides an API to create(), read(), update(), delete() (CRUD) files or bytes.
- Same API one would have in a centralized file system.



- So the client fires API calls to perform CRUD operations on files which are located on the server itself and using this mechanism the server can manage the file system better.
- Advantages:
 - Client gets only what's needed
 - Server can manage coherent view of file system
- Issues
 - Possible server and network congestion
 - Servers are accessed for duration of file access
 - Same data may be requested repeatedly
 - We can try to handle this by having two servers that serve the same purpose and store the same data, so clients can now appropriately make requests to one of the two servers and availability/fault tolerance is higher because data is duplicated on both servers.
 - But this introduces its own set of issues like load balancing, servers should have consistent data, etc.
 - State in server?
 - This is talking about the previous point in that the different servers introduced in the system should share the same state.

CAP Theorem

CAP stands for Consistency, Availability and Partition tolerance.

Largely a conjecture attributed to Eric Brewer (UC Berkeley) and later formally proven by Gilbert/Lynch (MIT):

A distributed system can satisfy any two of these guarantees at the same time, but not all three.

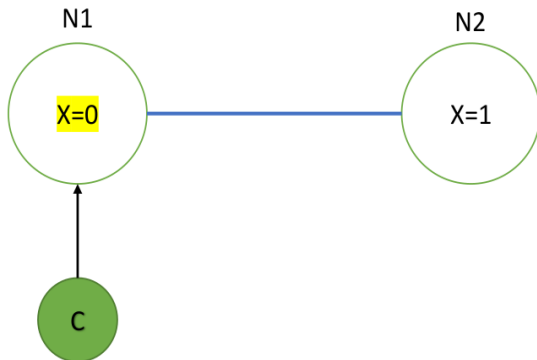
Consistency

All writes are atomic and subsequent requests (reads) are able to see the new value.

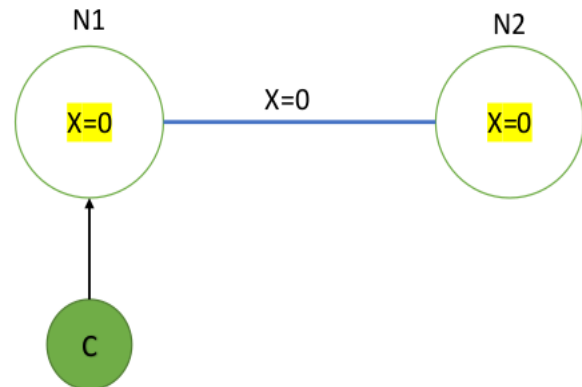
All nodes see the same data at any time, or return the latest written value by any client.

Consider the following scenario with two Nodes N1 and N2,

i) Both N1 and N2 had the value of $X = 1$, but then a client updated the value of X at N1 to 0, so now the system is inconsistent and the updated X value must be propagated from N1 to N2.



i) As described above



ii) Back to a consistent system

Availability

The distributed system is always available (and returns a value) as long as even a single node is running.

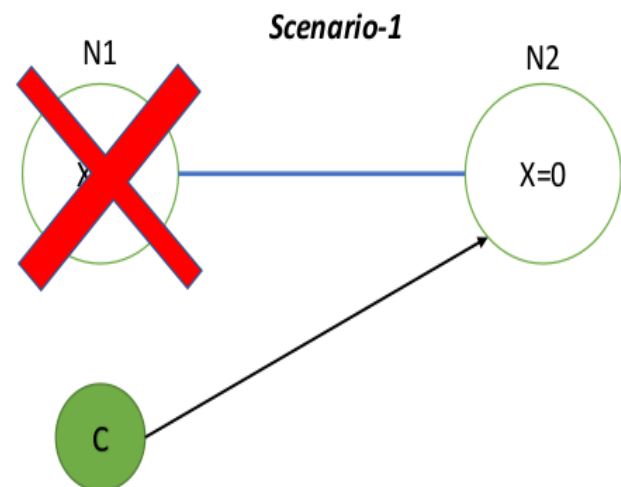
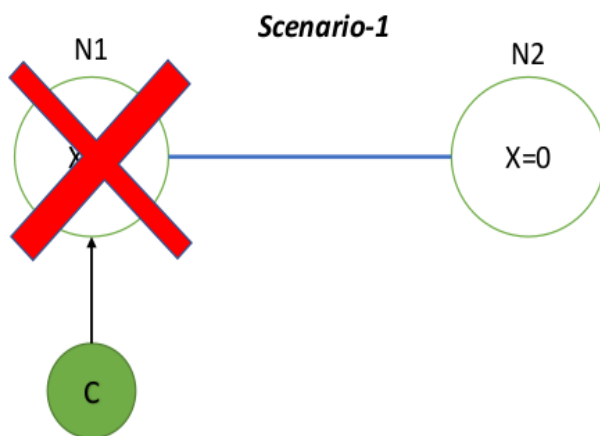
Partition tolerance

Single node (and link) failure does not prevent the whole system from operation.

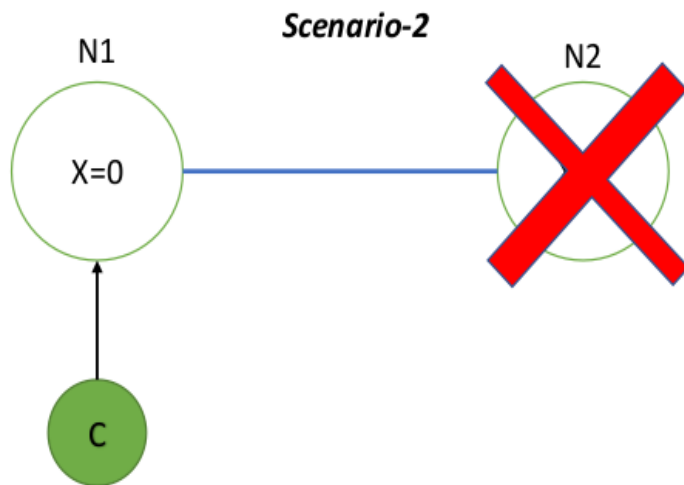
Consider the following scenarios which can disrupt the CAP of a distributed system,

i) Node N1 goes down with client C connected

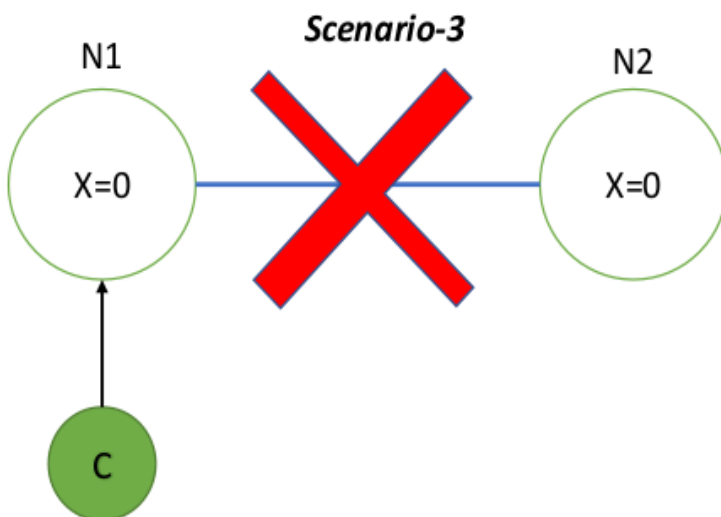
To restore availability, client C connects to the system via N2 which is still up.



ii) Node N2 which is not connected to any client right now goes down. No change is required in the system for availability to be restored as no clients were involved.



iii) The link between N1 and N2 goes down or is too slow to be considered operational, i.e. a network problem.



This can disrupt consistency and partition tolerance

This situation although undesirable is quite a common occurrence in the Internet world especially with large-scale (worldwide) systems like that of Google, Amazon, etc. so there will end up being partitions like these which work in isolation.

Now suppose C wants to access the value of X from N1, but N1 has no idea whether N2 or any other node for that matter has modified its value, i.e. consistency is in question.

So the system can be designed such that,

i) Because the system must always be available, a node will return whatever value it has been requested for without the guarantee of that value being consistent.

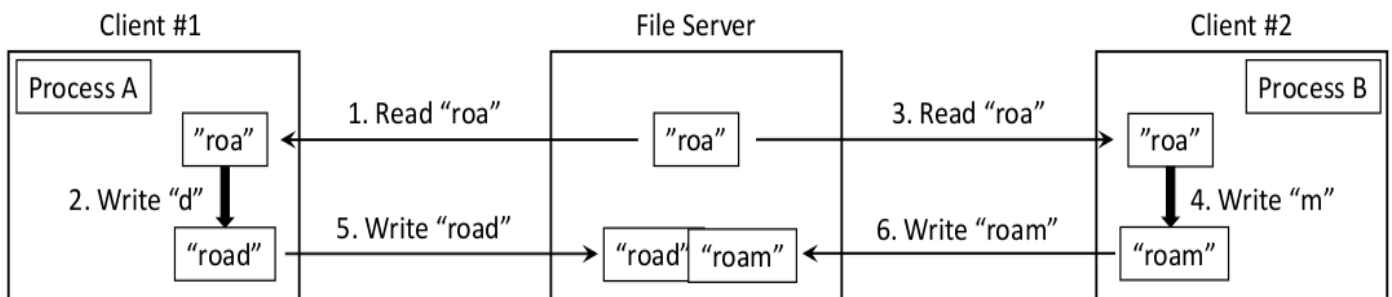
ii) Because the system should always be consistent, the node will not return any value in such a scenario because it is not sure of the consistency of the requested data which hampers availability.

So you cannot have a solution which guarantees both Consistency and Availability when Partition Tolerance is in the picture and that is what the CAP Theorem in essence is all about.

Distributed File Sharing Semantics

Session Semantics

- File changes are only visible to the client modifying it (e.g., in its local cache).
- Last process to modify (i.e., close) the file wins.
- Simple and efficient (but not transactional).



- Only when the file is closed at client-side is it written back to the file server.
- So after writing back to the server, if a client wants access to the same file again, then it must read the file from the File Server again.
- And in this example, Client #2 closed the file last, so its update overwrites the update done by Client #1, i.e., file server stores the value of the file as "roam".

Transaction Semantics

Fully ACID-compatible file accesses and modifications.

- **Atomicity:** Either all modifications inside a transaction succeed, or they all fail.
- **Consistency:** A transaction transforms the data(base) from one consistent state to another.
- **Isolation:** Each transaction is executed as if it were the only process accessing the data. Changes occurring in a transaction will not be visible to any other transaction until the transaction is "committed" – intermediate results are not updated!

- **Durability:** When a transaction finishes ("commits"), the results of the transaction are persistently stored and made available to other processes. Persistently stored means that the system can recover from failure.

You already know how to explain these with the help of bank transactions between two bank accounts.

Transaction semantics is considerably the most demanding semantics for a distributed file system. It is also relevant in databases, transactional main-memory, etc.

Transactions require a single master (or multiple synchronized masters) that coordinates client accesses to all resources.

All of the common distributed file system architectures basically give up on implementing clean transaction semantics!

Because of the difficulties associated with Transaction Semantics we typically go for Session Semantics while developing a system unless it specifically requires Transaction Semantics.

For Centralized Database Systems: Locking

Multiple clients may read from and write to the same table concurrently.

Consider the following scenario with 3 clients,

Client1:	-> Read table t	-> Write table t
Client2:		
	-> Read table t	-> Write table t
Client3:		-> Read table t ->?

In this scenario, Clients 1 and 2 both try to write into table t at the same time, which write should be considered as the actual write and which one should be ignored or should they both be considered somehow?

This is where locking would help, when Client 1 requests access to table t with the intention of writing into it, then Client 1 should be given Exclusive Access lock to table t and now Client 2 will not be able to get access to table t and so first the write of Client 1 will be written and there is no confusion as to which write should be considered.

Transaction: Coherent sequence of read/write/update operations of each client.

Locking allows for the safe execution of transactions.

Lock a resource (i.e., files or tables) at first client request then keep exclusive-access lock until client finishes.

But locking and consistency is achieved at the cost of concurrency!

Stateful vs Stateless Servers

Stateful: Server maintains client-specific state

- Shorter requests & better performance in processing requests.
- Cache coherence is possible since the master knows who is accessing what.
- Global file locking (and transactions) possible.

Stateless: Server maintains no information about client accesses

- Each request identifies files and offsets.
- Server can crash and recover: no state to lose
- Clients can crash and recover (as usual).
- No server space used for state information.
- But what if a file is deleted on master while a client is working on it?
- Global file locking (and transactions) not possible.

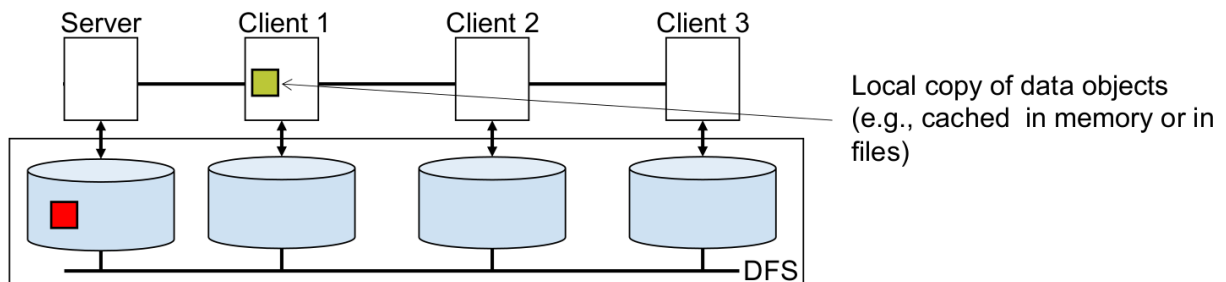
For implementing Transaction Semantics, the server has to be Stateful.

For implementing Transaction Semantics, the server can be Stateful or Stateless.

18/01/22

Caching

- Reduce latency to improve performance for repeated file accesses.
- Possible cache locations: server's disk, server's memory, client's disk, client's memory.
- Caching at the client can create cache-consistency problems which is unfortunate, since they are the best performing options (you can imagine why their performance is high).
- Cache validation is typically done by checking time-stamp, implementing checksums, etc.

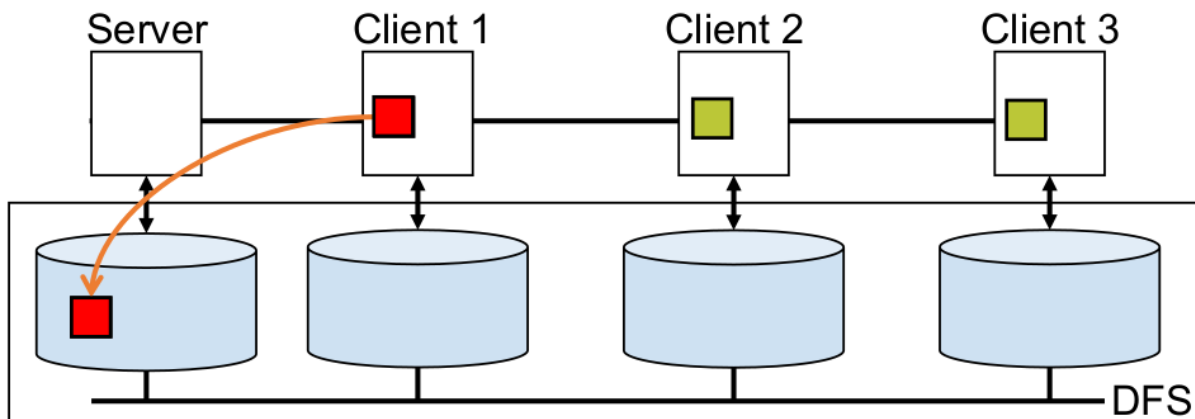


[DFS - Distributed File System]

Techniques for Caching

Write-through Caching

- Every change is immediately propagated to the master-copy.
- What if another client reads its own (out-of-date) cached copy?
- All accesses will require checking with the server.
- Or the server maintains state and sends invalidation signals.
- This improves read performance but does not improve on write performance (in fact, it remains the same as before).



- Every write being written to the server also increases network traffic as every little change done by a client has to be written onto the server.
- Say for example, the user writes 20 characters in a file (each of these must be written to the server) and then undos each one to reach an empty file and for doing this there were a total of 40 requests made to the server to update its copy of the file. You can see how this is so redundant and adds so much unnecessary network traffic.
- To avoid this we go to the second option which is...

Write-behind Caching

- Delay the writes and batch them.
- Remote files (server's copies) are updated periodically (period can be set as per use case).
- One bulk write is more efficient than lots of little writes.
- Problem: ambiguous semantics in case of conflicts!
- Example:
 - Client 1 reads file f and Client 2 reads and writes file f
 - Client 1 writes file f.
- Unfortunately, this is neither transaction semantics nor session semantics.

You can see why this is not transaction semantics anyways but for session semantics there is a doubt.

This doesn't follow session semantics through and through because in session semantics, a "session" was said to be completed only when the client closed the file meaning there'll be no new updates and all the updates done by the client will be written to the server as one update and now if client wants to make any more updates it has to fetch the file from the server. However in this case, when a periodic update is made at server-side, the client hasn't closed the file yet, meaning it can still make more updates to the same file. Meanwhile the updates that were previously written to the server have already been overwritten by some other client, so now there is an inconsistency issue as the client is still updating on the stale copy of the file and this is why it's not session semantics [not because of the inconsistency but because of the fact that updates are sent to the server without the client having closed the file].

Write-on-Close Caching

- Write a file back to the server only once it has been closed by the client.
- Even better is to wait another X seconds to see if the file will also be deleted.
- This implements session semantics but not transaction semantics.

Read-ahead Caching

- Be proactive and "prefetch" data that is likely going to be relevant for an application in the next step.
- Request chunks of file (or the entire file) before it is needed and put it into the client cache.
- Minimize latency at the time when it actually is needed.

Summary

Centralized Control: May support transactions, but poor scalability.

Write-through Caching: Improves read performance, but does not improve write traffic/performance.

Write-behind Caching: Better read and write performance, but possibly ambiguous semantics. Neither transaction nor session.

Write-on Close: Matches session semantics, but not transaction semantics.

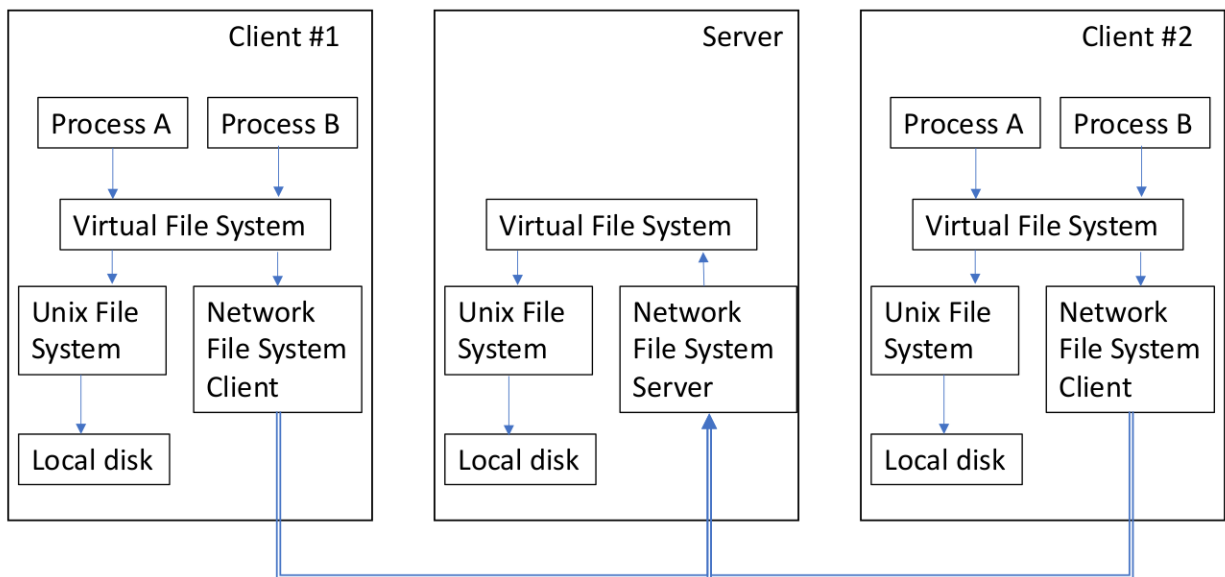
Distributed File System History

Network File System (NFS)

- Sun Microsystems (now Oracle), 1984-today
- Arguably the first distributed file system.

- Based on the simple client-server model.
- The model underlying NFS is Remote Access model => Clients are given transparent access to a file system on a server and this remote access is enabled with the help of the NFS client.
- All machines are located within the same physical network.
- Stateless.
- No transaction semantics. Not even session semantics (uses write-behind caching!).
- Optional locking mechanism for files.

Architecture



- Communication between clients and the server is done via the RPC mechanism (Remote Procedure Call) so there is an RPC component in the above diagram as well after the NFS Client.
- The RPC mechanism translates NFS operations into operations that are local to the server and vice versa, so it helps maintain heterogeneity in the OS of Client and Server without compromising anything else. Thus, NFS allows for heterogeneity.

NFS Design Goals

Heterogeneity is a first class citizen

- Different hardware and operating systems between clients and servers are not an issue.

Access transparency

- Remotely stored files accessed as local files through standard system calls (API).
- Separation of physical and logical storage.

Crash recovery

- No (shared) state whatsoever so both client and server can recover easily.

High performance

- Network I/O equal to (or faster than) disk I/O
- Caching: read-ahead, write-behind

Andrew File System (AFS)

- Carnegie Mellon University & IBM, 1986 V1, 1999 V2
- Named after Andrew Carnegie and Andrew Mellow, the “C” and “M” in CMU
- Acquired by IBM, and subsequently open-sourced.
- Still used today in some clusters (especially University Clusters)
- Two unusual design principles: whole file sharing & whole file caching. -- not in blocks!
- Based on the assumption that:
 - Most files are small (so whole file sharing and caching is not a big issue)
 - Most files are accessed by one user at a time (so consistency is also not a big issue)
 - Clients could cache as large as 100MB.
 - Reads are more common than writes.

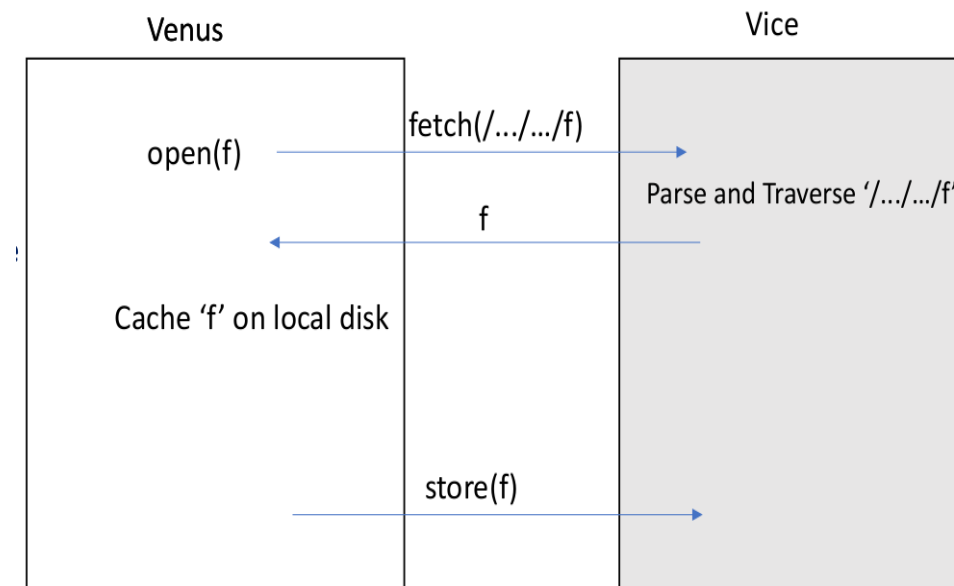
Architecture

Jargon: **Client System = Venus service & Server System = Vice service**

Reads and Writes are optimistic and are done on local copy of the file at Venus.

On closing a file, changes are propagated to Vice, i.e. Write-on-close Caching.

When a Venus opens a file, **Vice sends the entire file** and gives the Venus a **callback promise**.



The problem with this version was that the **path traversal costs** to parse the given pathname by Venus and fetch the Vice's copy of the file **were too high**.

And fetching happened quite often, so a lot of the time was being spent by the Vice in just looking up files.

So in AFS V2, all files were assigned a unique identifier and all Venuses now maintained a dictionary that mapped files to their identifiers, so some of the workload was shifted to the Venus (client-side).

Now whenever Venus requested for a file, it would send the file identifier in the request and so the Vice's job became easy, no path traversal required, just look up the file id in the dictionary and find the file's associated path.

Now another problem with this system is that “**clients issue too many testAuth protocol messages**”

A testAuth protocol message is sent by the client to the server to verify whether the cached version of the required file is the latest version or not.

And it's clear that a lot of testAuth messages would be sent by clients which would result in the servers spending a lot of its time just responding to these messages.

To avoid this, AFS V3 introduced the **Callback Promise**

It is a promise that if another client modifies and then closes a file, a callback promise will be sent from Vice to Venus.

Callback state at Venus is only binary: valid or cancelled.

Valid means that the callback is still active and Cancelled means a cancelled callback.

So the server actually informs a client whether a file that it has already opened has been modified by some other client or not which would mean that the client would have to fetch the new file copy in order to update its cache.

Now this line should make sense,

“When a Venus opens a file, **Vice sends the entire file** and gives the Venus a **callback promise**.”

This is how a particular client and file is registered with the Vice such that now whenever this file is modified by another client, the first client will be sent the callback promise in order to alert it.

Coda File System (not in this lecture!)

- Carnegie Mellon University, 2003-2009

Till now none of the DFS focused on fault tolerance and dealing with failure as such, they all used servers which are quite expensive instead of commodity hardware which is cheaper.

Google File System (GFS)

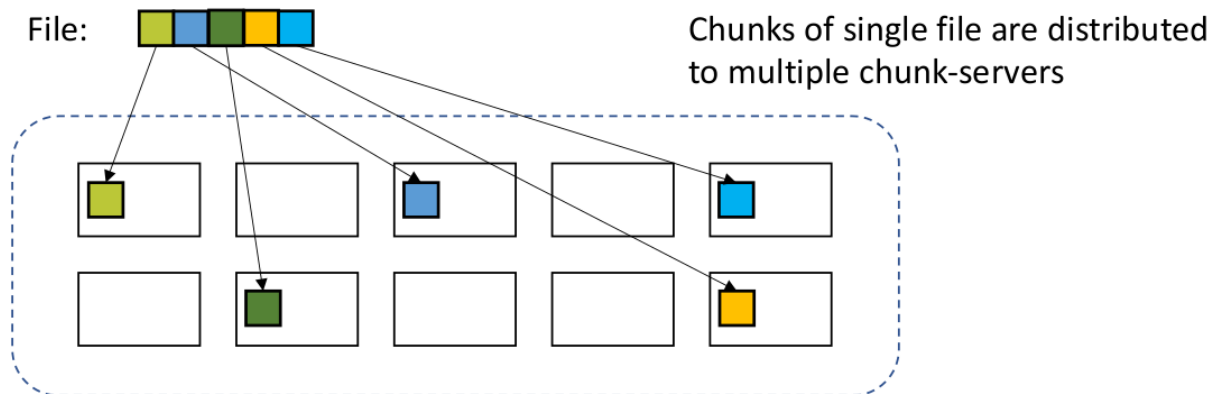
- Google Inc., since 2003
 - GFS is a Distributed File Storage utilising commodity systems.
 - GFS is used as a basis to design Hadoop.
 - Corresponding to GFS, the component in hadoop is called HDFS.
 - GFS consists of clusters which are composed of 100s of 1000s of commodity systems.
- [Note that from here on out we are no longer worried about session and/or transaction semantics as they do not play a big role here]

Design Considerations-1:

- Use commodity hardware rather than buying expensive servers.
- Scale-out with the right software.
- But commodity hardware fails all the time: disk failure, network issues or even OS bugs.
- Design Challenge: Perform reliably in a fault-tolerant manner in the face of constant failures

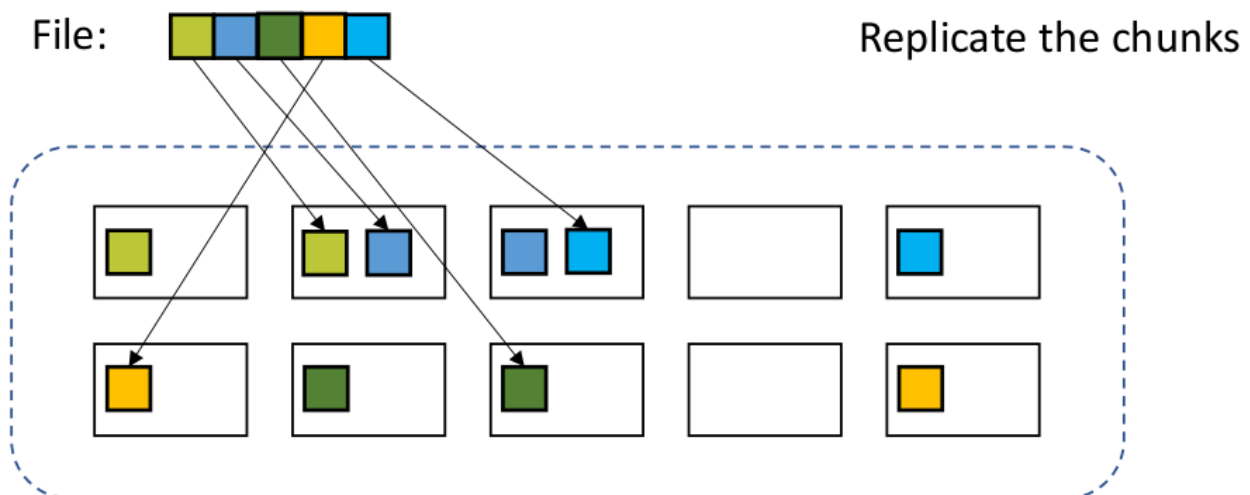
Design Consideration-2:

- Optimised to store and read large files: 100MB to multi-GB files.
- Files are split into chunks.
- Each chunk is of 64MB, identified by 64 bit ID and stored in Chunk-Servers.



Each commodity hardware acts as a Chunk-Server

Replicate the chunks since chunk-servers can fail any time.



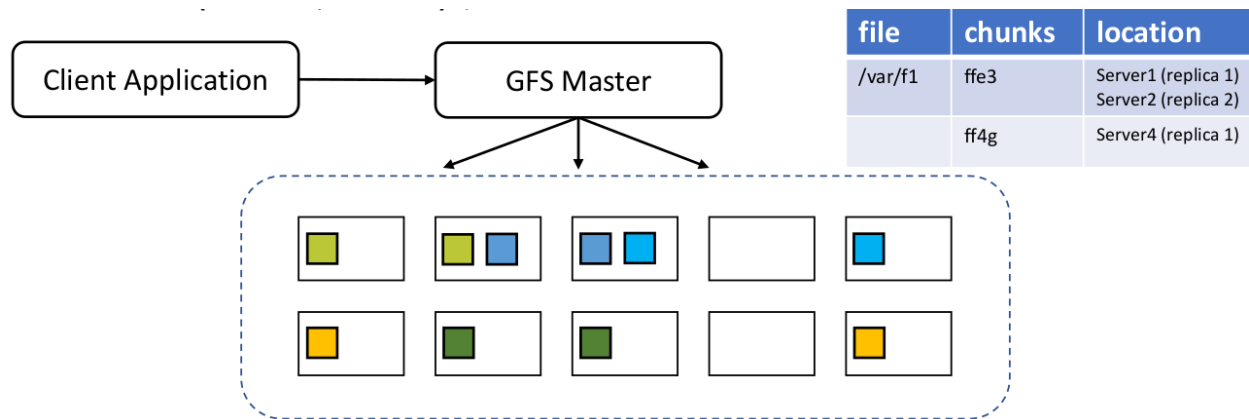
Replication is a very important concept!

By default each chunk would be replicated thrice.

How would we know which chunk of a file resides in which chunk-server?

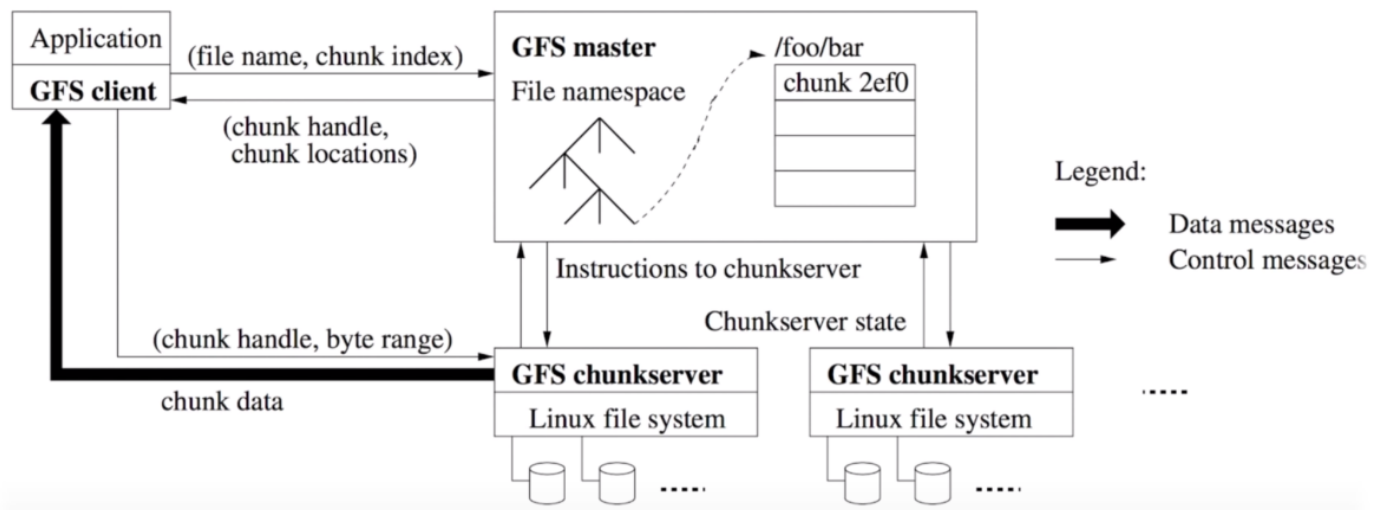
GFS Master: Stores the entire metadata of the cluster.

Metadata: Mapping from filenames -> (chunk-id, location);



GFS Read Operation

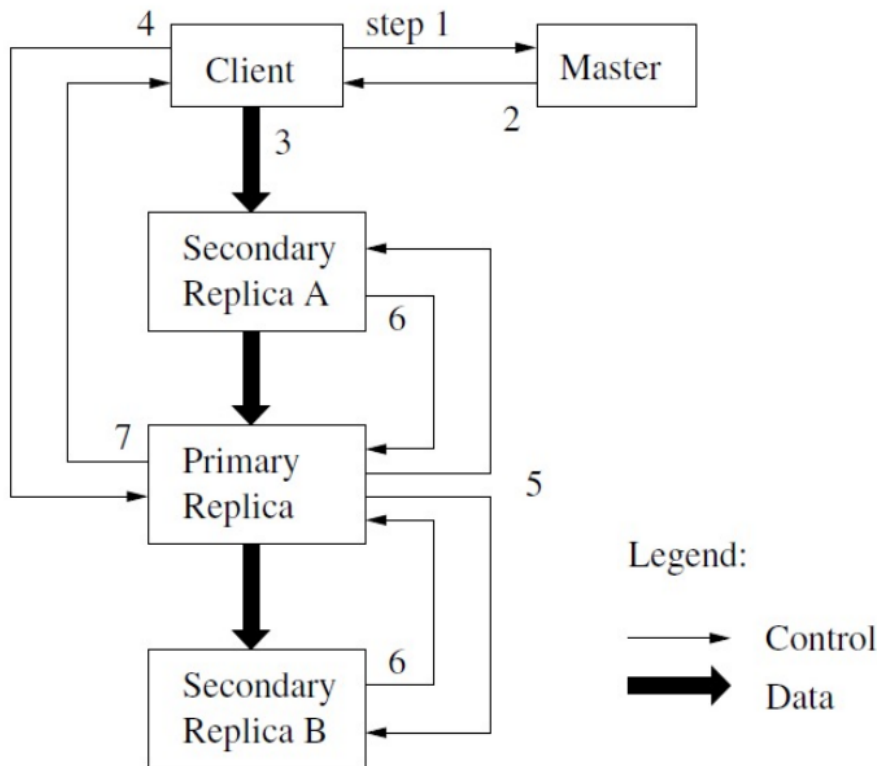
- GFS client requests to access the file using filename and chunk-index.
- GFS master gives the id of the chunk and all the IP addresses of chunk-servers having it's replica.
- Client uses the IP address and reads the chunk directly from the chunk-server
- **Note that Client does not read the data via GFS master!**
- Client caches the chunk handle and chunk location



GFS Write Operation

- GFS client asks the master for the location to write.
- GFS master gives 3 free chunk locations (in 3 different chunk-servers), i.e. Replicas A, B & C
- Client writes data to the closest chunk-server.
- Subsequently the data is propagated to the 2 other chunk-servers.
- Client sends a request to the primary replica to commit the data to disk.
- Primary replica coordinates with the other replicas and sends confirmation to the client.

Note: Primary replica need not be the same as the Closest replica, it is any one of the 3 replicas.



GFS Consistency Model

- Chunk-servers send regular "heartbeat" messages to master to inform that they are alive
- If the chunk-server is down, then the master ensures all chunks that were on it are copied on other servers.
- Ensures replica counts remain the same.

What if the single master fails?

- Create a new master and read the **operations-log**.
- Operations-log: append only log.
- **Stores all the file operations, timestamp, user details.**
- **Directly written to the disk and replicated remotely.**
- New master generates the entire file system namespace and chunk-ids, from the operations-log.

However recovering from a master failure with the help of the Operations log can be very tedious and time consuming which is why you can also have a Shadow master which does the same operations as that of the master and stores the same Ops log. So essentially **Shadow master is a replica of the master**. So whenever the master goes down, it can be restored using the Shadow master itself and the system never actually goes down!

Hadoop File System (HDFS)

- Shipped with Apache Hadoop since 2009, major split into HDFS and YARN in late 2012 (Hadoop 2.0), currently split into HDFS, MapReduce and YARN packages (Hadoop 3.1)
- Part of Apache's Hadoop framework (which is essentially an open-source Java implementation of Google's MapReduce)
- Generally follows the GFS architecture.
- Basis for many other open-source Apache toolkits like,
 - Yahoo's PIG/PNuts (file-based data storage and scripting language)
 - Apache HIVE (distributed data warehouse)
 - Apache HBase (distributed database)
 - Apache Cassandra (fault-tolerant distributed database)
 - ...

HDFS Design Goals

- Support for very large files (>1 TB), even Petabytes of overall data.
(See "Scaling Hadoop to 4000 nodes at Yahoo!", Yahoo! 2008 blog entry)
 - Streaming data access (write-once, read-many-times pattern)
 - Commodity hardware (many distributed and less expensive machines, frequent failures)
 - No low-latency file access (below tens of milliseconds) as HDFS is designed for delivering high throughput of data.
 - Not too many small files (millions of files are OK, but not billions!) as the number of files in HDFS is limited to the amount of memory in the master node.
 - Single writer to a file at any time, append-mode only
 - In principle HDFS allows for session and even transaction semantics, but no concurrency for mixed read/write or write/write operations at all!
 - Blocks are the same as "chunks".
 - Large files are broken down into blocks of 64-128 MB (again compare it to 64 MB GFS blocks and 16-32 KB DBMS blocks).
 - Minimise the amortised cost of seek operations: larger blocks of 128MB size guarantee consecutive layout of data on disk, disk seek times (~3-5ms) occur only across block boundaries.
 - Namenode is the same as the GFS master.
 - Manages the entire namespace of files and blocks, i.e. metadata.
 - Datanodes are the same as the chunk-servers in GFS.
- [Replication factor of HDFS is also 3, same as GFS]

HDFS Interfaces

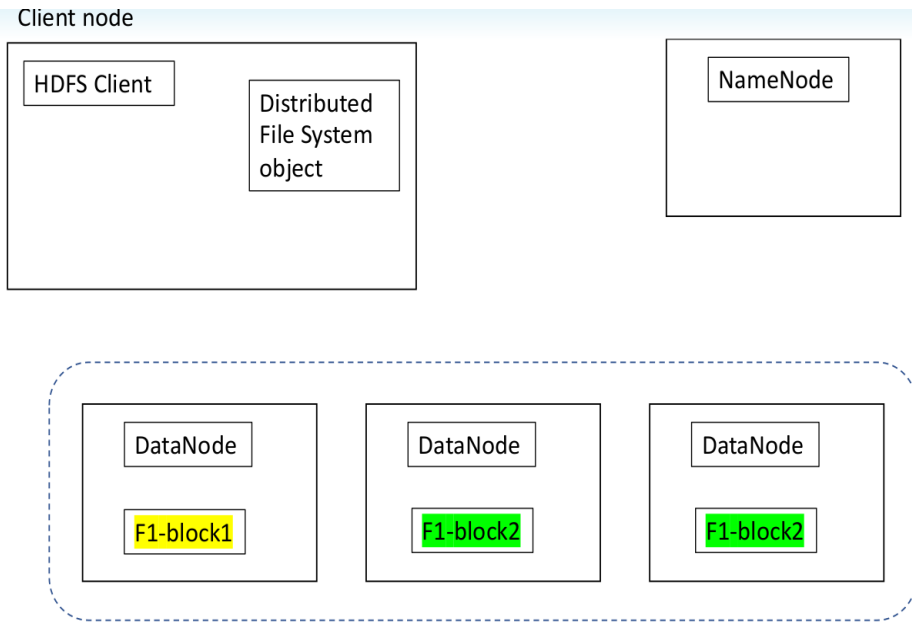
Hadoop provides many interfaces to allow users to access and manipulate its file system.

- Command-line (from a Linux/UNIX shell)

- Java API
- Thrift proxy (supporting C++, Perl, PHP, Python, Ruby)
- C via a Java Native Interface (JNI)
- WebDAV for mounting file systems via HTTP
- HTTP (read-only), FTP (read & write)
- Etc.

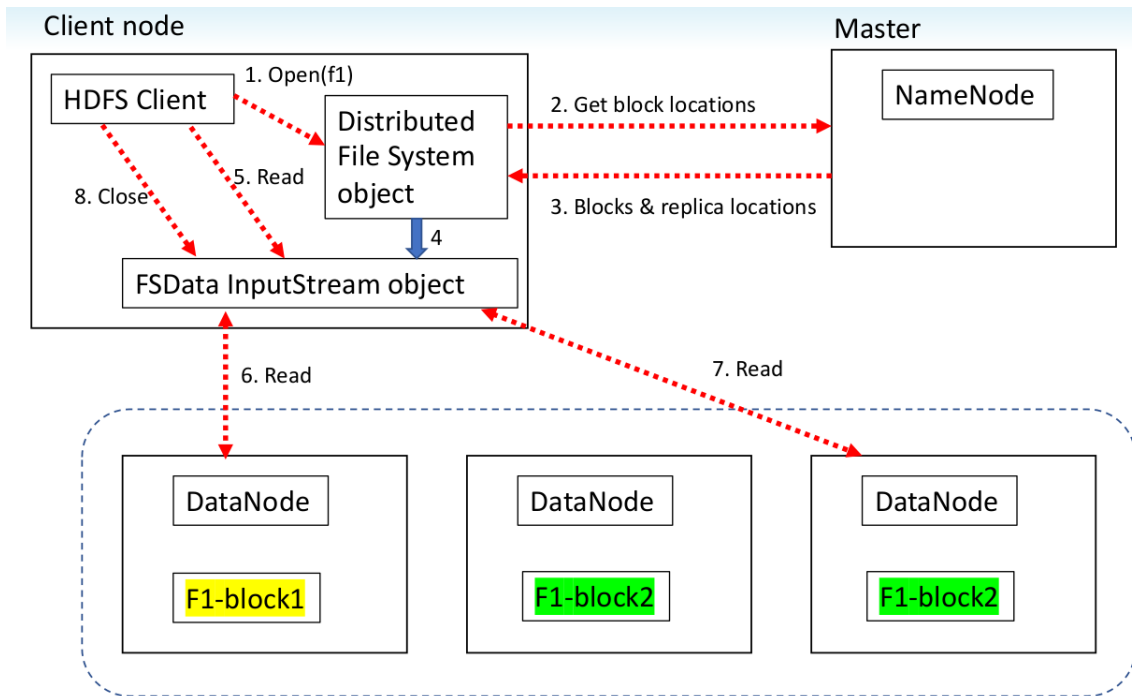
HDFS Architecture

Same as GFS

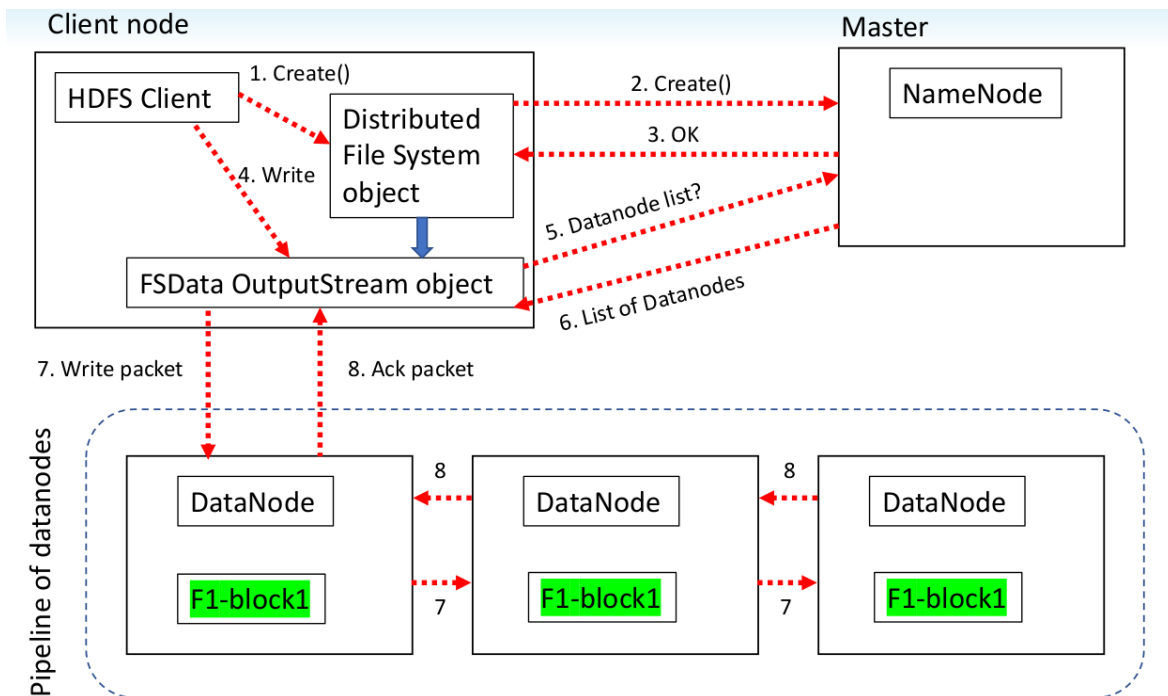


Both read and write operations are the same as GFS, just some terms are changed here and there.

HDFS Read Operation



HDFS Write Operation

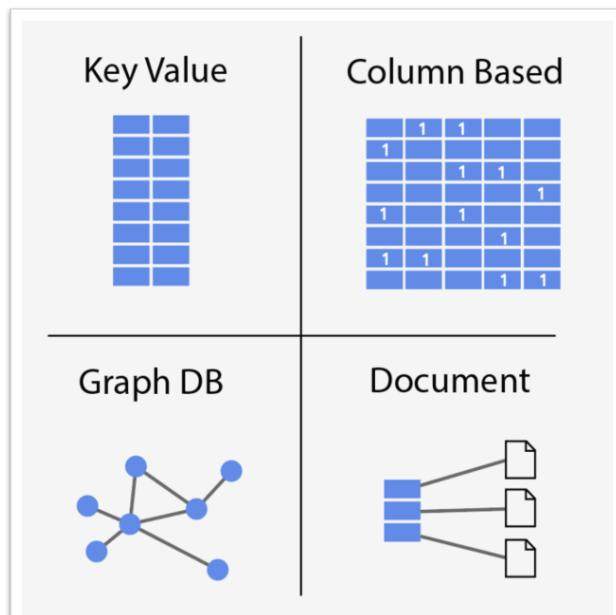


The primary replicated block is responsible for propagating the data to create two more replicas exactly as in GFS.

Google's BigTable

- What if we want to store and query structured data instead of just plain files?
- OSDI paper in 2006: **Bigtable: A Distributed Storage System for Structured Data**
- Internal applications that use Bigtable: Crawls, Analytics, Google Base, Google Earth, Maps, Orkut (now Google+), etc.
- Well, we need some sort of an index!

NoSQL Databases



[No need to learn NoSQL and stuff for exams] :)

NoSQL = Not Only SQL

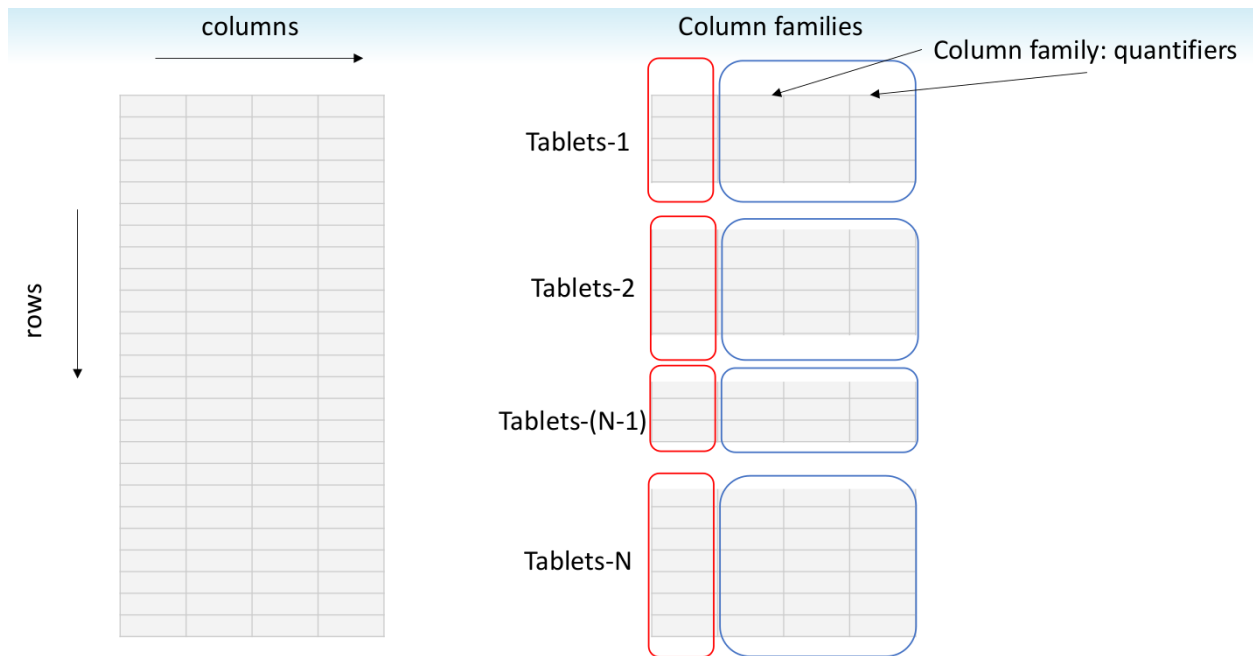
NoSQL can also store tabular data like relational tables but it can also store in so many more formats.

20/01/22

A BigTable is a sparse, distributed, persistent multidimensional sorted map. The map is indexed by a row key, column key and a timestamp, each value in the map is an uninterpreted array of bytes.

(row:string, column: string, time:int64)->string [Key -> Value]

A BigTable is first sorted by its row key and according to that the table is dynamically partitioned into contiguous row key ranges to obtain **smaller sets of the table called Tablets**.



These tablets are stored on different servers so it's a completely distributed database.

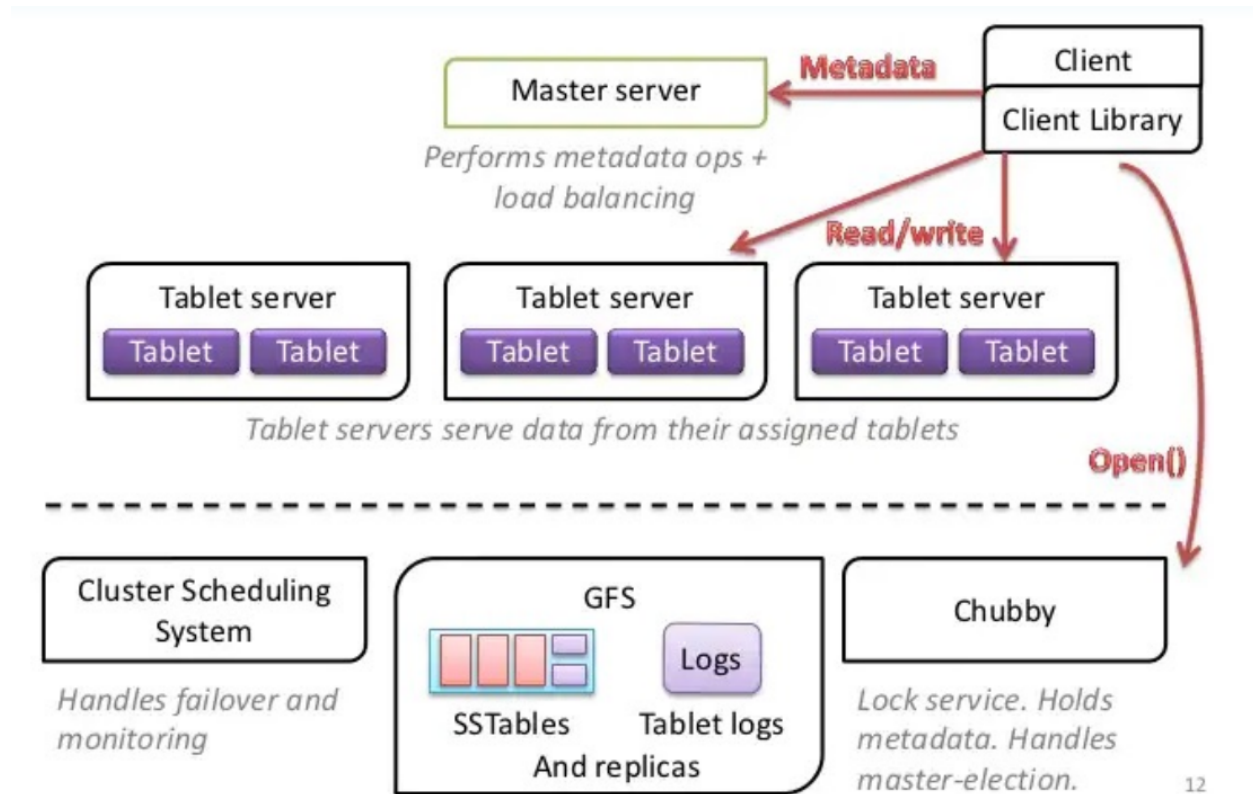
Each tablet is internally divided into column families and each column family consists of a set of columns.

Each column in a column family is a quantifier that can be used to access a data entry.

So first you use the row id (row, column, timestamp) to identify which tablet your requested data belongs to, then you use the column family along with the quantifier to access the specified data.

Architecture

BigTable is built on top of GFS and you can see that in effect in its architecture as well.



The data stored in the BigTable is actually stored in the GFS in structures called SSTables. They're highly indexed, immutable tables that provide access to data (immutable, so you can't modify written data, can only append to it). The high degree of indexing is what separates an SSTable from a Chunk in a regular GFS, this allows quick access to particular parts of data as per the indexing query.

So the Tablet servers don't actually store the data in the BigTable, instead they store mapping information in tables which maps the Tablet's data to the corresponding SSTable/s (can require more than one SSTable for one Tablet). That's why Tablet servers are typically lightweight processes as they don't really do much.

Data Storage Format

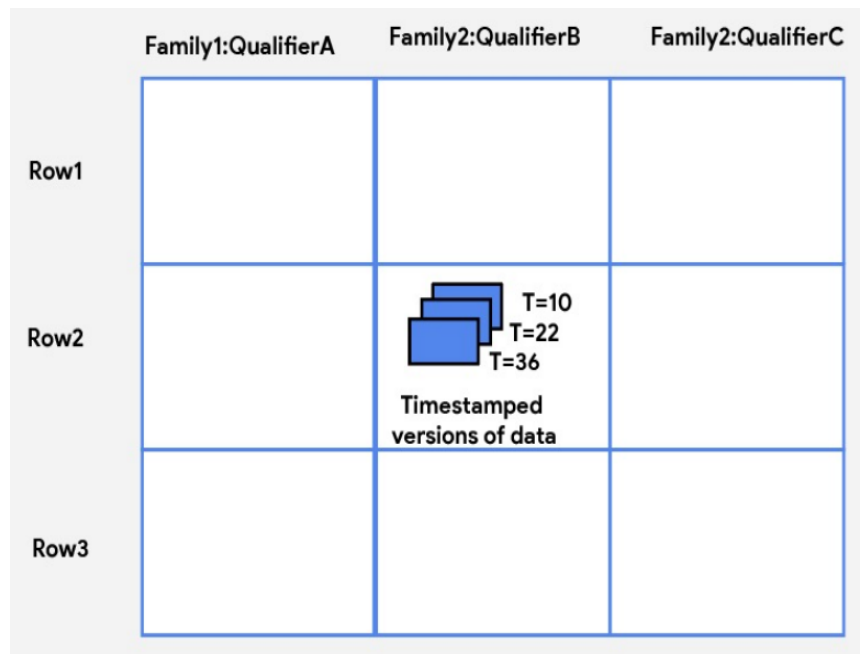
- Bigtable stores structured data in a simplified data model in comparison to a classical relational DBMS/data warehouse (no fixed schema, normal forms, etc.).
- Data is indexed using (uninterpreted) strings as row and column names.
- Data values are also stored as (uninterpreted) strings.
- That is, a table in BigTable is nothing else but a sparse, distributed, persistent, multi-dimensional, sorted map.

	Column-Family-1		Column-Family-2	
Row Key	Column-Qualifier-1	Column-Qualifier-2	Column-Qualifier-1	Column-Qualifier-2
r1	r1, cf1:cq1	r1, cf1:cq2	r1, cf2:cq1	r1, cf2:cq2
r2	r2, cf1:cq1	r2, cf1:cq2	r2, cf2:cq1	r2, cf2:cq2

So as mentioned before, to obtain a particular cell's value in a BigTable, you should know its Row Key, Column Family name and Column Quantifier name.

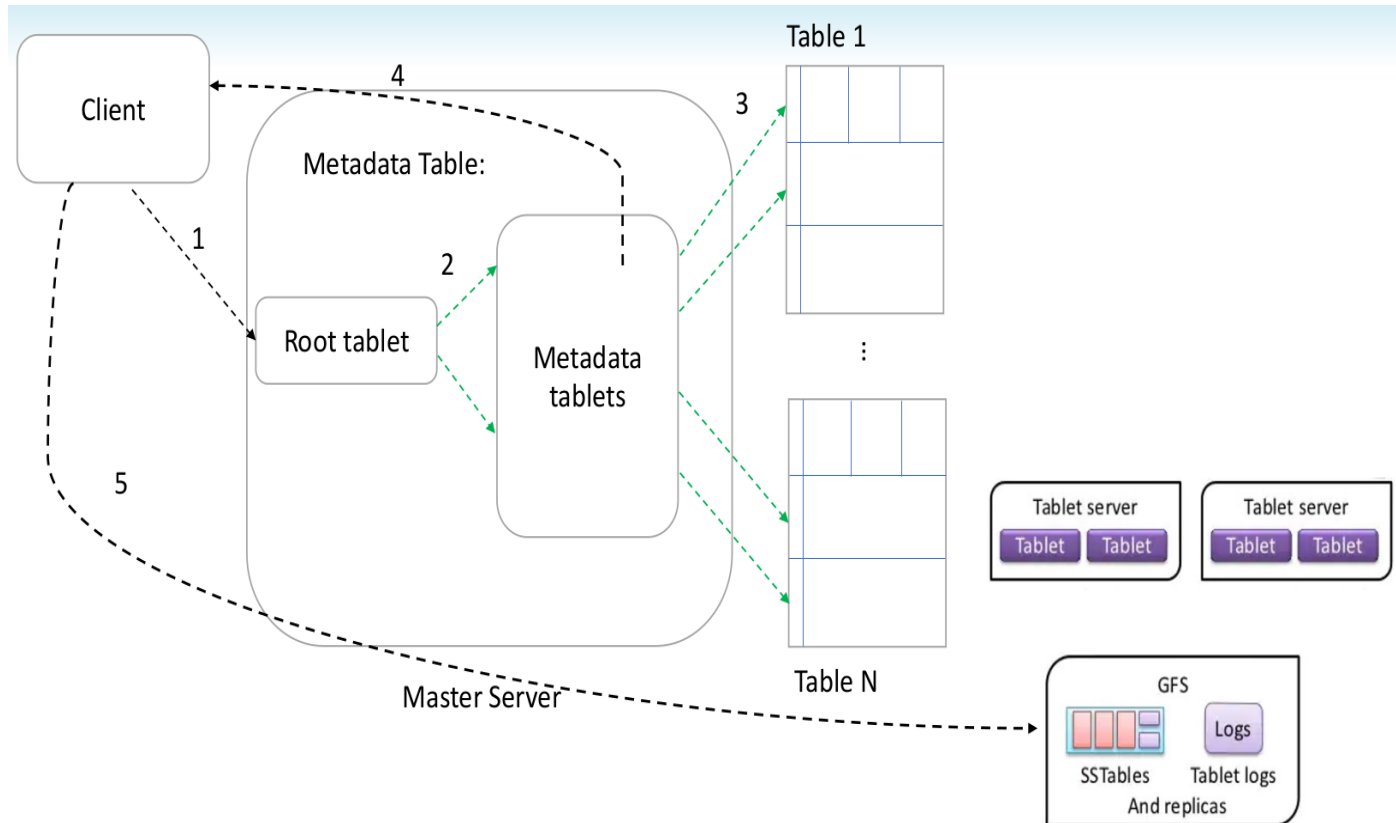
Cell Versions

BigTable allows you store multiple versions of a data element in the same cell which are identified uniquely by their timestamps (which is why timestamp is also part of the key to access a particular cell in a BigTable).



(row:string, column: string, time:int64)->string [Key -> Value] (Sorted Map)

Accessing Tablet Server



[Type: Table 1 ... Table N is actually Tablet 1 ... Tablet N]

The Master Server maintains the Sorted Map structure to index into a BigTable which will be a huge structure itself. If the index structure becomes too large then it itself can be considered as a BigTable and we can maintain an index to the index (like indirect addressing in OS). So the Root Tablet is the entry point to a BigTable that will store a reference to the Metadata Tablet which will give you indexing to the Tablet servers and each Tablet server will give you indexing into its corresponding SSTable which is stored in the GFS and actually stores the BigTable data.

Module 3: MapReduce

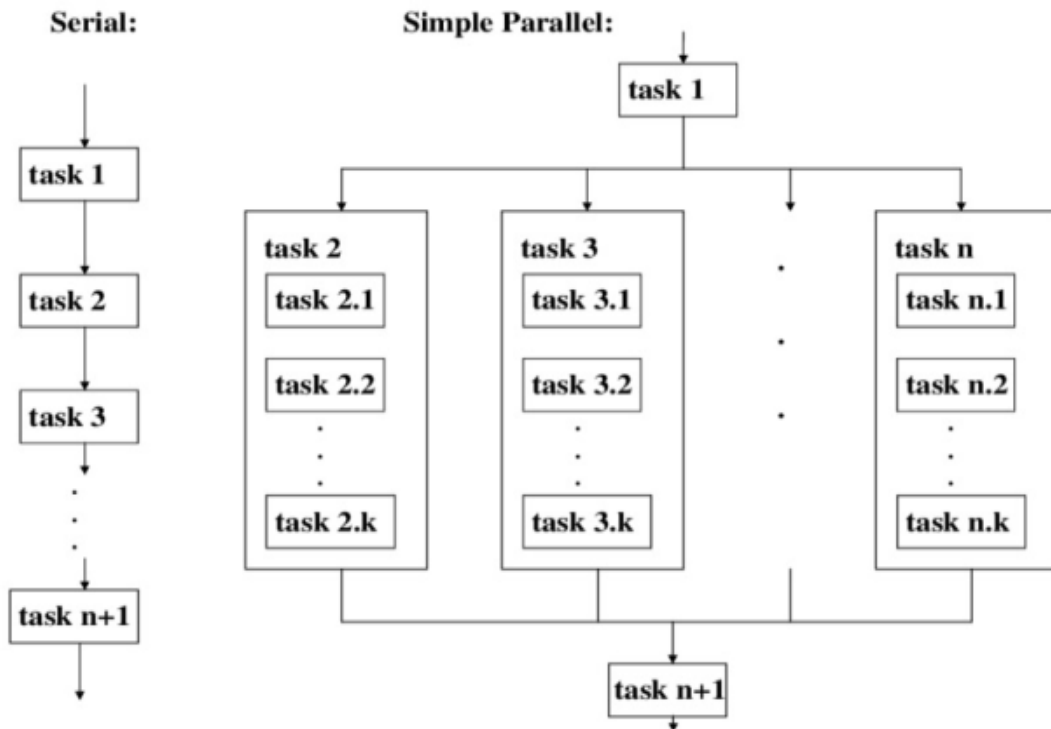
Parallel Processing

Traditional programming is serial.

However, parallel programming breaks processing into parts that can be executed concurrently on multiple processors

Challenge: Identify tasks that can run concurrently and/or groups of data that can be processed concurrently.

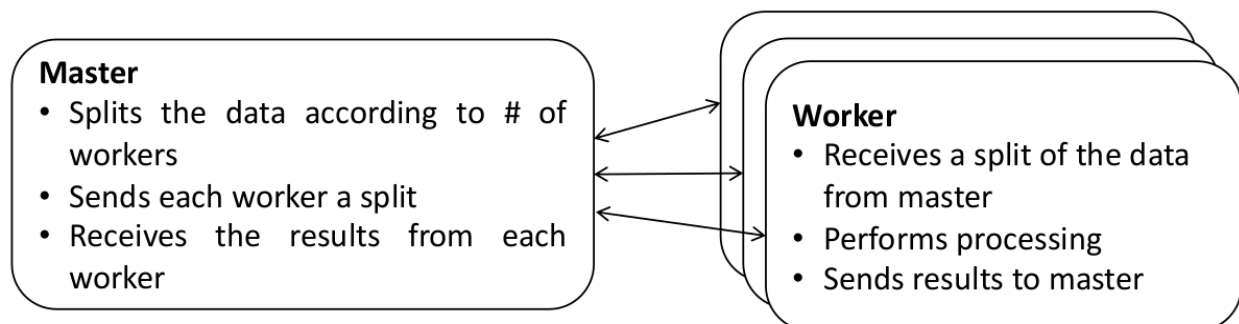
Note: Not all problems can be parallelized. Perhaps some part of the problem (tasks) can be done in parallel but eventually they may require serial processing.



You can see why the Parallel model, if applicable, can be very beneficial over the Serial model as we can complete the whole task very quickly in comparison.

Simplest Environment for Parallel Processing

- This would be when there is no dependency amongst data.
- This allows data to be split into lots of smaller chunks/shards.
- And each process can work on a chunk.
- This is the Master/worker approach.



For example, finding out the number of sentences in a big document.

The document can be split into small small chunks and each worker will then work on each individual chunk and find out the number of sentences in each chunk.

The total number of sentences can be returned to the master which can be added to obtain the total number of sentences in the entire document.

We can do similar processing for counting the number of words in a document.
And this gives us the MapReduce model.

MapReduce

Introduction

Created by Google in 2004

- Jeffrey Dean and Sanjay Ghemawat

Inspired by the LISP programming language which included the map() and reduce() functions. Similarly, each task in the MapReduce model can be either a Map task or a Reduce task.

Map(function, set of values)

- Applies function to each value in the set
- e.g., map (length(), ["", "a", "ab", "abc"]) $\Rightarrow$ (0 1 2 3)

Reduce(function, set of values)

- Combines all the values using a binary function.
- e.g., reduce (sum(), (1,2,3,4,5)) $\Rightarrow$ 15

Provides a framework for parallel computing and programmers get simple API.

They don't have to worry about handling,

- Parallelization
- Data distribution
- Load balancing
- Fault tolerance

It allows one to process huge amounts of data (terabytes and petabytes) on thousands of processors.

Implementations

Google: Original proprietary implementation

Apache Hadoop MapReduce: Most common (open-source) implementation and built to specs defined by Google.

Amazon Elastic MapReduce: Uses Hadoop MapReduce running on Amazon EC2.

Other Cloud Solutions:

- Microsoft Azure HDInsight
- Google Cloud MapReduce for App Engine

MapReduce tasks

MapReduce was invented by Julius Caesar!

He used the map() and reduce() model to count the number of Latin pages in each book in the Great Library of Alexandria!

The soldiers were divided into Centurions (80 soldiers) with one leader each. Each soldier counted the number of Latin pages in their assigned batch of books and reported that to the leader and the leader would keep track of the total count and finally the counts of all the leaders would be summed up to get the final result.

Now you can see how the map() and reduce() functions can be applied to the task of counting words in a document.

map() would apply the splitting of the provided document into words.

So each worker would be supplied a piece of the document and this map() task which will be executed by them.

Then each worker would execute the reduce() function which would add the number of words returned by its respective map function.

Finally the master would apply a reduce() function which adds up the number of words returned by each worker.

Map

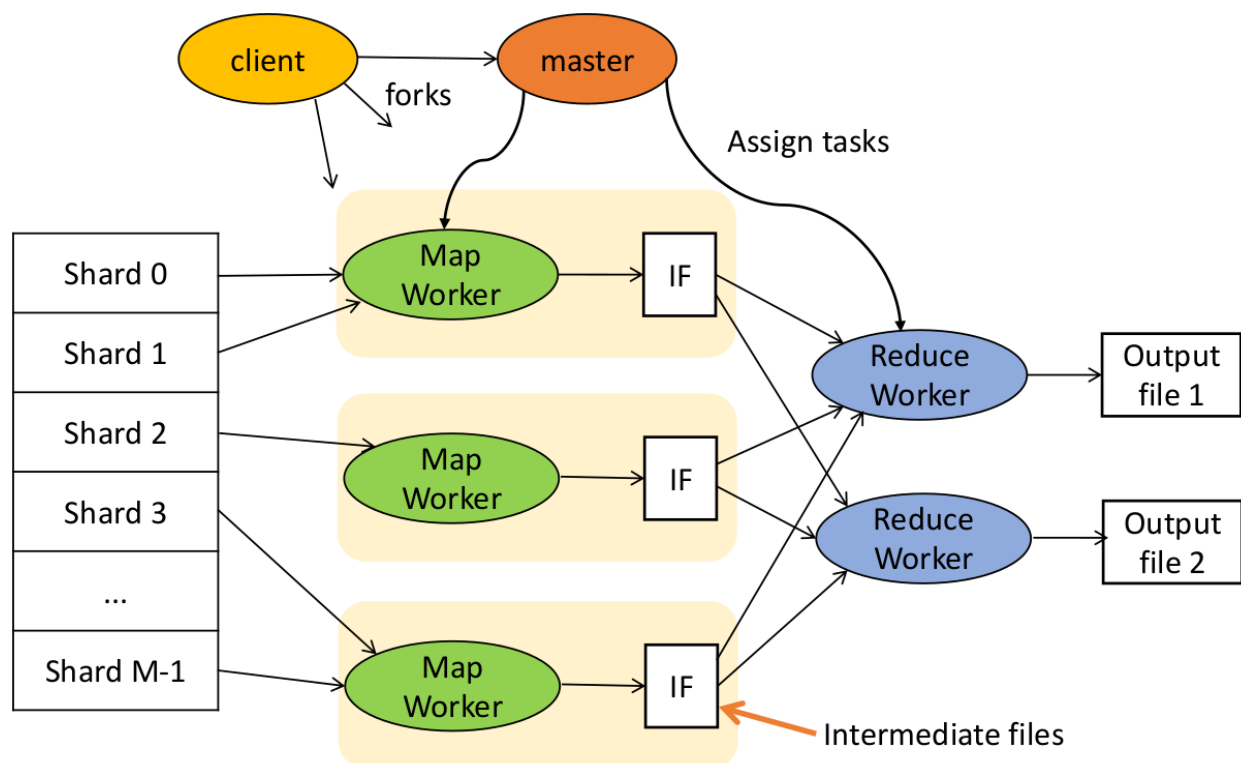
- Grab the relevant data from the source.
- User function gets called for each chunk of input.
- Spits out (key, value) pairs.
- This key can be logically determined by the user or determined dynamically by the program itself.
- For example, in the Julius Caesar anecdote, we can say that we want to count both Latin and Hebrew pages and now it makes sense to have two keys for a Map task output, h = Hebrew and l = Latin and accordingly aggregation will happen for the Reduce tasks and correspondingly you will get two counts in the output of a Reduce task, the Hebrew and Latin page counts!
- **This is why key designing is an important thing for the user as well.**

Reduce

- Aggregate the results.
- **User function gets called for each unique key with all values corresponding to that key.**

Overview of a MapReduce Workflow

- Automatically partition input data into M shards (for M Mapper workers)
- Map: (input shard) → intermediate(key/value pairs).
- Discard unnecessary data and generate (key, value) sets.
- **The MapReduce Framework groups together all intermediate values with the same intermediate key & passes them to the Reduce function.**
- Reduce: intermediate(key/set of values) → result files.
- Input: key & set of values.
- Merge these values together to form a smaller set of values.
- Intermediate results from Map are divided into R pieces corresponding to R reducers using a partitioning function (e.g., $\text{hash}(\text{key}) \bmod R$)
- The user specifies the # of partitions (R) and the partitioning function



Clearly, you can have more than one Reduce worker and according to the number of Reduce workers you will get output files.

Tasks done by a Map Worker

1. Map
 - Grab the relevant data from the source (parse into key, value)
 - Write it to an intermediate file
2. Partition
 - Identify which of R reducers will handle which keys.

- Map partitions data to target it to one of R Reduce workers based on a partitioning function (both R and partitioning function are user defined).

Tasks done by a Reduce Worker

1. Shuffle & Sort
 - Shuffle: Fetch the relevant partition of the output from all mappers
 - Sort by keys (different mappers may have sent data with the same key).
2. Reduce
 - Input is the sorted output of mappers
 - Call the user Reduce function per key with the list of values for that key to aggregate the results.
 -

So the real sequence of tasks in a MapReduce task is,

MAP -> SHUFFLE -> REDUCE

An elaboration on the key-value pair and assignment to Reduce workers,

Suppose you want to count the number of 0s and 1s in a binary file.

So the Map task will output a (1, 1) whenever it encounters a 1 in its shard and (0, 1) whenever it encounters a zero in its shard.

We can have R = 2 (2 Reduce workers) and the partition function can be defined such that all key-value pairs with key = 0 will be assigned to the first Reduce worker and all pairs with key = 1 will be assigned to the second Reduce worker and the output of both Reduce workers would represent the 0 and 1 counts respectively which will be available in their separate output files.

Steps of a MapReduce task

Step 1: Split input files into chunks (shards), M shards

Step 2: Fork processes

- Start up many copies of the program on a cluster of machines.
 - One master: scheduler & coordinator.
 - There are lots of workers.
 - Idle workers are assigned either:
 - map tasks (each works on a shard) – there are M map tasks
 - reduce tasks (each works on intermediate files) – there are R reduce tasks
- [R = # of partitions, defined by the user]

Step 3: Run Map Tasks

- Each Map worker reads the contents of the input shard assigned to it.
- Parses key/value pairs out of the input data.
- Passes each pair to a user-defined map function which produces intermediate key/value pairs which are buffered in memory.

Step 4: Create intermediate files

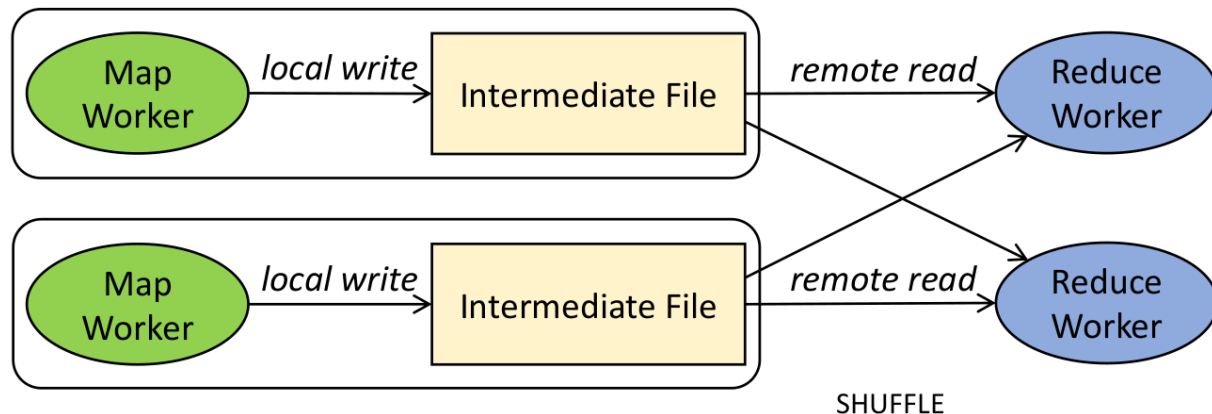
- Intermediate key/value pairs produced by the user's map function are buffered in memory and are periodically written to intermediate files.
- Each intermediate file is partitioned into R regions by a partitioning function.

Step 4a: Partitioning

- Map data would be eventually processed by the reduce workers
- Partition function decides which of R reduce workers will work on which key
- The default partition function is defined as: $\text{hash}(\text{key}) \bmod R$
- Therefore, each Map worker partitions the data by keys and each Reduce worker will later read their partition from the corresponding Map worker.

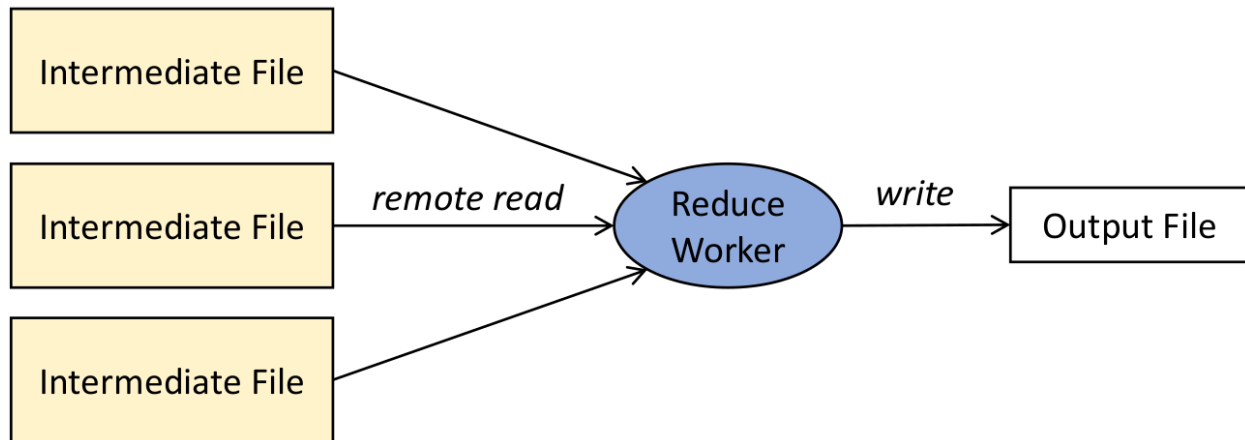
Step 5: Reduce Task: Sorting

- Reduce worker gets notified by the master about the location of intermediate files for its partition.
- **Shuffle:** Uses RPCs to read the data from the local disks of the map workers.
- **Sort:** When the reduce worker reads intermediate data for its partition – It sorts the data by the intermediate keys.
- All occurrences of the same key are grouped together.



Step 6: Reduce Task: Reduce

- The sort phase grouped data using the unique intermediate keys.
- User's Reduce function is given the key and the set of intermediate values for that key, i.e. $\langle \text{key}, (\text{value1}, \text{value2}, \text{value3}, \text{value4}, \dots) \rangle$
- The output of the Reduce function is appended to an output file.



Step 7: Return to user

- When all map and reduce tasks have completed, the master wakes up the user program.
- The MapReduce call in the user program returns and the program can resume execution.
- Output of MapReduce is available in R output files.

Important Points

- In a MR job, reducers do not start executing the `reduce()` method until all Map jobs are completed. But, Reducers start copying the output data from the mappers as soon as they are available.
- All reducers store their output `part-r-xxxxx` files in the output directory specified in the mapreduce job instead of in the local file system.
- Map tasks output files are not stored on HDFS, but are stored on a local file system.
- We cannot set the number of Map tasks in a MapReduce job, but this can be set at site level with “`mapreduce.job.maps`” configuration property in `mapred-site.xml` file.
- We can set the number of Reduce tasks of our choice, Reduce tasks can be setup at site level as well with “`mapreduce.job.reduces`” configuration property in `mapred-site.xml` file.
- Reducers cannot talk to each other.
- Hadoop is not a hardware aware system.

Hands-On Example: WordCount Program

Count # occurrences of each word in a collection of documents

Map Task: Parse data; output each word and a count (1), i.e. key-value pair as (w, 1)

Reduce Task:

Sort: sort by keys (words)

Reduce: Sum together counts of each key (word)

```
function map(String key, String value):
```

```
// key: document name
// value: document contents

    for each word w in value:
        EmitIntermediate(w, "1");
```

```
function reduce(String key, Iterator values):  
    // key: a word  
    // values: a list of counts  
    int result = 0;  
    for each v in values:  
        result += ParseInt(v);  
    Emit(AsString(result));
```

Example,

It will be seen that this mere painstaking burrower and grub-worm of a poor devil of a Sub-Sub appears to have gone through the long Vaticans and street-stalls of the earth, picking up whatever random allusions to whales he could anyways find in any book whatsoever, sacred or profane. Therefore you must not, in every case at least, take the higgledy-piggledy whale statements, however authentic, in these extracts, for veritable gospel cetology. Far from it. As touching the ancient authors generally, as well as the poets here appearing, these extracts are solely valuable or entertaining, as affording a glancing bird's eye view of what has been promiscuously said, thought, fancied, and sung of Leviathan, by many nations and generations, including our own.

it 1
will 1
be 1
seen 1
that 1
this 1
mere 1
painstaking 1
burrower 1
and 1
grub-worm 1
of 1
a 1
poor 1
devil 1
of 1
a 1
sub-sub 1
appears 1
to 1
have 1
gone 1

...
a 1
a 1
aback 1
aback 1
abaft 1
abaft 1
abandon 1
abandon 1
abandon 1
abandoned 1
abandoned 1
abandoned 1
REDUCE
abandoned 1
abandoned 1
abandoned 1
abandonedly 1
abandonment 1
abandonment 1
abased 1
abased 1

a 4736
aback 2
abaft 2
abandon 3
abandoned 7
abandonedly 1
abandonment 2
abased 2
abasement 1
abashed 2
abate 1
abated 3
abatement 1
abating 2
abbreviate 1
abbreviation 1
abeam 1
abed 2
abednego 1
abel 1
abhorred 3
abhorrence 1

25/01/22

Hadoop Installation (Pseudo-Distributed Mode)

[If you are familiar with Docker and how to work with that and a bit familiar with Hadoop then you can skip the tedious installation steps using this article:

<https://medium.com/analytics-vidhya/how-to-easily-install-hadoop-with-docker-ad094d556f11>]

Reference for this installation guide: <https://phoenixnap.com/kb/install-hadoop-ubuntu>

Must have Java installed (try to install Java 8 as that's the more compatible version, but Java 11 also certifiably works as I have tried it)

Install OpenSSH

```
sudo apt install openssh-server openssh-client -y
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
ssh localhost
```

Download Hadoop tar file from here: <http://hadoop.apache.org/releases.html>

Extract tar file using the command, "tar -xvzf <tar_file_name>"

Now transfer the extracted to wherever you want Hadoop to be installed at.

NOTE: Don't place Hadoop in a directory where you need root privileges. So place it in home directory or something like that.

Edit the .bashrc file (which is in home directory) as "nano .bashrc" and add the following lines to the end of that file,

```
# Hadoop Related Options
export HADOOP_HOME=<Path where you have placed the extracted hadoop folder>
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/nativ"
```

Now run, "source ~/.bashrc"

Restart terminal and now check whether Hadoop is installed by running "hadoop version".

Now open, "nano \$HADOOP_HOME/etc/hadoop/hadoop-env.sh"

Find the "#export JAVA_HOME=" line,

Replace it as,

"export JAVA_HOME=<JAVA_HOME path which you can find in .bashrc at the end>"

Open the following XML file "nano \$HADOOP_HOME/etc/hadoop/core-site.xml"

And add the following configuration to override the default values for the temporary directory and add your HDFS URL to replace the default local file system setting:

NOTE: In all the paths (in XML files) that follow replace \$HADOOP_HOME by the actual value of HADOOP_HOME otherwise later build will fail.

```
<configuration>
<property>
  <name>hadoop.tmp.dir</name>
  <value>$HADOOP_HOME/tmpdata</value>
</property>
<property>
  <name>fs.default.name</name>
  <value>hdfs://127.0.0.1:9000</value>
</property>
</configuration>
```

Set the values of these two properties according to your requirement.

Create a Linux directory in the location you specified for your temporary data, i.e.,

```
cd $HADOOP_HOME
```

```
sudo mkdir tmpdata
```

Use the following command to open the *hdfs-site.xml* file for editing:

"nano \$HADOOP_HOME/etc/hadoop/hdfs-site.xml"

Add the following configuration to the file and, if needed, adjust the NameNode and DataNode directories to your custom locations:

```
<configuration>
```

```

<property>
  <name>dfs.data.dir</name>
  <value>$HADOOP_HOME/dfsdata/namenode</value>
</property>
<property>
  <name>dfs.data.dir</name>
  <value>$HADOOP_HOME/dfsdata/datanode</value>
</property>
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
</configuration>

```

Create both the namenode and datanode in the respective locations.

`nano $HADOOP_HOME/etc/hadoop/mapred-site.xml`

Add the following configuration to change the default MapReduce framework name value to **yarn**:

```

<configuration>
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
</configuration>

```

Open the *yarn-site.xml* file in a text editor:

`nano $HADOOP_HOME/etc/hadoop/yarn-site.xml`

Append the following configuration to the file:

```

<configuration>
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>

```

```

</property>
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>127.0.0.1</value>
</property>
<property>
  <name>yarn.acl.enable</name>
  <value>0</value>
</property>
<property>
  <name>yarn.nodemanager.env-whitelist</name>
  <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PERPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value>
</property>
</configuration>

```

Format NameNode using “hdfs namenode -format”

Starting Hadoop Nodes

cd to sbin directory in hadoop folder and run “./start-dfs.sh”

Then run “./start-yarn.sh”

This warning is not an issue.

WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable.

Type this simple command to check if all the daemons are active and running as Java processes:

jps

Hadoop NameNode UI: <http://localhost:9870>

Hadoop DataNode UI: <http://localhost:9864>

YARN Resource Manager: <http://localhost:8088>

Running WordCount program on Hadoop

I assume you have the WordCount.java file already.

Create a Java project in your IDE (whichever one it is) and add the .java file to the “src” folder.

Add “hadoop-common-3.3.1.jar” and hadoop-mapreduce-client-core-3.3.1.jar libraries from your local Hadoop installation (located in the ./share/hadoop/common and ./share/hadoop/mapreduce subdirectories of your HADOOP HOME installation directory) as external JARs in your project.

For IntelliJ this involves “Project Structure -> Modules -> Dependencies -> + -> JARs or Directories.

Build a .jar file of this project, in IntelliJ this involves building artifacts (and in Eclipse I think it's exporting projects) (anyways look this up yourself).

Now add the input to the WordCount program into the HDFS (from the local file system), for this you have to put the input “tar” file in the HDFS as,

```
hadoop fs -put “path_to_tar_file” “path_to_tar_file_in_HDFS”
```

Example,

```
hadoop fs -put WIKIPEDIA.tar /
```

[Puts the tar in the home directory of HDFS]

Confirm whether transfer was complete by checking, `hadoop fs -ls /`

Now run the Hadoop task by running

```
hadoop jar WordCount.jar WordCount /WIKIPEDIA.tar /WordCountOutput
```

WordCount.jar => JAR file that was obtained as output from build

WordCount => Class in the JAR file which contains the main() function

/WIKIPEDIA.tar => Path to input file

/WordCountOutput => **Non-existing directory** in HDFS where output of Hadoop job will be saved

Now upon successful completion, you can check output of job by,

```
hadoop fs -ls /WordCountOutput
```

You will see a bunch of part files, each of these contain output of the Hadoop task.

Because our task was WordCount you will see counts of different words in these part files.

For example,

```
hadoop fs -cat /WordCountOutput/part-r-00000
```

Output:

```
yield 1
yo 2
yogurt 1
you 17
young 14
```

If your Hadoop job fails to start by saying Output Directory already exists or something like that even though the Output Directory doesn't exist, then in your WordCount.java file change the following lines,

```
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

TO

```
FileInputFormat.addInputPath(job, new Path(args[1]));  
FileOutputFormat.setOutputPath(job, new Path(args[2]));
```

Recompile the JAR file and rerun the Hadoop job and hopefully everything should work.
Credits to Shwetank Singh Rathore for spending two days on this and crying to eventually reach this solution.

If your Map task itself fails with some reason like “Could not find Mapper class” or something similar, then add the following line to the main function of the java file near the start.

```
job.setJarByClass(WordCount.class);  
(or whatever class your main function is in)
```

Credits to Me for spending one full day on this and crying to eventually reach this solution.

Check which Hadoop jobs are running by using the command,
yarn application -list

To kill a particular Hadoop job you can use the command,
yarn application -kill \$ApplicationId (ApplicationID you can obtain by using the list command)

27/01/22

Advanced MapReduce

The programmer has little impact on:

- WHERE a mapper or reducer runs.
- WHEN a mapper or reducer starts or finishes.
- WHICH input key-value pairs are processed by a specific mapper.
- WHICH input key-value pairs are processed by a specific reducer.

But this is a good thing because MapReduce automatically takes care of all these issues for you!

Techniques for controlling execution and managing the flow of data

- The ability to construct complex data structures as keys and values to store and communicate partial results.

- The ability to execute user-specified initialization code at the beginning of a map or reduce task, and the ability to execute user-specified termination code at the end of a map or reduce task.
- The ability to preserve state in both mappers and reducers across multiple input or intermediate keys.
- The ability to control the sort order of intermediate keys, and therefore the order in which a reducer will encounter particular keys.
- The ability to control the partitioning of the key space, and therefore the set of keys that will be encountered by a particular reducer.

Scalability & Efficiency

Scalability: Ensuring that there are no inherent bottlenecks as algorithms are applied to increasingly larger datasets.

Efficiency: Ensuring that algorithms do not needlessly consume resources and thereby reducing the cost of parallelization

Two very principal aspects of linear scalability:

- **an algorithm running on twice the amount of data should take only twice as long.**
- **an algorithm running on twice the number of nodes should only take half as long.**

You should always aim for Linear Scalability while designing an algorithm.

Data Splits & Record Readers

- By default, MapReduce will try to assign Map tasks to those worker nodes that also hold the corresponding chunks of input data in order to reduce network communication.
- That is, a Map task of a worker node can directly read a data chunk (usually 64 MB) from the part of the HDFS that this worker node manages.
- Reading from a data chunk is taken care of by the InputFormat class that is registered in the driver function (main function) of the MapReduce job. This class in turn creates a RecordReader class which creates the individual key-value pairs that serve as input for the Map function.

InputFormat

Hadoop relies on the InputFormat of the job to do three things:

1. Validate the input configuration for the job (i.e., checking that the data is there).
2. Split the input blocks and files into logical chunks of type InputSplit , each of which is assigned to a map task for processing.

3. Create the RecordReader implementation to be used to create key-value pairs from the raw InputSplit . These pairs are sent one by one to their mapper.

RecordReader

A RecordReader uses the data within the logical boundaries created by the InputSplit to generate key-value pairs.

[Start and End denote the logical boundaries]

- i) Start: is the byte position in the file where the RecordReader should start generating key-value pairs.
- ii) End: is where it should stop reading records.

These are not hard boundaries as far as the API is concerned—there is nothing stopping a developer from reading the entire file for each map task.

While reading the entire file is not advised, reading outside of the boundaries is often necessary to ensure that a complete record is generated.

The Hadoop MapReduce framework spawns one map task for each InputSplit generated by the InputFormat for the job.

For TextInputFormat the default split size is one file, and for LineRecordReader the default record size is one line of the file.

In this case, the default granularity of a key-value pair to be passed on to the Map function is a (URI, line) pair. Both split and record sizes can be adapted to particular kinds of input data, e.g., files, database records, log messages, etc. i.e. we can override the default behaviour to include a lot of things as specified in the CustomWordCount.java, CustomFileInputFormat.java and CustomLineRecordReader.java files.

URI = Path to the file, Line = Reference to a Line in the specified file (line number perhaps)

How many Map tasks?

- The number of maps is usually driven by the total size of the inputs, that is, the total number of blocks of the input files.
- The right level of parallelism for maps seems to be around 10-100 maps per-node.

Combiners and Partitioners

Combiners are an additional optimization to perform a "local" aggregation before the key-value pairs are emitted through the network.

But note that Hadoop doesn't provide a guarantee that the Combiner will always be executed after a Map task, it's all dynamically determined.

Examples:

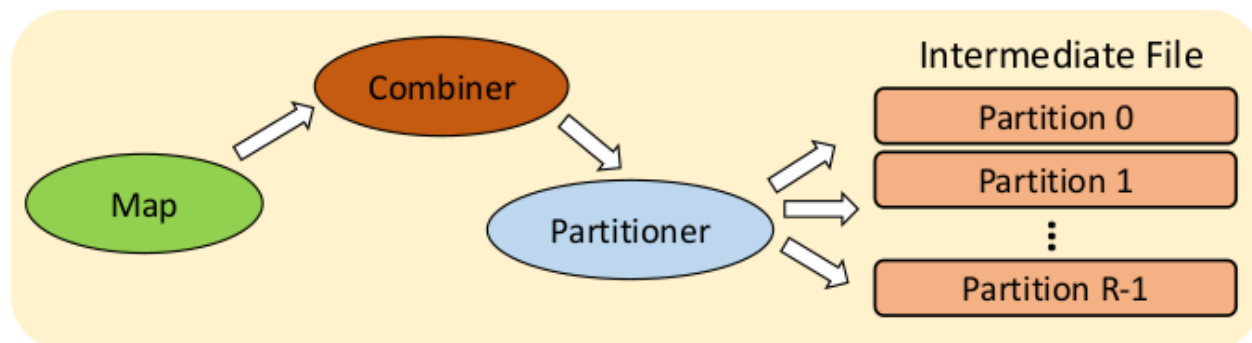
- Compute the partial sums of integers at each mapper.
- Compute the partial counts of word occurrences at each mapper.

Users can optionally specify a combiner, via `Job.setCombinerClass(Class)`

Usually we'll use the Reducer class itself as the Combiner class.

Partitioners need to ensure that the workload of all reducers is balanced (i.e., the partitioner is the counterpart of the scheduler, which needs to ensure that the workload of all mappers is balanced).

Simple hash-partitioning function: $\text{hash}(\text{key}) \% \# \text{worker_nodes}$



How many Reduces?

The number of reduces for the job is set by the user via
`Job.setNumReduceTasks(int)`

Increasing the number of reducers, increases the framework overhead, but increases load balancing and lowers the cost of failures.

Issues

Scheduling (load balancing)

- MapReduce jobs are divided into smaller units called tasks (e.g., map task may be responsible for processing an `InputSplit`).
- In large jobs, the total number of tasks may exceed the number of tasks that can be run on the cluster concurrently – so we queue the tasks and assign them to nodes as they become available.

- A dedicated task scheduler has to ensure that all Mappers are balanced.

Synchronization (among Mappers & Reducers)

- Reducers can only start once all Mappers are finished.
- A few long-running stragglers may delay the overall MapReduce execution time substantially.
- Stragglers are often caused by circumstances that are local to the worker node on which the straggler task is running.
- The worker machine can be overloaded because of the scheduler.
- Frequent (but recoverable) disk errors.

Solution:

- Abort stragglers when MapReduce computation is near its end (where the progress is monitored by a master node).
- For each aborted straggler, schedule a backup (replacement) task on another worker node.
- Can significantly improve overall completion time.

Fault-tolerance (failure management of tasks)

- In large clusters, disk failures are common and RAM experiences more errors than one might expect.
- Backup tasks are started to replace the original task when the MapReduce job is close to finishing.

Note on Sharing Data

- A static DistributedCache (Job.addCacheFile() in Hadoop 3.3.1) object allows for sharing large data volumes (via file- based access) among all instances of Mapper and Reducer classes.
- DistributedCache provides a container for an entire set of files that are to be shipped to the Mappers and Reducers.
- The framework will copy the necessary files to the worker node before any tasks for the job are executed on that node.
- Its efficiency stems from the fact that the files are only copied once per job and the ability to cache archives which are un-archived on the workers.
- Example: WordCount2.java, available on Moodle for download.

Useful Design Patterns for Text Processing with MapReduce

[We'll learn each of these in detail in the next section]

1. Local Aggregation

Implement your own Combiner function to pre-aggregate values that belong to the same key.

2. Pairs and Stripes

Two principle design patterns to extract word-pairs from documents.

Pairs

Key, value

((2,1), a)

((2,2), b)

((2,3), b)

((2,4), c)

Column: 1 2 3 4

Row 2:

a	b	b	c

3. Secondary Sorting

Employ compound keys to better control the data flow.

4. Relational Joins

- Map-side and Reduce-side join techniques.

Other relevant topics:

Inverted Indexing: Using MapReduce to create an inverted index from a collection of documents.

Example, Task is to find out which documents a word belongs to.

So when given a word, you must be able to return a list of documents in which that word is present.

And this is the inverted index, "word" -> [d1, d3, ...] => "word" appears in documents d1, d3, ...

You can see how a MapReduce model can help with this task.

Local Aggregation

- Writing a custom combiner function allows for preprocessing the output of a Mapper before the final key-value pairs are emitted.

- Thus, the Combiner serves just like a "Mini-Reducer" that operates over the intermediate output of each Mapper.

- But the Combiner as discussed before is an optimization technique that's not always guaranteed to be executed, so perhaps we can try some optimization locally that's always guaranteed to be executed.

- Possible granularities to replace the Combiner:

- Within the Mapper class (instantiated for every worker node).
- Within the Map method (called for every key-value pair).

Note: Both depend on the InputFormat (i.e., size of the splits) and the RecordReaders used to initialize the MapReduce job.

1) Local Aggregation in the Mapper class

```

1: class MAPPER
2:   method INITIALIZE
3:      $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:   method MAP(docid  $a$ , doc  $d$ )
5:     for all term  $t \in \text{doc } d$  do
6:        $H\{t\} \leftarrow H\{t\} + 1$ 
7:   method CLOSE
8:     for all term  $t \in H$  do
9:       EMIT(term  $t$ , count  $H\{t\}$ )

```

- An associative array (e.g., a HashMap) is used to preserve state across multiple calls of the Map method (for each input key-value pair).

- We can accumulate partial term counts in the associative array across multiple documents and emit key-value pairs only when the mapper has processed all documents.

- Emission of intermediate data is deferred until the Close method

2) Local Aggregation in the Map method

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:      $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:     for all term  $t \in \text{doc } d$  do
5:        $H\{t\} \leftarrow H\{t\} + 1$ 
6:     for all term  $t \in H$  do
7:       EMIT(term  $t$ , count  $H\{t\}$ )

```

- An associative array (e.g., a HashMap) is used to pre-aggregate the counts of each distinct word in a document.

- Instead of emitting a key-value pair for each term in the document, this version emits a key-value pair for each unique term in the document.

- Given that some words appear frequently within a document, this can yield substantial savings in the number of intermediate key-value pairs emitted, especially for long documents.

Both of these techniques are always guaranteed to execute so they will end up making the MapReduce job faster in comparison to using Combiners where execution is not guaranteed.

Limitations of Local Aggregation:

How would you compute the Mean over a large array of input numbers by using local aggregation?

Example:

Mean(1,2,3,4,5) != Mean(Mean(1,2), Mean(3,4,5))

Unlike the Sum operation, Mean is not associative.

So Local Aggregation can only be used for associative operations.

Solution: Instead of emitting a single aggregate value, we can emit pairs of partial sums and counts in the Mapper and Combiner classes.

The final Mean value is computed in the last step of the Reducer class.

```
1: class MAPPER
2:   method MAP(string t, integer r)
3:     EMIT(string t, pair (r, 1))

1: class COMBINER
2:   method COMBINE(string t, pairs [(s1, c1), (s2, c2) ...])
3:     sum ← 0
4:     cnt ← 0
5:     for all pair (s, c) ∈ pairs [(s1, c1), (s2, c2) ...] do
6:       sum ← sum + s
7:       cnt ← cnt + c
8:     EMIT(string t, pair (sum, cnt))

1: class REDUCER
2:   method REDUCE(string t, pairs [(s1, c1), (s2, c2) ...])
3:     sum ← 0
4:     cnt ← 0
5:     for all pair (s, c) ∈ pairs [(s1, c1), (s2, c2) ...] do
6:       sum ← sum + s
7:       cnt ← cnt + c
8:     ravg ← sum/cnt
9:     EMIT(string t, integer ravg)
```

MAP(string t, integer r)

string t => Key for Map task

integer r => Number to find average of

So MAP task outputs, "(t, (r, 1))" as the key-value pair

So for each number encountered outputs the number as well as 1 which is the count of occurrences of that number.

And the COMBINER task outputs the map key and the value will be the sum of all the numbers in that map task and the number of numbers, i.e. count that was encountered in that map task.

Finally in the REDUCER task for each key we receive a bunch of (sum, count) pairs from the Combiners, we can add the sums and counts separately to obtain a global sum and count and thus we obtain the average as sum / cnt .

Note that we can replace the Combiner by Local Aggregation as well.

NOTE: We can only have 1 Reducer in this case, otherwise we will again obtain multiple averages which as we know is not associative. So we can parallelize the workflow by delegating the process of summation to Map tasks but computing the average itself has to be done serially by a single Reducer.

Pairs and Stripes

- Let's continue the idea of emitting complex objects (e.g., pairs of partial sums and counts in the previous example) to compare two common design patterns for processing large collections of documents.

- Suppose we would like to create a large co-occurrence matrix of words that occur together at a subsequent position in a document, i.e. find out how many times a particular word is followed by another word in a document,

$\text{Matrix}[i, j] = 1 \Rightarrow \text{Word "i" precedes word "j" one time}$

- For n words, this obviously results in a large $n \times n$ matrix.

- The Pairs algorithm detects co-occurring words in each document and emits a count of 1 for each pair.

- The Stripes algorithm emits, for each individual word, an entire array (i.e., "stripe") of co-occurring words together with their local counts.

The Pairs Algorithm

```
1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $w \in \text{doc } d$  do
4:       for all term  $u \in \text{NEIGHBORS}(w)$  do
5:         EMIT(pair ( $w, u$ ), count 1)    > Emit count for each co-occurrence
1: class REDUCER
2:   method REDUCE(pair  $p$ , counts [ $c_1, c_2, \dots$ ])
3:      $s \leftarrow 0$ 
4:     for all count  $c \in \text{counts } [c_1, c_2, \dots]$  do
5:        $s \leftarrow s + c$                 > Sum co-occurrence counts
6:     EMIT(pair  $p$ , count  $s$ )
```

- The Mapper employs a nested loop to create pairs of neighboring words, which are each emitted with a count of 1.
- The Reducer is identical to the earlier WordCount example, except that it obtains word pairs as keys.

Note: **NEIGHBORS(w)** is a custom function that we can use to define the neighbourhood of a word. For example, we can set a neighbourhood of size 3, so each word's 3 adjacent words in the forward and backward directions will be the neighbourhood of that word.

The Stripes Algorithm

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $w \in \text{doc } d$  do
4:        $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
5:       for all term  $u \in \text{NEIGHBORS}(w)$  do
6:          $H\{u\} \leftarrow H\{u\} + 1$  ▷ Tally words co-occurring with  $w$ 
7:       EMIT(Term  $w$ , Stripe  $H$ )

1: class REDUCER
2:   method REDUCE(term  $w$ , stripes  $[H_1, H_2, H_3, \dots]$ )
3:      $H_f \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:     for all stripe  $H \in \text{stripes } [H_1, H_2, H_3, \dots]$  do
5:       SUM( $H_f, H$ ) ▷ Element-wise sum
6:     EMIT(term  $w$ , stripe  $H_f$ )

```

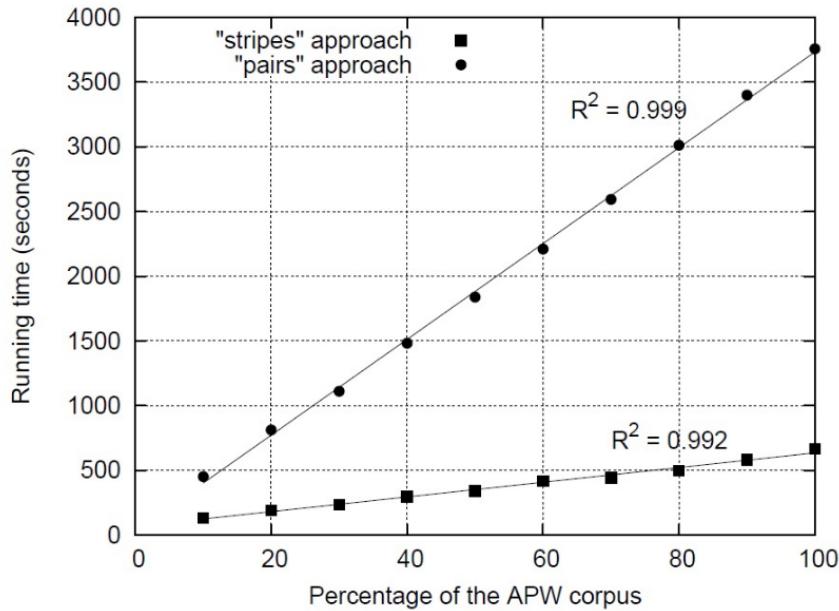
Note: The for loop in the Reducer sums up all the Hashmaps provided to it and copies it to the new Hashmap H_f to obtain the output.

For example, let stripes = [{ 'a': 5, 'b': 1, 'c': 2 }, { 'a': 2, 'b': 1 }, { 'b': 2 }]

Then $H_f = \{ \text{'a': 7, 'b': 4, 'c': 2} \}$

- The Mapper pre-aggregates the local counts of each neighbor into an associative array and emits these stripes for each word (not necessarily unique word) encountered in the document.
- The Reducer combines the partial counts into global counts. The results are the stripes of our matrix.

Performance Comparison



Both plots show a (nearly) perfect linear scale-out for larger data sizes.

Comparison of the Pairs and Stripes algorithms on the Associated Press Worldstream (APW) corpus with 2.27 million documents and a size of 5.57 GB, performed on a 19-node Hadoop cluster.

Obviously the Stripes algorithm would perform better because of the local aggregation that was done.

Secondary Sorting

The MapReduce framework guarantees that keys emitted by the Mapper are shuffled together correctly and sorted at the Reducer.

What if in addition to sorting by key, we also need to sort by value?

Example: `map(docId, doc) -> [(term, (docId, count))]`

Google's MapReduce has built-in functionality for setting a secondary sorting condition; Hadoop does not have this.

Solution: Use composite keys plus a custom Partitioner.

`map(docId, doc) -> [(term, docId), count]`

Because we output key-value pairs with a composite key, sorting is done on the basis of both "term" and "docId". However, the default partitioning function which assigns key-value outputs of Map tasks to Reducers requires a non-Composite key, so we define a Custom Partitioner function which defines a custom hash function for partitioning the key-value pairs to Reducers. In the given code, we use only the term contents of the (term, docId) key to hash and obtain the Reducer number to which the key-value pair should be assigned to. But note that this partition function can be anything really (can even include both term and docId hashes as well).

Custom Partitioner

Step 1: Define your own Partitioner class and override the `getPartition()` method:

```
public static class TermPartitioner extends Partitioner<Text, Text> {  
    @Override  
    public int getPartition(Text key, Text value, int numReduceTasks) {  
        String [] termDocIdPair = key.toString().split( " \\t " );  
        String term = termDocIdPair[0];  
        return term.hashCode() % numReduceTasks;  
    }  
}
```

Step 2: Register your new Partitioner class in the driver:

```
job.setPartitionerClass(TermPartitioner.class);
```

Relational Joins

Suppose we have two input relations S and T,

<u>S</u>	<u>k_i, a_i, b_i</u>	<u>T</u>	<u>k_i, c_i, d_i</u>
	k_1, a_1, b_1		k_1, c_1, d_1
	k_2, a_2, b_2		k_1, c_2, d_2
	k_3, a_3, b_3		

- Let k_i denote the join key (shared among both relations), s_i and t_i be the identifiers of the tuples, and S_i and T_i be the other attributes of S and T
- Obviously, the goal is to bring together (i.e., "join") tuples from S and T that share the same join key.

There are two types of Relational joins available in MapReduce, Map-side joins & Reduce-side joins.

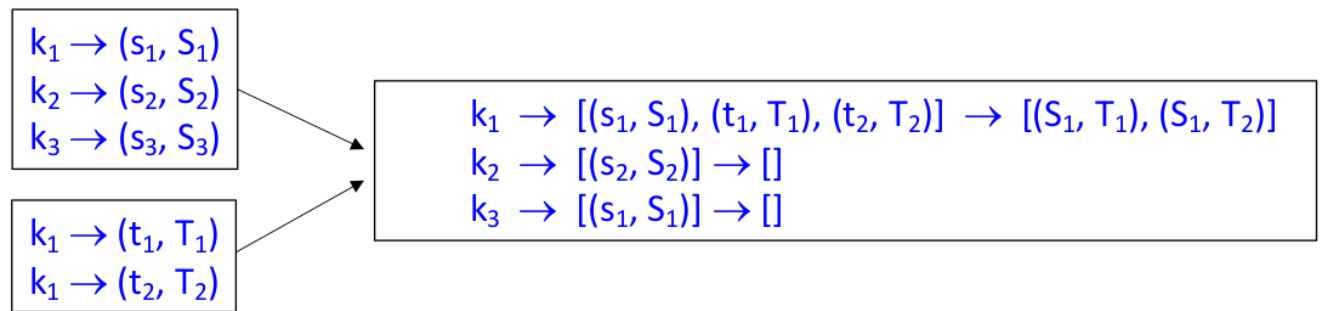
Reduce-side Joins

- Map emits the join key as the intermediate key and the remaining tuple as the intermediate value (for both relations S and T).

For the above table,

- Reduce obtains all tuples belonging to the same join key and combines all tuples from S with

the tuples from T via a nested loop.



Optimize with the secondary sorting idea to group tuples from S and T together.

Drawback:

The aforementioned algorithm is called "parallel sort-merge join" in databases.

Joining tuples at the reducer however requires shuffling both relations completely through the network.

Suppose each relation consists of 1 M tuples, but only 1 tuple in S joins with 1 tuple in R. Then 1,999,998 tuples are shuffled in vain.

Map-side Joins

- **For a Map-side join to be possible, we need to assume that both relations are already available sorted and partitioned by their join keys.**

- We perform a standard (but parallel) merge-join by mapping over the larger relation and by reading the corresponding partition of the smaller relation directly inside the Map function.

Tuples with matching join keys are again joined together via a nested-loop, but this time in the Map function.

- No Reducer join is needed, the output of the Map function are already the joined tuples.

Bonus Join: Memory-Backed Joins

- If one of the relations fits into main-memory, we can load that relation into a shared main-memory data structure which is then made accessible by all Mappers.

- We again map over the larger relation and probe each key of that relation via a random-access into the shared main-memory data structure.

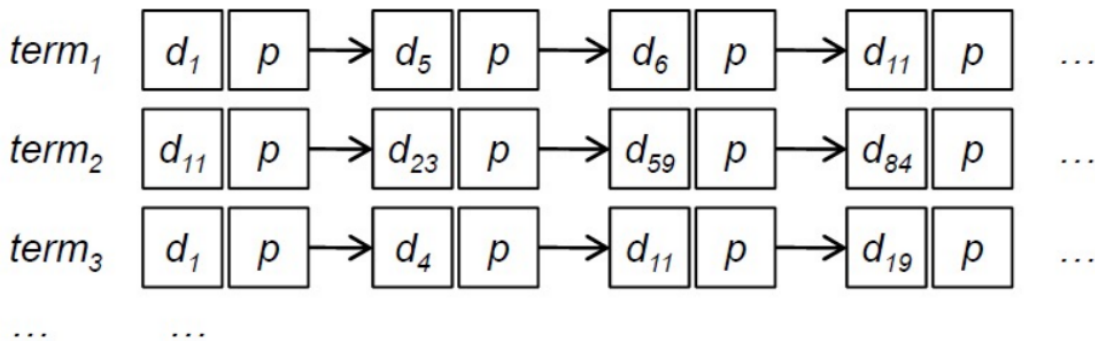
- This is similar to a standard (but parallel) hash join.

- However, sharing large main-memory data structures across an entire compute cluster is tricky.

Inverted Indexes

- An inverted index contains, for each term, a list of (docId, score) pairs (so called "postings") with pointers to all documents that contain that term.

terms postings



- We have already seen the benefit of using an inverted index for querying text documents.
- In MapReduce, the challenge lies in parsing the documents and to create posting lists that are sorted by docId efficiently.

Basic Inverted Index Implementation

```

1: class MAPPER
2:   procedure MAP(docid  $n$ , doc  $d$ )
3:      $H \leftarrow$  new ASSOCIATIVEARRAY
4:     for all term  $t \in$  doc  $d$  do
5:        $H\{t\} \leftarrow H\{t\} + 1$ 
6:     for all term  $t \in H$  do
7:       EMIT(term  $t$ , posting  $\langle n, H\{t\} \rangle$ )

1: class REDUCER
2:   procedure REDUCE(term  $t$ , postings  $[\langle n_1, f_1 \rangle, \langle n_2, f_2 \rangle \dots]$ )
3:      $P \leftarrow$  new LIST
4:     for all posting  $\langle a, f \rangle \in$  postings  $[\langle n_1, f_1 \rangle, \langle n_2, f_2 \rangle \dots]$  do
5:       APPEND( $P, \langle a, f \rangle$ )
6:     SORT( $P$ )
7:     EMIT(term  $t$ , postings  $P$ )

```

But this assumes the posting list for each term to fit into memory. Can we use Secondary Sorting to make things better?

Inverted Indexing with Secondary Sorting

- Instead of emitting this:
 $\text{map}(\text{docId}, \text{doc}) \rightarrow [(\text{term}, (\text{docId}, \text{count}))]$

- We directly emit pairs of terms and document ids as keys in the Mapper:
map(docId, doc) -> [(term, docId), count]
(See also our earlier secondary sorting example)

```
1: class MAPPER
2:   method MAP(docid  $n$ , doc  $d$ )
3:      $H \leftarrow$  new ASSOCIATIVEARRAY
4:     for all term  $t \in$  doc  $d$  do
5:        $H\{t\} \leftarrow H\{t\} + 1$ 
6:     for all term  $t \in H$  do
7:       EMIT(tuple  $\langle t, n \rangle$ , tf  $H\{t\}$ )

1: class REDUCER
2:   method INITIALIZE
3:      $t_{prev} \leftarrow \emptyset$ 
4:      $P \leftarrow$  new POSTINGSLIST
5:   method REDUCE(tuple  $\langle t, n \rangle$ , tf  $[f]$ )
6:     if  $t \neq t_{prev} \wedge t_{prev} \neq \emptyset$  then
7:       EMIT(term  $t$ , postings  $P$ )
8:        $P.RESET()$ 
9:        $P.ADD(\langle n, f \rangle)$ 
10:     $t_{prev} \leftarrow t$ 
11:   method CLOSE
12:     EMIT(term  $t$ , postings  $P$ )
```

08/02/22

Public Cloud Services with AWS

For developing a website, you'll require frontend and backend which are developed and are ready for deployment to the public.

For Computing in AWS, you'll require an EC2 instance which serves as a single server. In a single EC2 instance system your frontend, backend and everything else will be stored which clearly doesn't seem like the optimal option as we can only do vertical scaling. For scaling the service and to safeguard against failures, you'll require multiple EC2 instances (horizontal scaling) but how to determine which instance will be assigned which task in a way that the

resources are being utilized efficiently? For this you will have to connect a load balancer that will do just this.

And ofcourse, the application will require access to a database or multiple databases for which you can choose the type of database which suits your application, SQL, NoSQL, combination of both, etc.

All of the communication over the network happens using TCP.

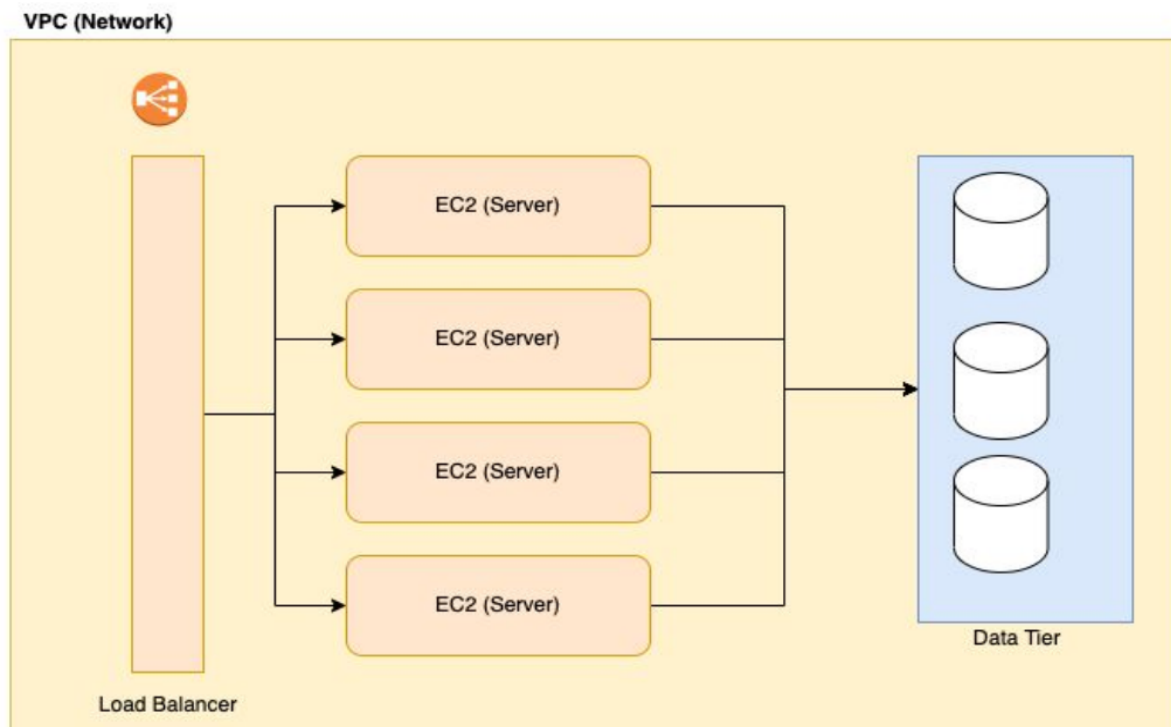
The database/s will be linked to the EC2 instances as they'll require access to the DB.

To provide fault tolerance and availability, you will have multiple DB instances storing the same data. This constitutes the data tier for your service and you can provision this in the master-slave model where the master DBs are “write-only” and the slave DBs are “read-only”. So the EC2 instances can only write into the master DB instances and read from the read-only DB instances and the master DBs will propagate the changes to their slave DB instances.

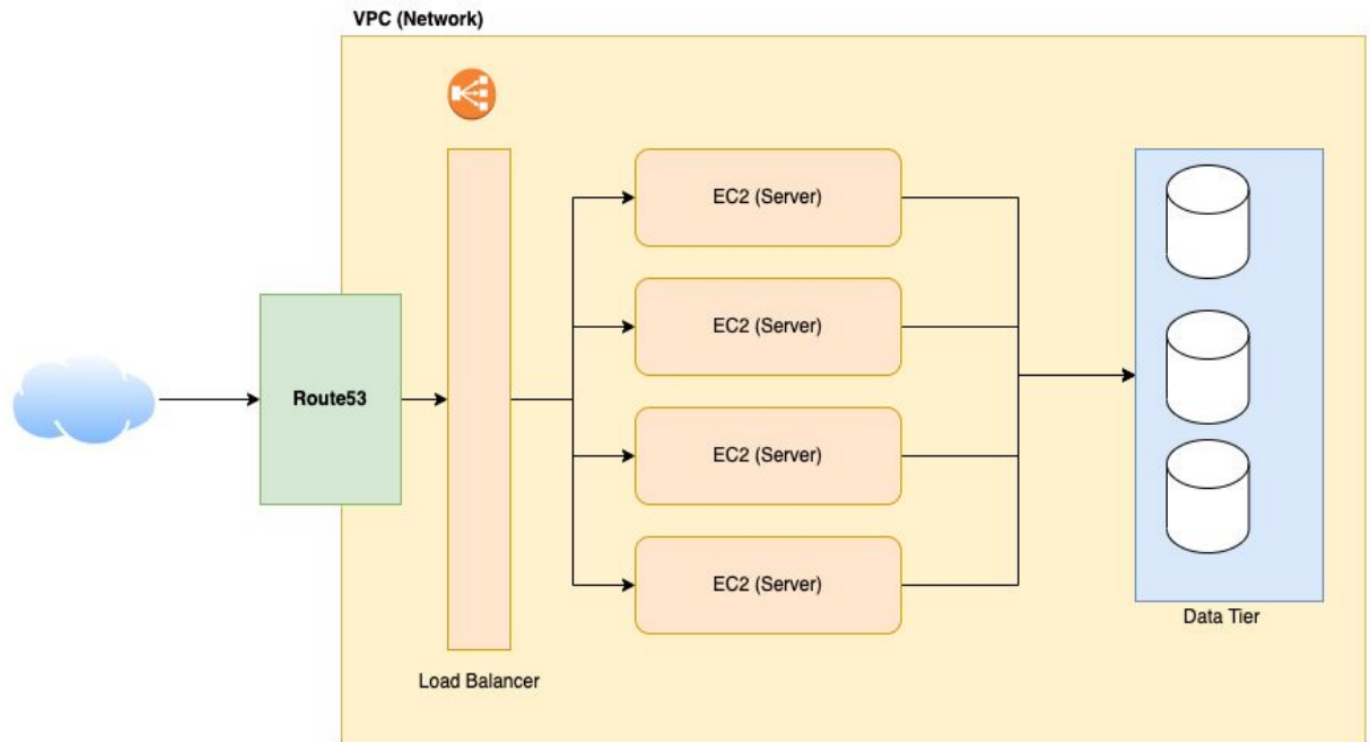
A general guiding principle for system design is that reads are more frequent than writes.

The created service is fully accessible over the cloud/Internet for anyone, which might not be a desirable outcome as you want to restrict access and also reduce network traffic. For this your service is placed in a Virtual Private Cloud (VPC) which will only allow certain IP addresses to access the service or only allow access to requests to satisfy certain criteria.

This is where we've reached till now,

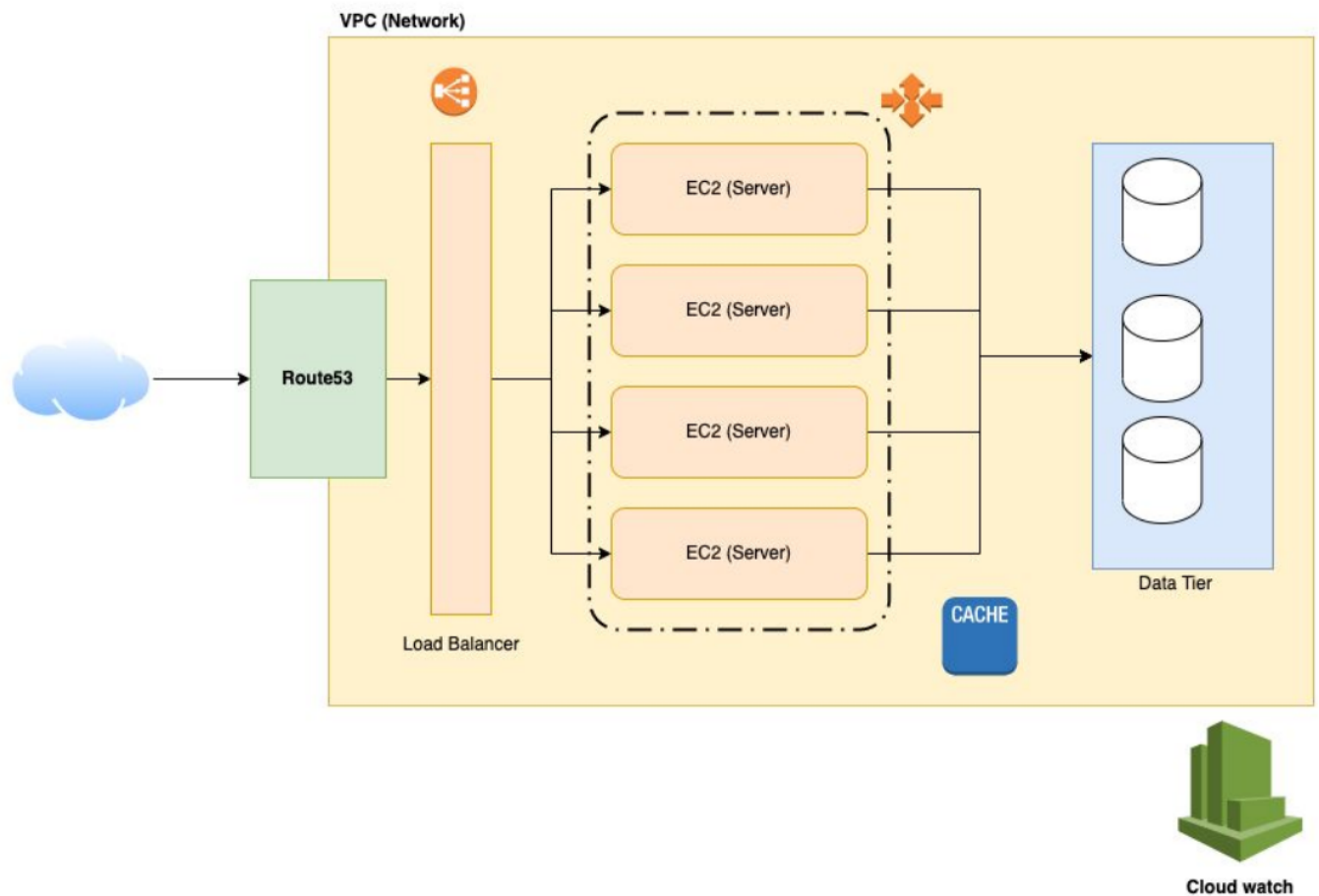


Now you'll add a DNS service (Route53) to enable easy access over the Internet by using Domain Names instead of IP addresses. With this you can set up routing tables and loads of other stuff like that.



Now you need something to monitor the system, i.e. system load, which EC2 instances are up, system health, etc.

If there is huge amounts of traffic and none of the EC2 instances are free then you can connect a new EC2 instance and attach it to the load balancer. This process can be automated which is called Auto-Scaling and for this there are templates. So basically you need System Monitoring and you can also use Cache which will cache previously executed request cycles, so if the same request is received again, then the response can be immediately sent back. You also need monitoring of logs for which you can use AWS CloudWatch.



AWS Services: Computing

Amazon EC2	Service to set up a virtual server using AWS
ELB	Load balancer service that implements traffic control for EC2 instances
Auto Scaling	A service that automatically increases, decreases, or adjusts EC2 instances (this is called scaling).
AWS Lambda	A service that allows you to store and execute only programming code without a server (called serverless).
Amazon Lightsail	VPS (Virtual Private Server) service that provides a cohesive configuration of virtual servers, storage, databases, and networking at a low price.

ELB = Elastic Load Balancer [has various types based on requirements]

Lightsail allows you to skip a lot of configuration nitty gritty as it automatically has EC2 instances configured and ready to go along with many services.

EC2 Families

	Type	Description	Mnemonic
General Purpose	a1	Good for scale-out workloads, supported by Arm	a is for Arm processor – or as light as A1 steak sauce
	t-family: t3, t3a, t2	Burstable, good for changing workloads	t is for tiny or turbo
	m-family: m6g, m5, m5a, m5n, m4	Balanced, good for consistent workloads	m is for main or happy medium
Compute Optimized	c-family: c5, c5n, c4	High ratio of compute to memory	c is for compute
Memory Optimized	r-family: r5, r5a, r5n, r4	Good for in-memory databases	r is for RAM
	x1-family: x1e, x1	Good for full in-memory applications	x is for xtreme
	High memory	Good for large in-memory databases	High memory is for... high memory.
	z1d	Both high compute and high memory	z is for zippy
Accelerated Computing	p-family: p3, p2	Good for graphics processing and other GPU uses	p is for pictures
	Inf1	Support machine learning inference applications	Inf is for inference
	g-family: g4, g3	Accelerate machine learning inference and graphics-intensive workloads	g is for graphics
	f1	Customizable hardware acceleration with field programmable gate arrays (FPGAs)	f is for FPGA or feel as in hardware
Storage Optimized	i-family: i3, i3en	SDD-backed, balance of compute and memory	i is for IOPS
	d2	Highest disk ratio	d is for dense
	h1	HDD-backed, balance of compute and memory	H is for HDD

Burstable systems => Can quickly adapt to changing demands/spikes.

AWS Lambda: Serverless Computing

Serverless Computing is just that, computing without a server. So you just have to provide the code/function to execute and a configuration file “.yaml”, etc. which specifies the computing resources required and AWS Lambda will handle it for you.

Serverless Computing is generally event-driven, so generally there's a trigger event associated with these Lambda functions that will determine when the functions are executed.

Serverless Demo

The YML file for configuration,

```
1 service:
2   name: first-serverless
3
4 plugins:
5   - serverless-bundle
6   - serverless-pseudo-parameters
7   - serverless-localstack
8
9 provider:
10  name: aws
11  runtime: nodejs12.x
12  memorySize: 256
13  stage: ${opt:stage, 'dev'}
14  region: ap-south-1
15
16 functions:
17   hello:
18     handler: src/handlers/hello.handler
19     events:
20       - http:
21         method: GET
22         path: /hello
23
24 custom:
25   bundle:
26     linting: false
27   localstack:
28     debug: true
29     stages:
30       - local
31       - akshay
32   endpointFile: localstack_endpoints.json
33
```

events: => Specifies the trigger events for the specified function

For example, here, the “handler” function in the path “src/handlers/hello.js” will be executed when an HTTP GET request is made to the “/hello” path of the service.

```
1 async function hello(event, context) {
2   return {
3     statusCode: 200,
4     body: JSON.stringify({ message: 'Hello from https://codingly.io' }),
5   };
6 }
7
8 export const handler = hello;
9
10
11
```

AWS Services: Database

Amazon RDS	Relational database services compatible with MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB
Amazon Aurora	Distributed, accelerated, high-performance relational database compatible with MySQL and PostgreSQL
Amazon DynamoDB	Key-value and document-based NoSQL databases that deliver performance in the millisecond range, regardless of size
Amazon ElastiCache	Fully managed in-memory data store compatible with Redis or Memcached
Amazon Redshift	A data warehouse that provides a business data analysis platform on AWS

AWS Services: Storage

Amazon Simple Storage Service (S3)	Object-based storage service for storing large amounts of frequently used data in the medium to long term.
Amazon Elastic Block Store (EBS)	Block-type storage dedicated to EC2 instances that can be attached to EC2 via the network.
Instance Store	Block-type storage physically attached to EC2 Storage for temporary storage of data
Amazon Elastic File System (EFS)	A service that provides an NFS file system that is suitable for storing file data in a directory structure.
Amazon S3 Glacier	A storage service that has the same availability and durability as S3, but at a lower price. Used for data archiving and long-term backup.

AWS Services: Networking

Amazon VPC	A service to build a virtual networking environment by selecting IP address ranges, creating subnets, and configuring route tables and network gateways.
Amazon CloudFront	A content delivery network (CDN) service that delivers content worldwide through low-latency, high-speed transmission.
Amazon Route 53	A service that provides DNS server functions for domain registration and routing.
AWS VPN	A service that connects AWS and on-premises environments via site-to-site VPN using the Internet
AWS Direct Connect	Leased line service that establishes a private connection between AWS and your data center or office

VPC is to create a restrictive network that is resilient to hacking or any other malicious intent.

AWS Services: User Management

AWS Identity & Access Management (IAM)	A service to create and manage users who use AWS services.
AWS Single Sign-On (SSO)	AWS service that provides users with single sign-on access between on-premises and AWS environments.
Amazon Cognito	A service that provides user authentication functions for web applications built on AWS.

AWS Services: Security

AWS KMS (Key Management Service)	A service to create and manage encryption keys. It can be used to control the encryption of a wide range of AWS services and applications.
AWS Shield	A service that protects AWS resources from distributed denial of service (DDoS) attacks.
Amazon Guard Duty	A threat detection service that continuously monitors for malicious operations. Automatic detection using machine learning is possible.

AWS Services: Monitoring

Amazon CloudWatch	A monitoring service that can be automatically applied to most AWS resources to retrieve metrics values and set alarms, etc.
AWS CloudTrail	Logging and monitoring service to track user activity and API usage
AWS Personal Health Dashboard	Monitoring service that displays the status of AWS services as abnormal or normal and healthy.
AWS Systems Manager	An operational support service that enables integrated management of monitoring data from AWS and on-premise environments and automates operational tasks

AWS Services: Cost Management

Price Calculation Tools	Tools to calculate AWS costs, such as a TCO calculation tool, and a pricing calculator.
AWS Cost Explorer	Visualization tool to perform AWS cost analysis to understand AWS costs and usage and economic efficiency
AWS costs and Usage reports	Reports to see details of AWS costs and usage
AWS Budgets	Budget monitoring tool that allows you to set alerts when budget thresholds are exceeded for each AWS service.

Services Comparison

				
Virtual Servers	Instances	VMs	VM Instances	
Platform-as-a-Service	Elastic Beanstalk	Cloud Services	App Engine	
Serverless Computing	Lambda	Azure Functions	Cloud Functions	
Docker Management	ECS	Container Service	Container Engine	
Kubernetes Management	EKS	Kubernetes Service	Kubernetes Engine	
Object Storage	S3	Block Blob	Cloud Storage	
Archive Storage	Glacier	Archive Storage	Coldline	
File Storage	EFS	Azure Files	ZFS / Avere	
Global Content Delivery	CloudFront	Delivery Network	Cloud CDN	
Managed Data Warehouse	Redshift	SQL Warehouse	Big Query	

Using more than one cloud provider in a single platform is called Multi-cloud.

It can be because a particular cloud provider provides better service in a particular department; maybe because there's no geographically viable options for some services as there are no

nearby servers for some services in some Cloud Providers; can also be used to provide a backup option, for example, if your AWS system is down, then your GCP system can pick up the work.

10/02/22

Cloud Services Demo Contd.

Scaling a System

Load testing a Node.js application

Node.js is a single-thread environment, so it will use only one core for an application.

To load test this application you can use a tool like Apache Benchmark (AB).

For the TA, his application was able to serve approx 10000 requests per second using a single instance of the Node.js application running.

Now to scale the system, make a Docker image of the application and use Docker Compose to add a load balancer [HAProxy] and use it to run 4 instances of the Node.js application which will obviously use 4 CPUs, i.e. 4 cores of your local system.

[if you don't understand any of this then don't worry, in time you will learn :)]

Wikipedia quote "HAProxy is a free and open source software that provides a high availability load balancer and proxy server for TCP and HTTP-based applications that spreads requests across multiple servers. It is written in C and has a reputation for being fast and efficient"

This is what the Docker Compose file looks like,

```
1 version : '3'
2
3 services:
4   lb:
5     image: haproxy:1.7
6     ports:
7       - "8080:8080"
8     volumes:
9       - ./haproxy:/usr/local/etc/haproxy
10
11   nodeapp1:
12     image: nodeapp
13     environment:
14       - APPID=1111
15
16   nodeapp2:
17     image: nodeapp
18     environment:
19       - APPID=2222
20
21   nodeapp3:
22     image: nodeapp
23     environment:
24       - APPID=3333
25
26   nodeapp4:
27     image: nodeapp
28     environment:
29       - APPID=4444
```

This also helps hide your instances' IP Addresses from the public internet. Anyone that will access your application will have to access it via the HAProxy (Load Balancer) and we can place adequate security measures at this Proxy.

HAProxy Configuration

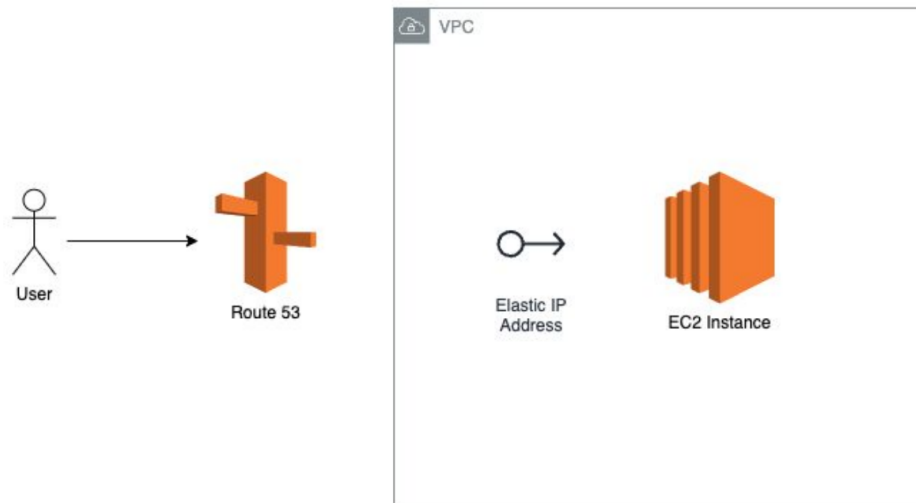
```
1 frontend http
2   bind *:8080
3   mode http
4   timeout client 10s
5   use_backend all
6
7 backend all
8   mode http
9   server s1 nodeapp1:1111
10  server s2 nodeapp2:2222
11  server s3 nodeapp3:3333
12  server s4 nodeapp4:4444
```

Now on load testing, this application should be able to serve way more requests than the single instance one.

High-level overview of Scaling of Systems with AWS

Scaling an architecture from 0 users to 1 million concurrent users.

1 User

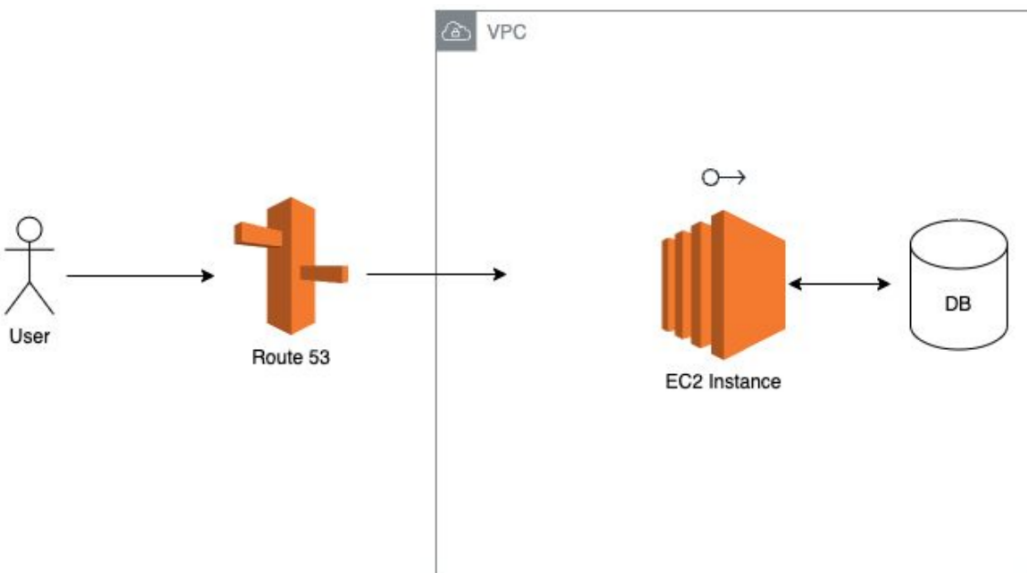


Route53 is DNS service which maps Domain Name to an IP Address, i.e. EC2 Instance, so users can access your application using Domain Name instead of IP Address.

Elastic IP Address helps your EC2 Instance to have the same IP Address across any number of sessions. So even if you restart your EC2 Instance for some reason, it will have the same IP Address which is not the case always.

Note: Always ensure that Elastic IP Address is connected to some EC2 instance, i.e. no dangling addresses. Otherwise you'll be charged separately for it.

> 100 Users



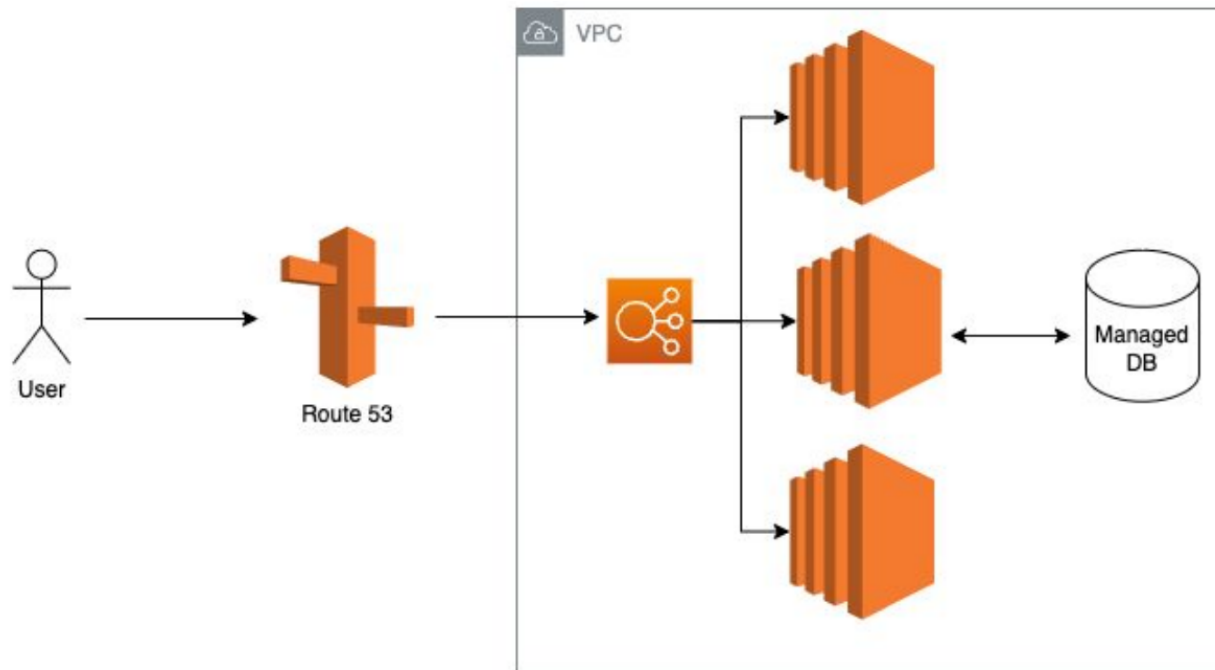
Because the entire application is on the same EC2 instance, there's going to be a lot of load on it and application performance will be slow. So the DB is separated out from the frontend and backend of the system into a managed DB service like DynamoDB, DocumentDB, etc.

Elastic IP Address is still used as indicated by the icon on the EC2 instance.

Do note that the EC2 instance and the DB are in the same network (VPC), so network latency on average is going to be quite low.

You can also do vertical scaling on the EC2 instance to better handle the increased load.

> 500 Users

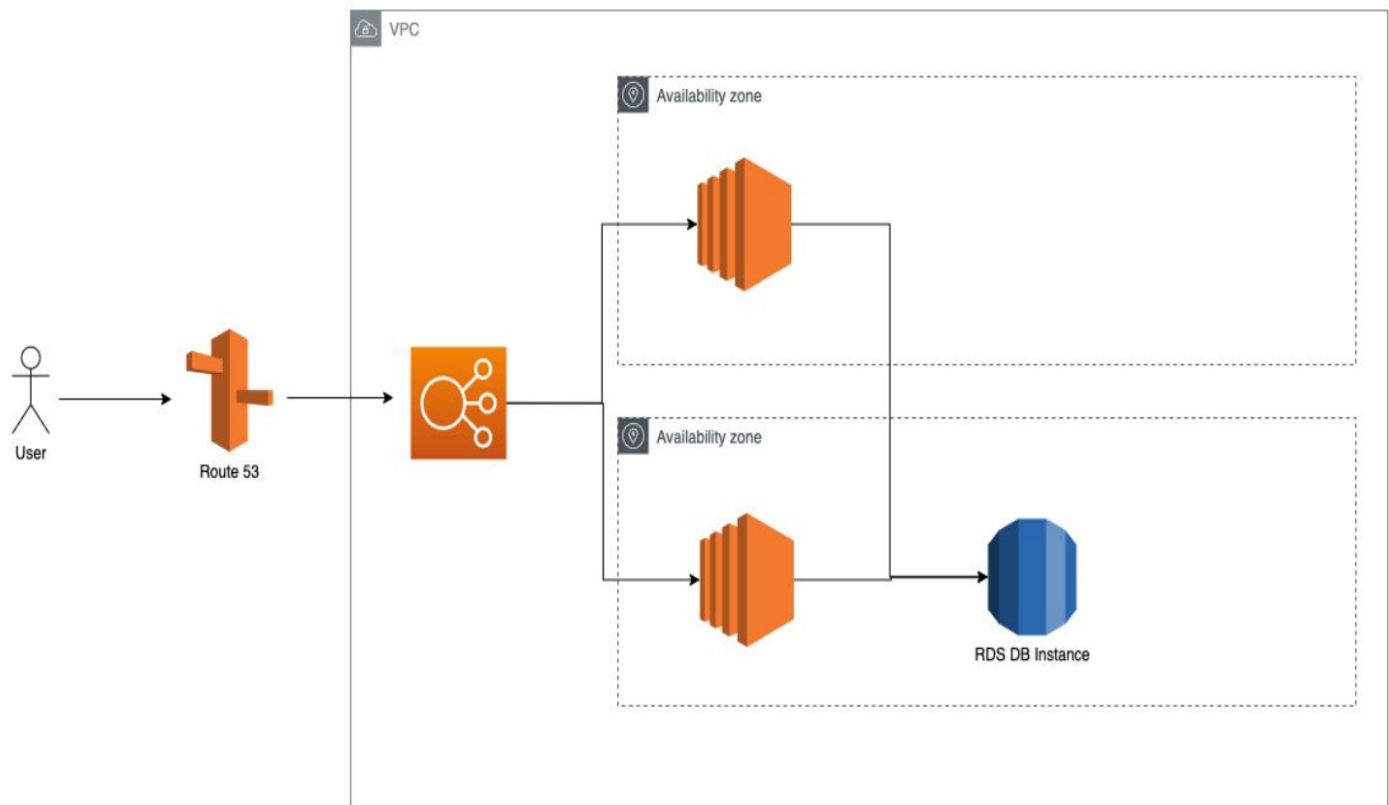


Horizontally scale the system with the help of multiple EC2 instances and Elastic Load Balancer (ELB). There's two key types of ELB, Application LB and Network LB. Network LB is the traditional LB version and Application LB provides additional features like cookie sessions, etc.

If there's very high demand then can't the Load Balancer itself get overloaded? Yes, but ELB is a managed service, so we don't have to worry about what is going on in the backend, how many instances are ELB spinning up, etc.

But if we had our own load balancers then this would be a concern, then we could have a network of load balancers and further implement the concept of availability zones to reroute traffic even further, but this is a very high-level view and a vague description of what can be implemented, the actual solutions can be much different from all this.

> 10,000 Users



An availability zone is like a separate data center itself.

For AWS, they have 3 availability zones in the Mumbai region.

So we can place the various EC2 instances in each of these availability zones, to make the system more fault tolerant. Because if one of the data centers is down then the other ones will help maintain availability, and it is very, very rare that all of the availability zones are down at the same time, so even though an availability zone going down would slow down performance, atleast users will still be able to access your application.

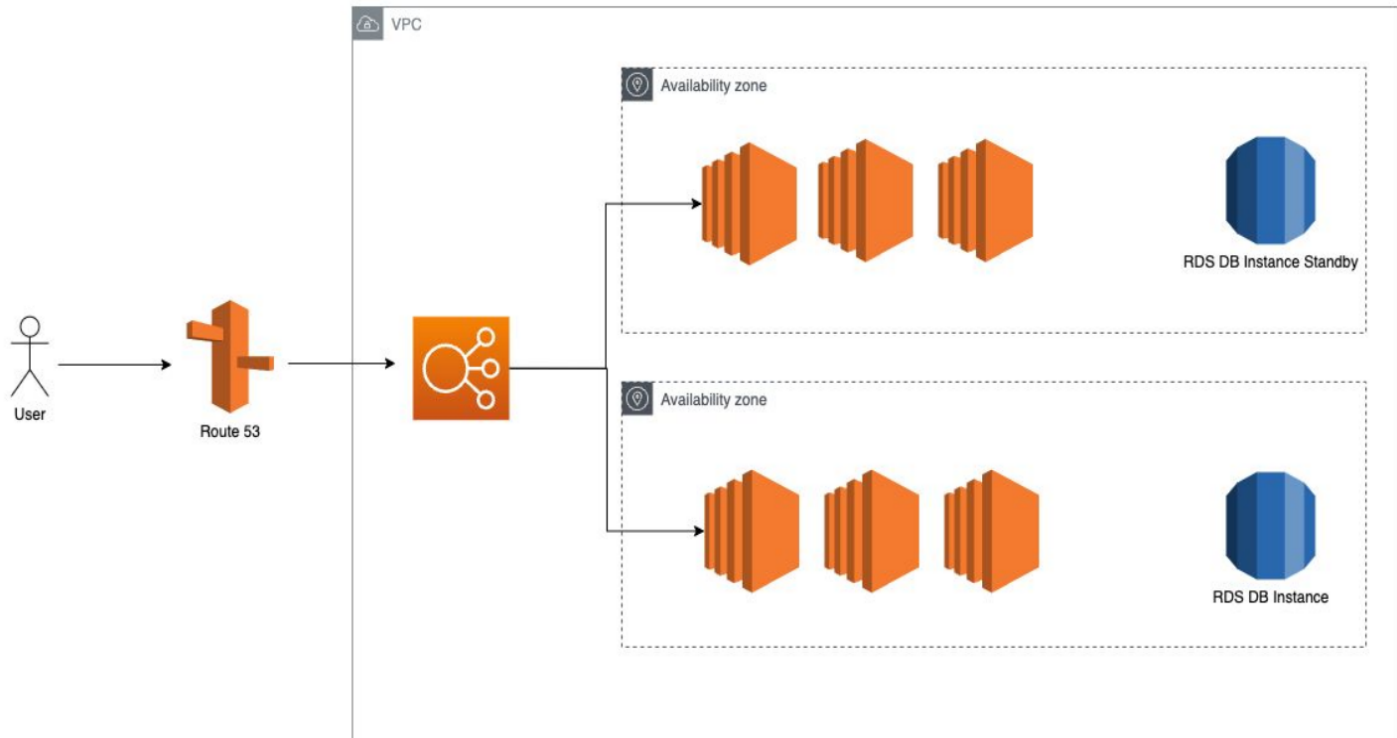
Note that the RDS DB instance is in only one availability zone, so if that zone goes down, then the application is still rip :)

Note: Each availability zone can have its own Load Balancer as well (it's mostly a mandatory requirement for large scale systems).

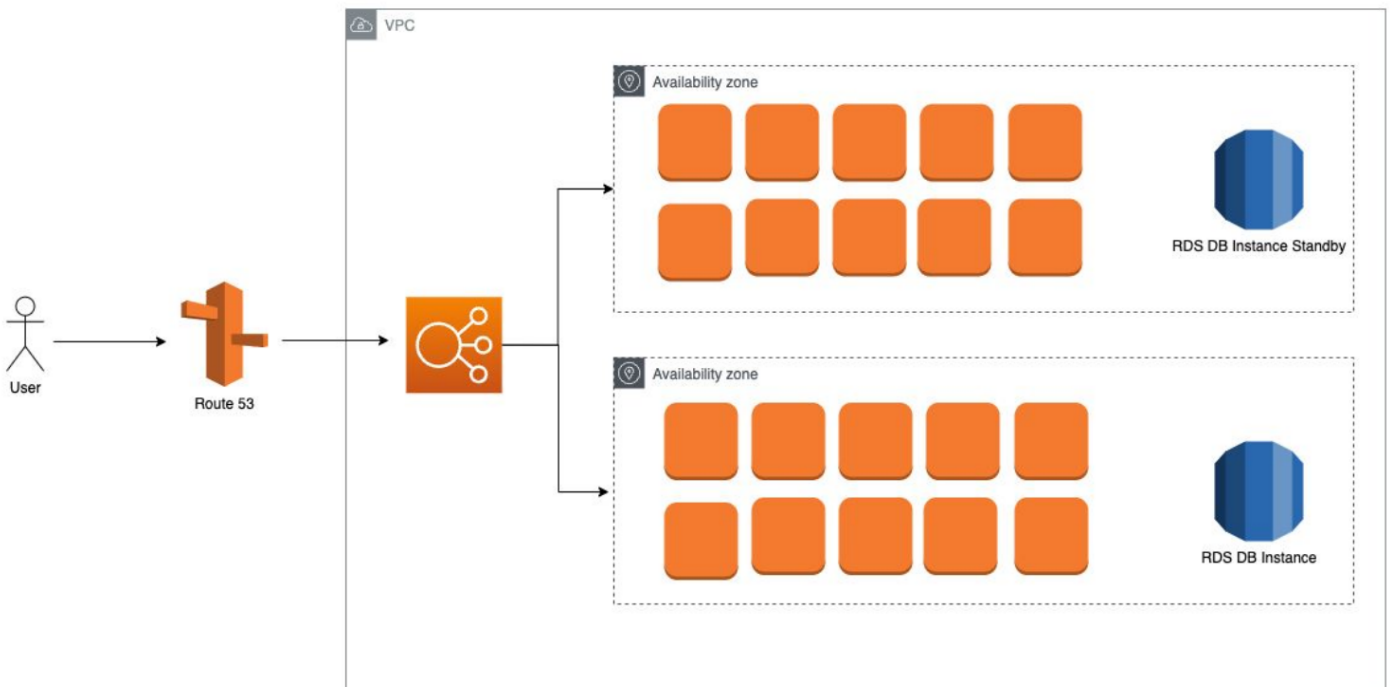
> 1,00,000 Users

We now add auto-scaling to the mix that will require access to the availability zones and VPC and according to the current demand, spin up new EC2 instances as per requirement.

We also add a Standby DB instance in the other availability zones to protect the system from DB failures on the master DB instance.



> 5,00,000 Users

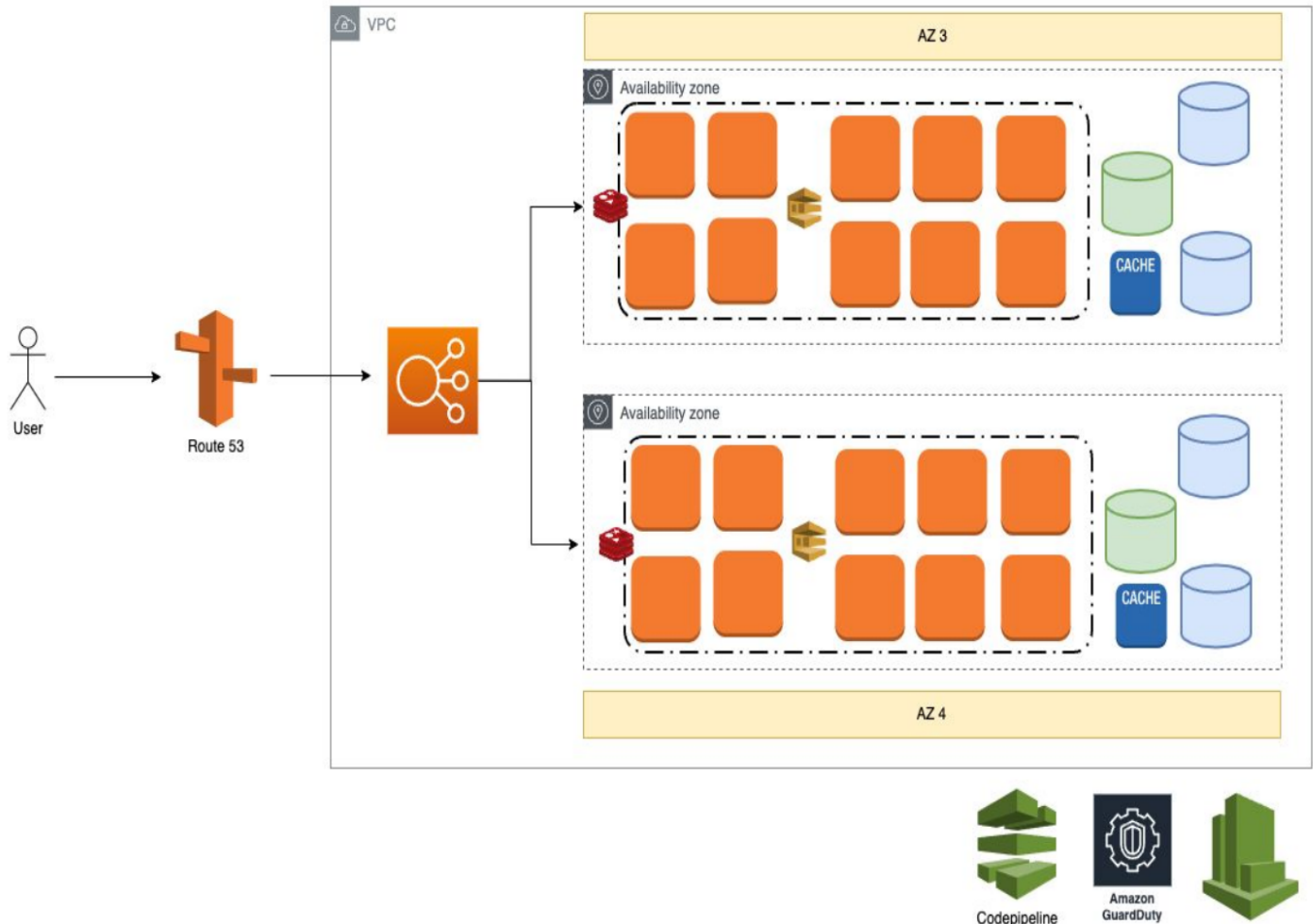


The rounded orange squares are just EC2 instances, basically the point is that we use a lot of EC2 instances.

To optimize the DB architecture even further we can use the master-slave architecture in each availability zone where master DB is used for writes and slave DBs are used for reads. The

writes made in the master DB will be propagated to the slave DBs using event management and polling mechanisms.

> 1,000,000 Users



Redis (Red dice type structure in the diagram) instances are added to the Availability Zones. Redis is an in-memory data storage mechanism that maps requests to their responses.

Basically Redis serves as a cache.

Suppose a User requests for some service from the application, then it has to go through the following sequence of events,

Route 53 -> Load Balancer -> Availability Zone -> EC2 Instance -> Execute given request and access DB instances for R/W operations -> Return response back via the same route back to the User

Now the benefit of adding Redis is the following,
When a response is sent back to the user, it is also cached by Redis.

So now if the user makes the same request then first a check will be made in the Redis store and the response can be immediately served back to the User without having to ever access any EC2 instance.

Note: You can place Redis stores at any layer in the design, behind Load Balancer, Availability Zone, before Data Tiers (in order to avoid DB accesses and preventing latencies related to ORM, etc.)

In this same architecture, further optimization is done with the help of Message Queues (represented as a box in between the EC2 instances in the above diagram), these help store large amounts of request data (such as videos, images, etc.) as an intermediate storage entity and quickly send back response to the User without having to go through the process of committing to DB and then sending back the response.

So once data gets stored in Message Queue, you are pretty certain that it will eventually get committed to DB and so you are free to return the response to the User that your files have been uploaded.

Then for monitoring the system, you can use Amazon CloudWatch.

GuardDuty => Threat Detection using AI/ML tools.

CodePipeline => For DevOps (like Jenkins I guess)

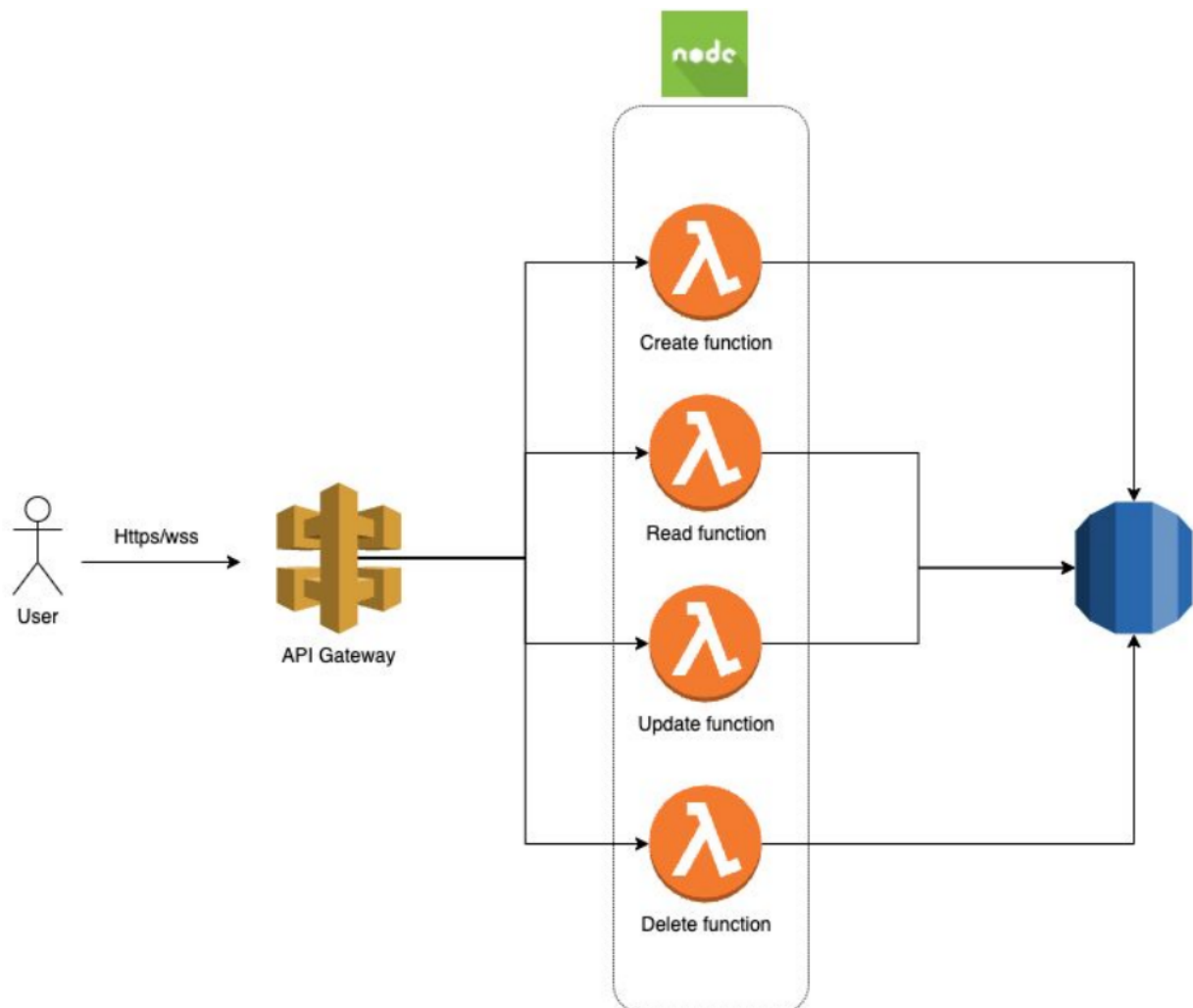
AZ = Availability Zone, so basically we can add more Availability Zones in this as per requirement.

Serverless Computing with Lambda Functions

Why is Serverless required?

- Scaling and managing an application requires a lot of manual work.
- Spend more time on writing code than on monitoring whether is system is up and running
- Pay less than what you are paying (Pay for what you use).
- Serving the application to millions and millions of users without the worry of maintaining the servers.

Use Case 1: CRUD Operations with DB

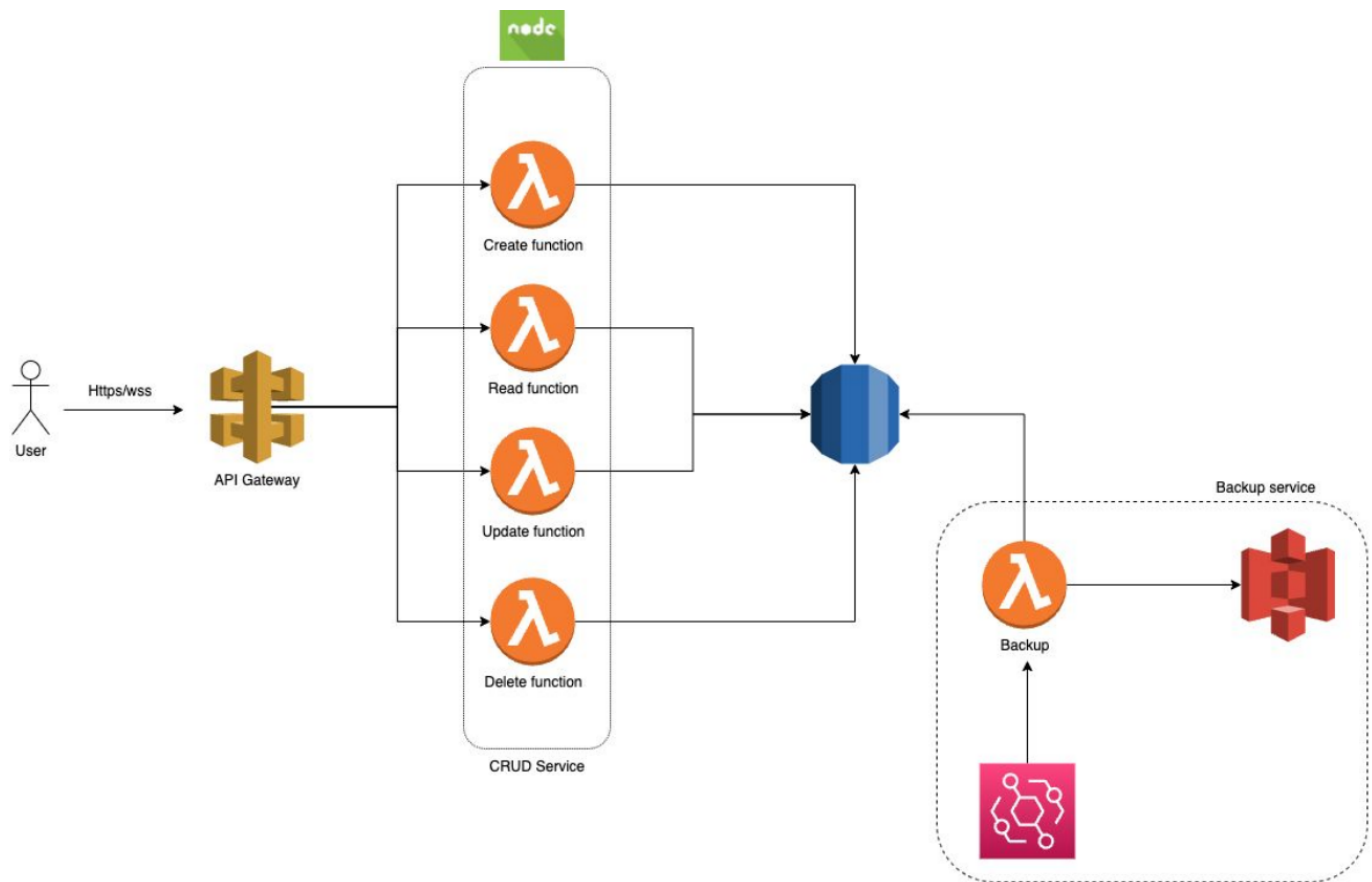


So there's 4 Lambda functions for each of the CRUD operations and they are written in Node.js

As discussed before, Lambda functions are event triggered, so there has to be a service that will take care of determining when to trigger which Lambda function and that is what the API Gateway service does. It will also return the response of the Lambda function back to the user.

So if an API request (HTTP/Web Socket) is made on a path like `/create` then API Gateway will take care to ensure that the "Create" Lambda function is executed.

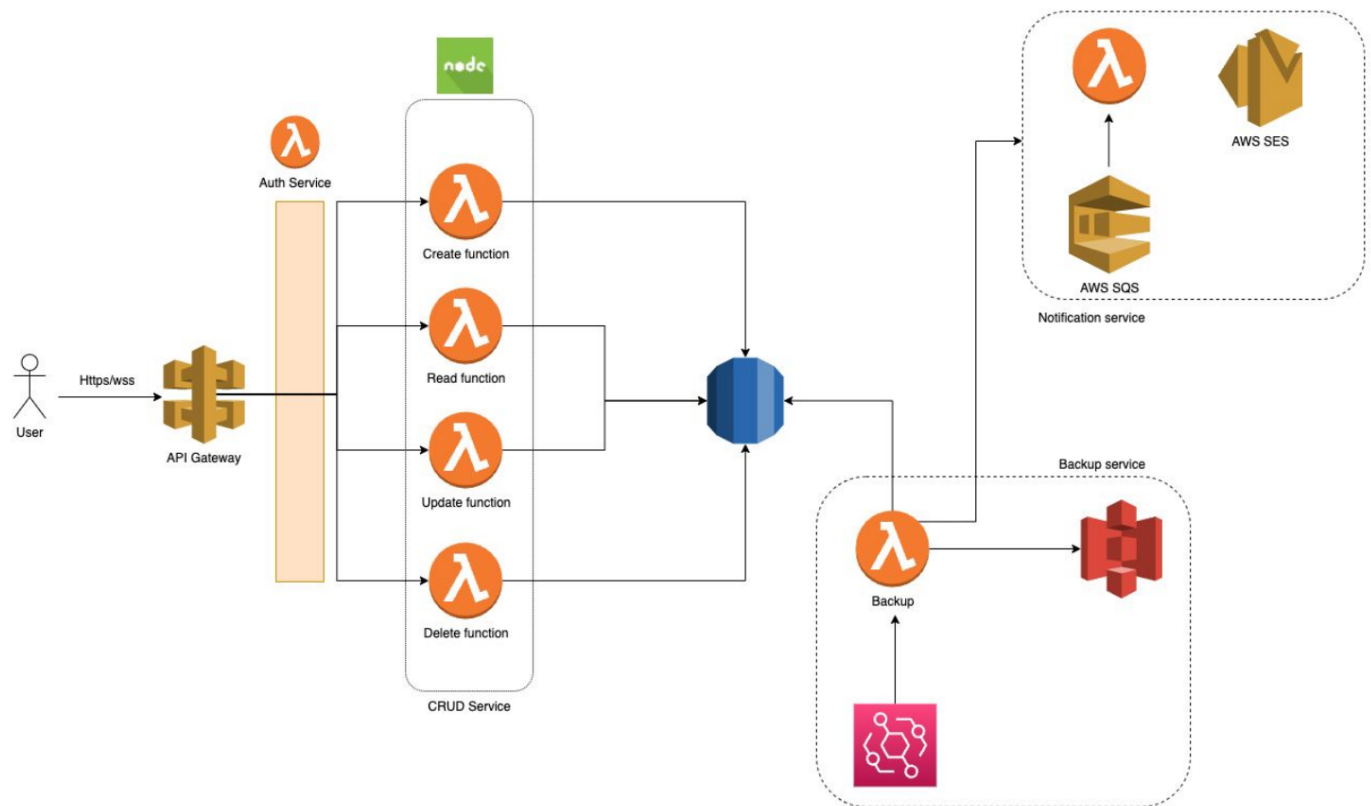
Use Case 2: Backup Service



In this case, a new Lambda function “Backup” is added that will periodically backup the DB data in an S3 store (the red puzzle type block).

But how to trigger this function? An API Gateway can be used but it doesn’t make sense as this is a backend service and only has one function. So the “Event Bridge” service (the magenta coloured puzzle block) is used which can be used to perform cron jobs which is essentially what the backup service is really. Basically, the Event Bridge service takes in certain parameters using which an event will be triggered at certain time intervals or when certain conditions are met. This event will be used to trigger the Backup Lambda function.

Use Case 3: Auth Service, Notification Service



You can add an Authentication Lambda function before the CRUD layer, so API Gateway first sends the request to the Authentication service and upon successful authentication the request is sent to the appropriate Lambda function.

You can keep a Notification system where every time a backup is made, a notification is sent to your email or something like that.

The process of sending an Email can be done via SES (Simple Email Service) but it requires someone to give the appropriate call to it to execute. This can be done by a Lambda function. But Lambda function itself requires a trigger to execute, for this you can use a Message Queue, AWS SQS (Simple Queue Service). So whenever a backup happens, the Backup service Lambda function adds an entry in the SQS and the Notification Lambda function is listening on the Queue and when a new entry appears, it is basically equivalent to an event being triggered and the SES task of sending the email is started.

When to use Lambda vs When to use EC2?

For a Medium-scale application, Lambdas make the most sense as they allow you to focus on developing the actual code of the application and are significantly cheaper than EC2 instances. This is because Lambdas charge per request made and the time the functions execute for and the memory that they consume which are very low charges. Whereas EC2 charges on an hourly basis and these charges ain't no joke either.

But if you are a large-scale application like Google, Facebook, etc. then EC2 would make more sense as you actually have that much demand and having your own EC2 instances would be preferable.

Sidenote: LocalStack is an interesting platform to thoroughly test your AWS cloud application on the local machine itself.

For more info on system design, <http://highscalability.com/>

15/02/22

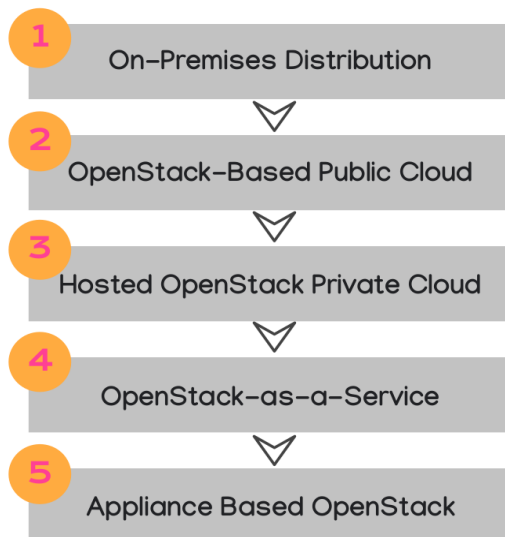
OpenStack

OpenStack is a free open standard cloud computing platform, mostly deployed as infrastructure-as-a-service in both public and private clouds where virtual servers and other resources are made available to users .

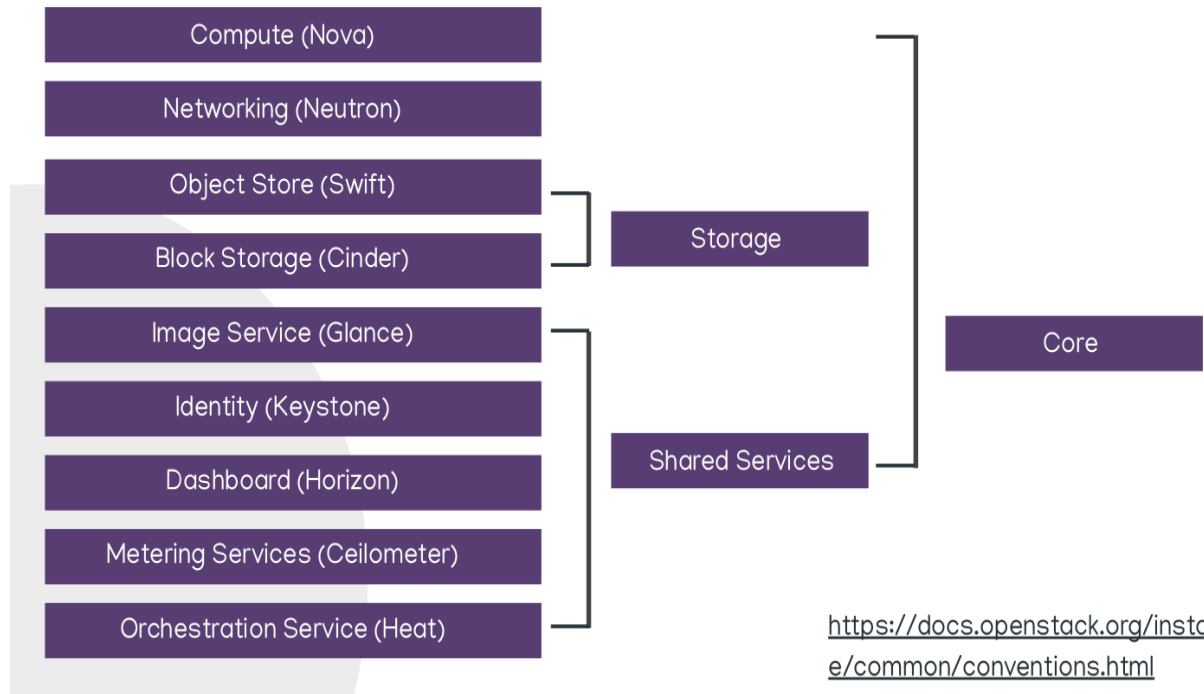
OpenStack is a cloud operating system that controls large pools of compute, storage, and networking resources throughout a datacenter, all managed and provisioned through APIs with common authentication mechanisms. A dashboard is also available, giving administrators control while empowering their users to provision resources through a web interface.

OpenStack believes in open source, open design, and open development; all in an open community so anyone can participate.

OpenStack Model



OPENSTACK SERVICES



Keystone

Keystone is an OpenStack service that provides API client authentication, service discovery, and distributed multi-tenant (role-based) authorization by implementing OpenStack identity API.

- Common authentication system
- Central catalog of services and endpoints
- Supports LDAP, AD, MYSQL
- Provides a token for subsequent auth requests.

Provides 4 types of services,

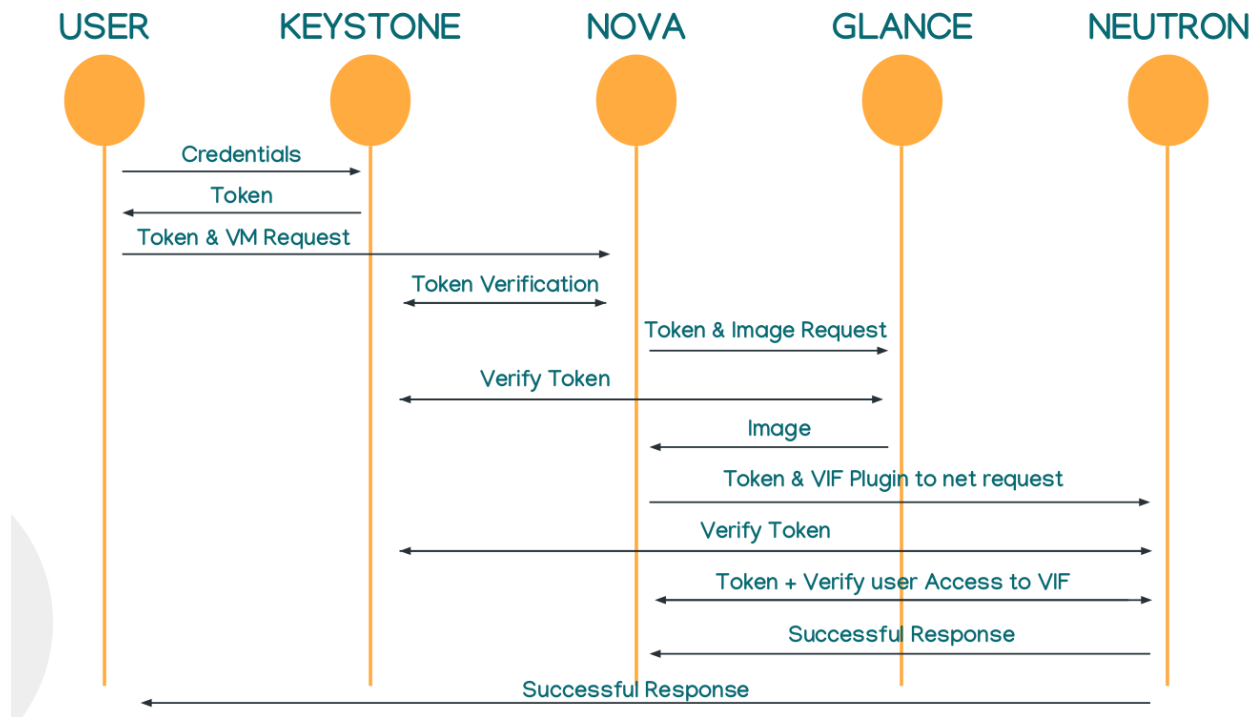
Token Service

Catalog Service

Policy Service

Assignment Service

KEYSTONE - PROCESS FLOW



Glance

- The Glance project provides a service where users can upload and discover data assets.
- This currently includes image and metadata information.
- Glance image service includes discovering, registering and retrieving virtual machine images.

Nova

- Nova is the OpenStack project that provides a way to provision compute instances (aka virtual servers).
- Nova supports creating virtual machines, bare metal servers (through the use of ironic), and has limited support for system containers.
- It runs as a set of daemons on top of existing Linux servers to provide that service.
- It is a component within the OpenStack cloud computing platform developed to provide on-demand access to compute resources by provisioning and managing large networks of virtual machines.
- Nova works with Vmware, Xen, Kvm, Hyper-v and other virtualization technologies.

Neutron

- Neutron is an OpenStack project to provide "Network Connectivity as a Service" between interface devices (e.g., vNICs) managed by other OpenStack services (e.g., Nova).
- It implements the OpenStack Networking API.

- Features,

Network, Subnet and Port abstractions.

Plugins support multiple technologies.

Modular architecture.

Central or distributed environment.

Accepts API requests and routes them to appropriate plugins for action.

Neutron can be scaled to thousands of nodes and networks

Auto allocation for creating networking topologies.

Ability to logically separate provisioning and tenant networks.

Swift

Swift is a highly available, distributed, eventually consistent object/blob store.

Organizations can use Swift to store lots of data efficiently, safely, and cheaply.

Object storage service for OpenStack .

Swift proxy is responsible for tying together the swift of rest architecture.

For each request, it looks up the container, account or object.

Cinder

Cinder is the OpenStack Block Storage service for providing volumes to Nova Virtual Machines, Ironic Bare-metal hosts, containers and more.

Some of the goals of Cinder are to be/have,

Component-based Architecture: Quickly add new behaviors

Highly available: Scale to very serious workloads

Fault-tolerant: Isolated processes avoid cascading failures

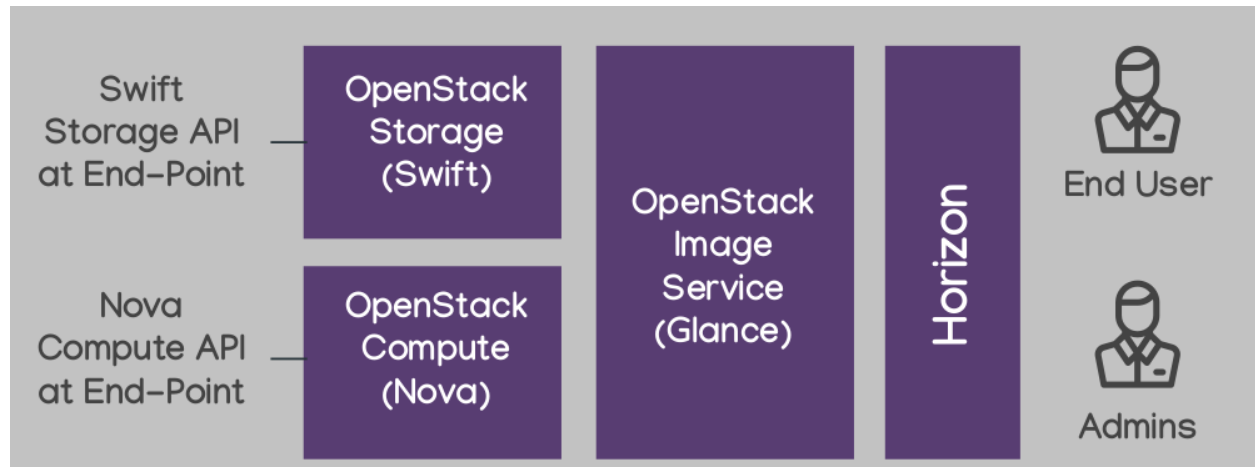
Recoverable: Failures should be easy to diagnose, debug, and rectify

Open Standards: Be a reference implementation for a community-driven API

Horizon

- Horizon is the canonical implementation of OpenStack Dashboard, which provides a web-based user interface to OpenStack services including Nova, Swift, Keystone, etc.

- It is based on the Django model called Django–OpenStack.



Ceilometer

- Ceilometer is a data collection service that provides the ability to normalize and transform data across all current OpenStack core components with work underway to support OpenStack components.
- Its data can be used for billing, resource tracking and alarming capabilities for all OpenStack services.

Heat

Heat is the main project in the OpenStack orchestration program . It implements an orchestration engine to launch multiple composite cloud applications based on templates in the form of text files that can be treated like code.

22/02/22

MapReduce with AWS EC2

Deploy an EC2 instance.
t2.micro and all that stuff.

Tenancy Models for EC2 instances

Shared Tenancy: Your EC2 instance will be on a machine that can contain EC2 instances of other users.

Dedicated Instance: Your EC2 instance will be on a machine that's dedicated for you only. But on restart, your EC2 instance can get relocated to a different machine. It'll still be dedicated for you but a different machine nonetheless.

Dedicated host: Same as dedicated instance except even on restart the underlying machine for the EC2 instance remains the same.

17/03/22

Virtualization

[Below info is copied from SPE notes, it's atleast more competent info :)]

Introduction to Virtualization

Software demands more and more from operating systems to applications.

- More : data, processing power, memory, network bandwidth and storage space.

Virtualization is a technology.

- Create multiple simulated environments from a single, physical hardware system.

Software called a hypervisor connects directly to the hardware.

- Splits one system into separate, distinct, and secure environments known as virtual machines (VMs).

[Hypervisor = Supervisor of the Supervisor, Supervisor of the OS is the kernel and Hypervisor is Supervisor of the Kernel]

The hypervisor/VMM (Virtual Machine Manager) is the control system at the core of virtualization.

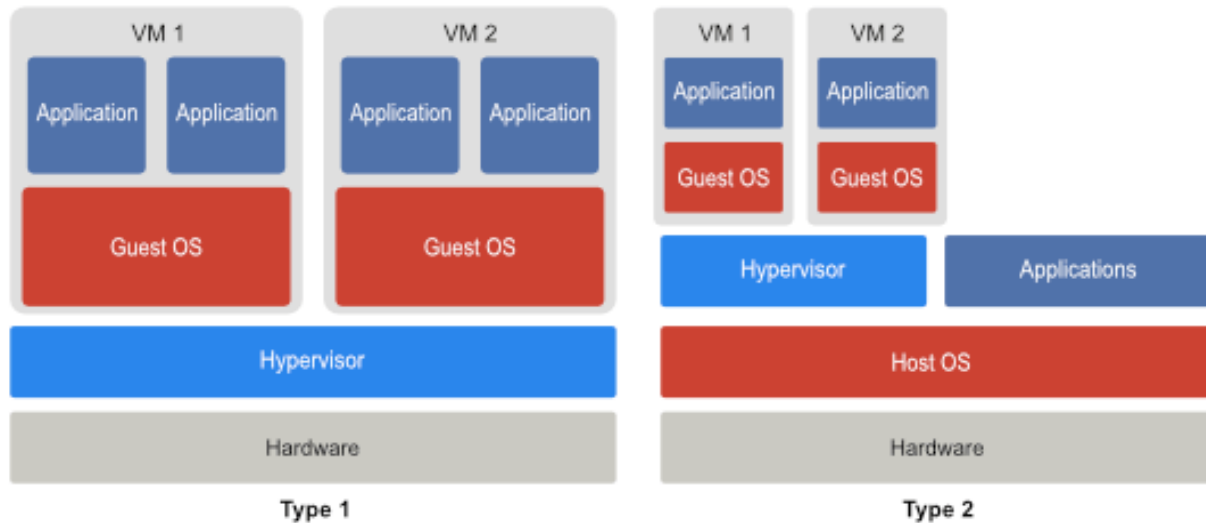
- Acts as the control and translation system between the VMs and the hardware.

Hypervisors need OS - level components to run VMs

- Memory manager, Process scheduler, I/O stack, DD, security manager and Network stack.

The physical hardware, equipped with a hypervisor is called the Host, while the many VMs that use its resources are Guests.

Hypervisor Types



Type 1 Hypervisor = Bare-metal Hypervisor => There is no host OS and instead Hypervisors like KVM manages the system on top of which Guest OS and VMs can be hosted.

Type 2 Hypervisor = Hosted Hypervisor => There is a host OS on which Hypervisors like VirtualBox is installed and on top of which various VMs can be hosted.

Benefits of Virtualization

- Reduce power consumption and air conditioning needs (move to be more eco-friendly).
- Trim the building space and land requirements.
- Reduce downtime and provide high availability for critical applications (Blue-Green model).
- Simplify IT operations.
- Allow IT organizations to respond faster to changing business demands

Virtual Machine Monitor/Hypervisor

A software that securely partitions the resources of a computer system into one or more VMs. It controls the guest OS.

VMM runs in kernel mode and guest OS runs in user mode.

VMMs allow several OSs to run concurrently on a single hardware platform.

VMMs enforce isolation among these systems, thus enhancing security.

Virtualization simulates the interface of a physical object by any one of the four means,

Multiplexing: Create multiple virtual objects from one instance of a physical object [multiple virtual processors from one physical processor core].

Aggregation: Create one virtual object from multiple physical objects. [like DFS].

Emulation: Construct a virtual object from a different type of physical object [part of the disk will be treated as RAM].

Multiplexing and Emulation: Example, Virtual Memory [disk is treated as memory, i.e. emulation and paging enables multiplexing because one physical hard disk is divided into many virtual pages]

Virtual Machines

A VM is an isolated environment that appears to be a whole computer but actually only has access to a portion of the computer's resources.

Types

Process VM

A process VM is a virtual platform created for an individual process and destroyed once the process terminates.

Process VMs can be divided into,

Same ISA (Instruction Set Architecture - defines a processor's set of instructions) as underlying host machine.

Different ISA than underlying host machine.

System VM

A system VM provides a complete system, each VM can run its own OS which in turn can run multiple applications.

They implement OS-level virtualization technology.

System VMs might not have a host OS [VMM will have to handle everything in this case], whereas Process VMs will always have a host OS.

Classification of System VMs

Traditional VMs - Baremetal Hypervisors

Hosted VM - Hosted Hypervisors [VMWare]

Hybrid VMs - Like Hosted Hypervisors but the VMM can directly interact with the hardware without having to go through the Host OS because it coexists with the host OS [VMWare Workstation].

Advantages of System VMs with Host OS:

Easier to build and install.

The VM could use several components of the host OS.

Disadvantages of System VMs with Host OS:

Increased overhead and associated performance penalty.

The I/O operations, page faults and scheduling requests from a guest OS are not handled directly by the VMM. Instead they are passed on to the guest OS.

Full Virtualization

In this kind of virtualization, the Guest OS can remain in its original form and does not need to be modified to enable communication with the VMM.

Clearly this is very tricky to implement.

There can be multiple Guest OSs on a given VMM and at any given point in time, only one of these guest OS can be in the kernel mode and rest all will be in the user mode without having access to privileged instructions. However these guest OSs will not be able to understand which instructions are accessible or not depending on the current state which is an issue.

So these guest OSs will actually give out privileged instructions and the VMM will have to manage it somehow.

When a guest OS wants to enter into the kernel mode, it will send a trap signal to the VMM and the VMM will emulate the scenario as if a trap was sent to the actual kernel. VMM has to trap the privileged instructions and emulate their behavior.

[Trap and Emulate Strategy = Strategy to design VMMs]

CPUs have multiple privilege levels, not just two.

For example, Ring 0,1,2,3 in x86 CPUs

Ring 0 = Kernel/OS, Rings 1-3 = User Processes

In the case of virtualization, VMM/Host OS = Ring 0, Guest OS = Ring 1 and everything else goes from Rings 2-3

Limitations with the above strategy,

Performance overhead.

Not all architectures are suitable for implementing this strategy.

Problem specifically with x86 systems

- Certain CPU registers in x86 will show the Privilege level/mode of the OS.
- The guest OS may identify that it is running at a lower privileged mode - than the highest privilege level.
- Some sensitive x86 instructions will behave differently than expected in ring-1, it will not trap to VMM. Eg, popf

22/03/22

Why x86 is not Virtualizable

- Eflags is a CPU register in the x86 architecture.
- IF flag is the Interrupt Enable flag - for enabling/disabling the interrupts.
- OS uses popf instruction to pop the values from the top of the stack and write it onto the Eflags registry.
- popf is a behavior sensitive instruction that acts differently in ring-1 and ring-0.

Some of these flags can only be modified in ring-0 only. So popf will not be able to modify all the flags on Eflags when in ring-1 and only those flags which are accessible to the unprivileged level will get updated. An example of a privileged level flag is IF.

Therefore the behavior sensitive instructions like popf while running in non-privileged mode does not trap - behaves differently in ring-0 and ring-1.

Popek Goldberg Theorem

An architecture is virtualizable if the set of Sensitive Instructions (Behavior + Control) is a subset of the Privileged Instructions.

Note: Instructions (Sensitive Instructions) within an IS can be classified into Behavior (different behavior depending on whether its executed in user or kernel mode) and Control Instructions.

And this theorem basically states in a nutshell why the x86 architecture is not virtualizable.

For virtualizable architectures, we can easily obtain full virtualization without even modifying the source code of the guest OS via the trap-and-emulate strategy.

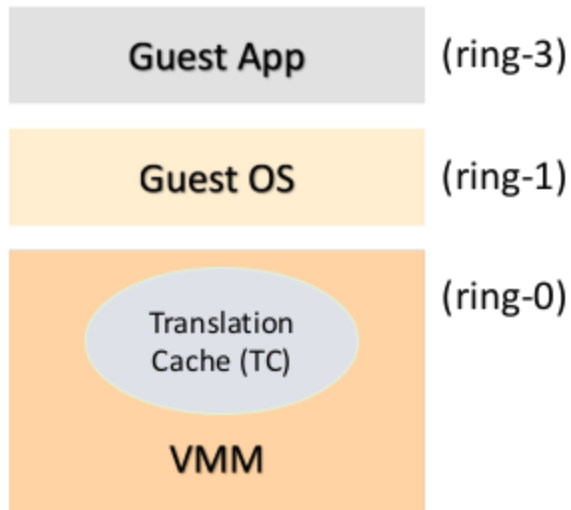
For non-virtualizable architectures, it is difficult to implement full virtualization with the trap-and-emulate strategy without modifying the source code.

Binary Translation

An approach to overcome issues related to the trap-and-emulate strategy in Full Virtualization with non-virtualizable architectures without modifying the source code of the guest OS.

Binary of the Guest OS is translated **instruction-by-instruction** and stored in the **Translation Cache (TC)**.

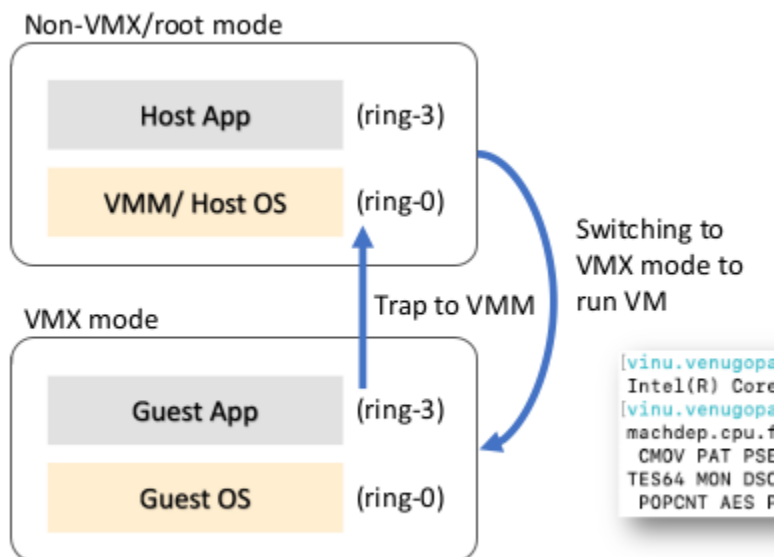
Translated Guest OS code is executed from TC in ring-1.
Privileged instructions would trap to the VMM.



Hardware-assisted Virtualization

- Developed by Intel and AMD for their server platforms and was designed to improve the performance of the processor.
- All modern CPUs have hardware support for virtualization, and VMMs are built over this support.
- Comes with a special mode of operation known as VMX mode for running VMs.
- E.g. QEMU/KVM hypervisor in Linux.

VMX Mode



VMX mode exists in parallel with the root mode and has its own set of 4 rings like the root mode. So the Guest OS will believe that it has the correct privilege associated to execute all instructions and we can then place a trap to the VMM in case of privileged instructions.

Para-Virtualization/Xen Architecture

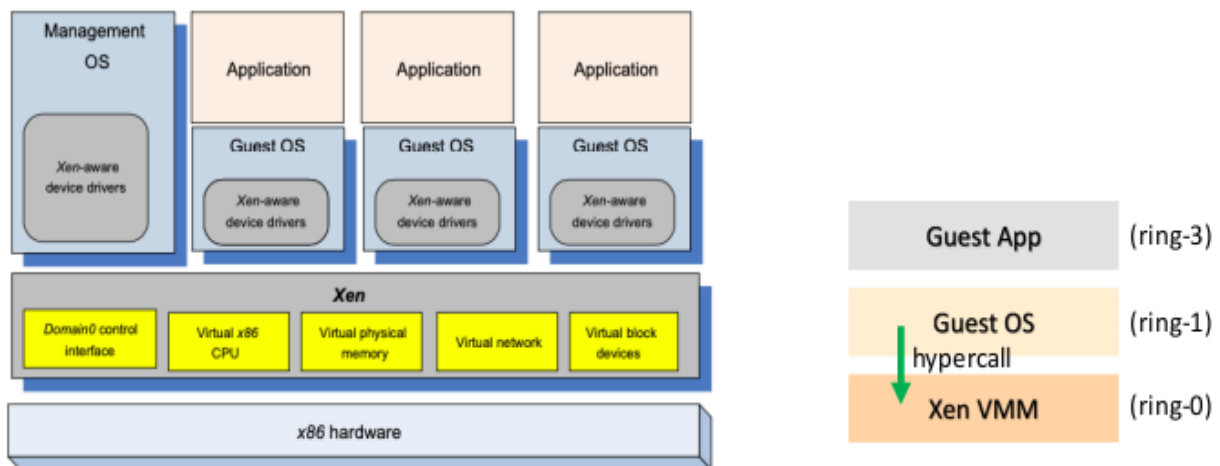
- Rewrite source of the guest OS to make it virtualizable.
- Guest OS makes **hyper-calls** to the VMM rather than invoking privileged operations.
- Example: Xen Hypervisor family.
- XenoLinux is a modified Linux OS that runs on Xen hypervisor
- Note that the application interface need not change.

Advantage: Better performance than binary translation in full-virtualization

Disadvantage: Requires changes to a small % of the source code of the OS.

Architecture:

[VMs are known as Domains in this case]



CPU Virtualization in Xen

- Privileged instructions would trap to Xen
- Hypercalls are made from Guest OS (domains) to Xen which is synchronous in nature as the domains will pause while the Xen VMM processes the hypercall.
- However the communication from Xen to the domains is asynchronous in nature.
- Domains register event handler callback functions.

Interrupt handling in Xen

- When a trap event occurs, Xen pushes the trap frame onto the kernel stack of the OS.
- Next, the guest interrupt handler will get invoked.
- To achieve virtualization, the guest OSs need to register an interrupt descriptor table with Xen.

29/03/22

Containerization & Docker

Background?

We know of virtualization, where the primary motive is to provide a platform that lets you create useful IT services using resources that are traditionally bound to hardware. It allows you to use a physical machine's full capacity by distributing its capabilities among many users or environments.

The virtualization technology uses a hypervisor to create a new virtual machine by hardware level virtualization. Some of the popular virtualization software are VMWare, Virtual Box and KVM (Kernel based Virtual Machine). This sort of virtualization is known as **Hardware Virtualization**, where the hardware provides architectural support that facilitates building a virtual machine monitor and allows guest OSes to be run in isolation.

How does virtualization work?

Software called hypervisors separate the physical resources from the virtual environments—the things that need those resources. Hypervisors can sit on top of an operating system (like on a laptop) or be installed directly onto hardware (like a server), which is how most enterprises virtualize. Hypervisors take your physical resources and divide them up so that virtual environments can use them.

Resources are partitioned as needed from the physical environment to the many virtual environments. Users interact with and run computations within the virtual environment (typically called a guest machine or virtual machine). The virtual machine functions as a single data file. And like any digital file, it can be moved from one computer to another, opened in either one, and is expected to work the same.

When the virtual environment is running and a user or program issues an instruction that requires additional resources from the physical environment, the hypervisor relays the request to the physical system and caches the changes—which all happens at close to native speed (particularly if the request is sent through an open source hypervisor based on KVM, the Kernel-based Virtual Machine).

Containers

There's a new sort of virtualization today which is widely used across the IT market, where the kernel allows the existence of multiple isolated user space instances. These isolated user space instances are containers. Such instances may look like real computers from the point of view of

programs running in them. A computer program running on an ordinary operating system can see all resources (connected devices, files and folders, network shares, CPU power, quantifiable hardware capabilities) of that computer. However, programs running inside of a container can only see the container's contents and devices assigned to the container. This sort of Virtualization is called OS-level virtualization.

Containers are a lighter-weight, more agile way of handling virtualization — since they don't use a hypervisor, you can enjoy faster resource provisioning and speedier availability of new applications. Rather than spinning up an entire virtual machine, containerization packages together everything needed to run a single application or microservice (along with runtime libraries they need to run). The container includes all the code, its dependencies and even the operating system itself. This enables applications to run almost anywhere — a desktop computer, a traditional IT infrastructure or the cloud.

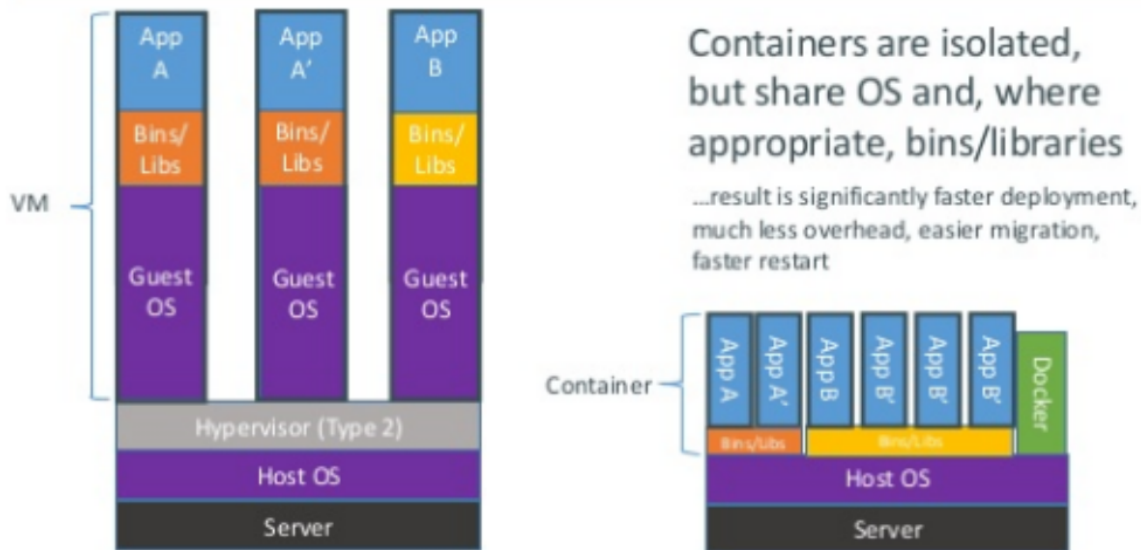
What's the difference?

While a Virtual Machine is a whole other guest computer running on top of your host computer (sitting on top of a layer of virtualization), Container is an isolated portion of the host computer, sharing the host kernel (OS) and even its bin/libraries if appropriate.

Instead of virtualizing the underlying hardware, containers virtualize the operating system (typically Linux or Windows) so each individual container contains only the application and its libraries and dependencies. Containers are small, fast, and portable because, unlike a virtual machine, containers do not need to include a guest OS in every instance and can, instead, simply leverage the features and resources of the host OS.

Just like virtual machines, containers allow developers to improve CPU and memory utilization of physical machines. Containers go even further, however, because they also enable microservice architectures, where application components can be deployed and scaled more granularly. This is an attractive alternative to having to scale up an entire monolithic application because a single component is struggling with load. Now you know why Containers?

Containers vs. VMs



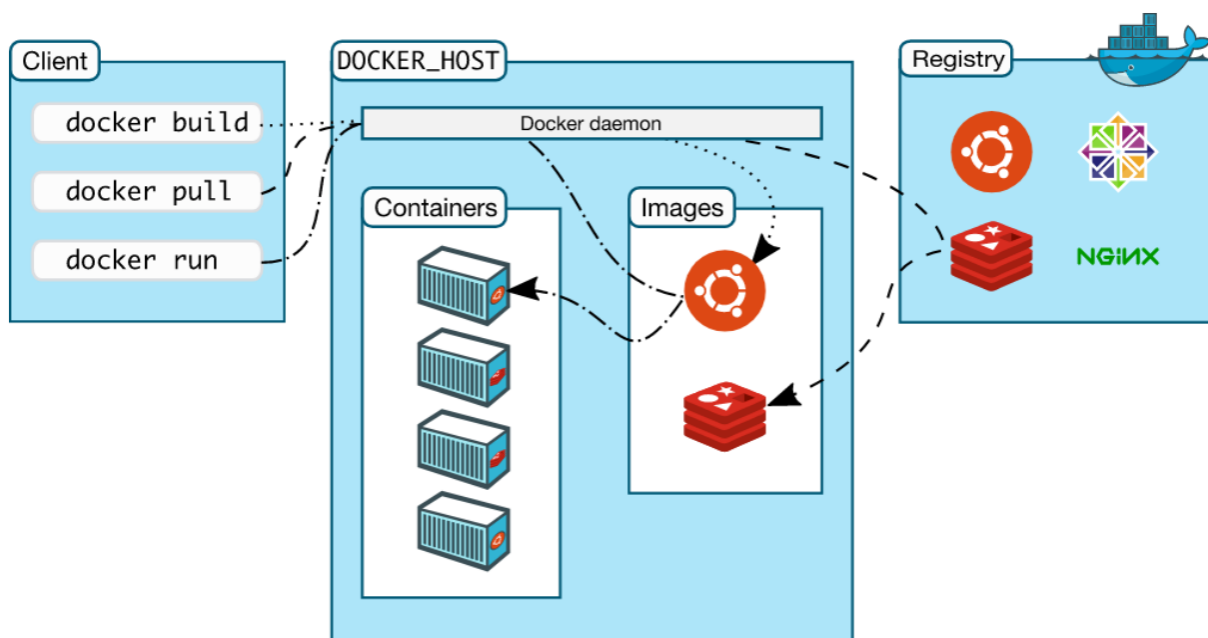
Docker

Docker is a Container. Docker can package an application and its dependencies in a virtual container that can run on your Linux machine.

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production.

Architecture

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, which lets you work with applications consisting of a set of containers.



Docker daemon - The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes. A daemon can also communicate with other daemons to manage Docker services.

Docker client - The Docker client (docker) is the primary way that many Docker users interact with Docker. When you use commands such as docker run, the client sends these commands to dockerd, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

Docker registry - A Docker registry stores Docker images. Docker Hub is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default. You can even run your own private registry.

Docker Objects

When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects.

Image - An image is a read-only template with instructions for creating a Docker container. Often, an image is based on another image, with some additional customization. For example, you may build an image which is based on the Ubuntu image, but install the Apache web server and your application, as well as the configuration details needed to make your application run.

You might create your own images or you might only use those created by others and published in a registry. To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it. Each instruction in a Dockerfile creates a layer

in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt. This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

Containers - A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.

A container is defined by its image as well as any configuration options you provide to it when you create or start it. When a container is removed, any changes to its state that are not stored in persistent storage disappear.

Example docker run command

The following command runs an ubuntu container, attaches interactively to your local command-line session, and runs `/bin/bash`.

```
docker run -i -t ubuntu /bin/bash
```

When you run this command, the following happens (assuming you are using the default registry configuration):

- If you do not have the ubuntu image locally, Docker pulls it from your configured registry, as though you had run `docker pull ubuntu` manually.
- Docker creates a new container, as though you had run a `docker container create` command manually.
- Docker allocates a read-write filesystem to the container, as its final layer. This allows a running container to create or modify files and directories in its local filesystem.
- Docker creates a network interface to connect the container to the default network, since you did not specify any networking options. This includes assigning an IP address to the container. By default, containers can connect to external networks using the host machine's network connection.

```
docker container inspect <container_id>
```

Check `NetworkSettings["Networks"]` and compare it with the output of, `ifconfig` or `ip a s`

```
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:8f:18:92:d6 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
    valid_lft forever preferred_lft forever
```

- Docker starts the container and executes `/bin/bash`. Because the container is running interactively and attached to your terminal (due to the `-i` and `-t` flags), you can provide input using your keyboard while the output is logged to your terminal.
- When you type `exit` to terminate the `/bin/bash` command, the container stops but is not removed. You can start it again or remove it.

Docker Lifecycle

- `docker create` creates a container but does not start it.
- `docker rename` allows the container to be renamed.
- `docker run` creates and starts a container in one operation.
- `docker rm` deletes a container.

Normally if you run a container without options it will start and stop immediately, if you want to keep it running you can use the command, `docker run -t -d container_id` this will use the option `-t` that will allocate a pseudo-TTY session and `-d` that will detach automatically the container (run container in background and print container ID).

Dockerfile

Docker builds images automatically by reading the instructions from a Dockerfile -- a text file that contains all commands, in order, needed to build a given image. A Dockerfile adheres to a specific format and set of instructions which you can find at [Dockerfile reference](#).

Dockerfile instructions

FROM - It is the base image of your docker image. Whenever possible, use current official images as the basis for your images for it to intake subsequent instructions.

```
FROM ubuntu:20.04
```

LABEL - You can add labels to your image to help organize images by project, record licensing information, to aid in automation, or for other reasons. For each label, add a line beginning with `LABEL` and with one or more key-value pairs.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
```

RUN - The `RUN` instruction will execute any commands in a new layer on top of the current image and commit the results.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
```

CMD - The CMD instruction should be used to run the software contained in your image, along with any arguments. CMD should almost always be used in the form of CMD ["executable", "param1", "param2"...]. There can only be one CMD instruction in a Dockerfile. If you list more than one CMD then only the last CMD will take effect.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
```

EXPOSE - The EXPOSE instruction indicates the ports on which a container listens for connections.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
```

ENV - The ENV instruction is also useful for providing required environment variables specific to services you wish to containerize.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
ENV PG_MAJOR=9.3
```

ADD or COPY - Although ADD and COPY are functionally similar, generally speaking, COPY is preferred. That's because it's more transparent than ADD. COPY only supports the basic copying of local files into the container, while ADD has some features (like local-only tar extraction and remote URL support) that are not immediately obvious. Consequently, the best use for ADD is local tar file auto-extraction into the image, as in *ADD rootfs.tar.xz /*

```
FROM ubuntu:20.04
```

```
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
ENV PG_MAJOR=9.3
COPY requirements.txt /tmp/
ADD https://example.com/big.tar.xz /usr/src/things/
```

ENTRYPOINT - An ENTRYPOINT allows you to configure a container that will run as an executable.

WORKDIR - The WORKDIR instruction sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile. If the WORKDIR doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
ENV PG_MAJOR=9.3
COPY requirements.txt /tmp/
ADD https://example.com/big.tar.xz /usr/src/things/
WORKDIR /a/b
```

Example: Build a node application container

```
FROM node:12-alpine
RUN apk add --no-cache python2 g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Some further points

docker <TAB-to-Autocomplete>

On multiple tab presses, will show you a list of commands that can be run after the docker keyword.

docker container ls

Show all active containers

docker container ls -a

Show all containers

docker build -t <tag_name> <path_where_dockerfile_is_stored>

When this Docker image is built, it will be assigned a tag as tag_name

If you are currently in the same directory as the Dockerfile for this image, then you can just replace path with “.”

So the path can be absolute or relative.

docker build -t jasvin/docker_devops .

docker image ls

Show all the pulled images

docker history image_name

Shows you the commands that have been run in order to create the image. Each command forms its own layer on top of the previous image. So each stage shown in the output of history command is its own layer and on top of it we are running the next command and so on till the final image that we had pulled is ready.

For example, *docker history ubuntu*

We do this same thing with the images that we create as well, we pull the Ubuntu image, create a container using it, then install some packages on that container and each added package thus forms a new layer on top of the base image layer, i.e. Ubuntu (should be ringing some bells to what we had learned in SPE).

Note that when a new layer is being added on top of the previous one, we are entering the container into a new state, so while the addition process is going on, the previous state of the container is just an intermediate container that will be removed once the addition of the new layer is complete and then that layer will encapsulate the entire container.

Run docker history on an image that you built and see the magic in action.

<container_id>

While specifying container ids, you only need to specify the unique prefix of the id amongst all other container ids and the command will still work. No need to type out the entire id.

Copy file from host machine to container

docker container cp path_to_file <container_name/container_id>:<absolute_path>

Example,

docker container cp hello.txt 69420:/

69420 = Unique prefix of the container id

hello.txt = Relative file path of the txt file on host machine

/ = Absolute file path of the copied file on the container

Viceversa is also possible,

```
docker container cp 69420:/hello.txt newhello
```

For help on commands use the help option, like,

```
docker container --help
```

Mapping Host Port to a Container Port

```
docker run -d -p 9000:80 httpd
```

Create a container as a daemon (option -d) with the Apache server base image (httpd) and map port 9000 requests on the host machine to port 80 on the container which is Apache's default port.

Pushing to Docker Hub

To push an image, we first need to create a repository on Docker Hub.

1. Sign up or Sign in to Docker Hub. <https://hub.docker.com/>
2. Click the Create Repository button.
3. Name your repo. Make sure the Visibility is Public.
4. Click the Create button!

Next, build your Docker image using the docker build command (which means you must have a Dockerfile and all), assign it a tag in the format, `<username/dockerhub_repository:tagname>` name.

tagname is usually used to specify versions like Ubuntu 20, 18, etc.

If no *tagname* is specified then it is considered to be the latest version.

Now login to Dockerhub using, *docker login*

After this you can do,

```
docker push <username/dockerhub_repository:tagname>
```

05/04/22

Persist the DB

When a container runs, it uses the various layers from an image for its filesystem.

Each container also gets its own "scratch space" to create/update/remove files.

Any changes won't be seen in another container, even if they are using the same image.

While containers can create, update, and delete files, those changes are lost when the container is removed and all changes are isolated to that container. With volumes, we can change all of this.

Volumes provide the ability to connect specific filesystem paths of the container back to the host machine. If a directory in the container is mounted, changes in that directory are also seen on the host machine. If we mount that same directory across container restarts, we'd see the same files.

To persist the data, we can put that data on the host and ask our container to mount that data when it starts, making it able to pick up where the last one left off. By creating a volume and attaching (often called "mounting") it to the directory the data is stored in, we can persist the data. As our container writes to some file, it will be persisted to the host in the volume.

Named Volumes

Think of a **named volume** as simply a bucket of data. Docker maintains the physical location on the disk and you only need to remember the name of the volume. Every time you use the volume, Docker will make sure the correct data is provided.

1. Create a volume by using the docker volume create command.

```
docker volume create todo-db
```

Stop and remove the running container, as it is still running without using the persistent volume.

3. Start the container, but add the `-v` flag to specify a volume mount. We will use the named volume `"todo-db"` and mount it to `/opt`, which will capture all files created at the path.

```
docker run -it -v todo-db:/opt ubuntu
```

Now go to the `opt` directory in the created container and make some files and put in some data. Now remove the container, the data should be lost.

But create a new container with the same persistent named volume, you can even provide a different mount point as follows,

```
docker run -it -v todo-db:/home ubuntu
```

Now the data that you had added to the `/opt` directory in the previous container would be present in the `/home` directory of the current container.

docker volume ls

You can run the inspect command on a container running with a volume in order to check its mount points.

You can also check the location of the volume on the host machine as well, this is the location where the mount point files will be added and persistence is achieved.

Bind Mounts

With bind mounts, we control the exact mount point on the host. We can use this to persist data, but it's often used to provide additional data into containers. When working on an application, we can use a bind mount to mount our source code into the container to let it see code changes, respond, and let us see the changes right away.

Difference between named and bind mounts are,

1. Host location is chosen by Docker in named mounts, whereas you are in charge of host location in bind mounts.

2. For bind mounts it is,

```
docker run -it -v /path/to/data:/usr/local/data ubuntu
```

This command is mounting the host machine directory */path/to/data* on the directory *usr/local/data* on the container.

Using bind mounts is very common for local development setups. The advantage is that the dev machine doesn't need to have all of the build tools and environments installed. With a single docker run command, the dev environment is pulled and ready to go.

And do note that these bind mounts will be persistent over any number of containers.

So if you create multiple containers bound to the same directory on the host machine, then all of them would be able to see the changes made to them. Be it done via the host machine or via any of the containers.

Multi-Container apps

Up to this point, we have been working with single container apps. But, we now want to add MySQL to the application stack. The following question often arises - "Where will MySQL run? Install it in the same container or run it separately?" In general, each container should do one thing and do it well. A few reasons:

- There's a good chance you'd have to scale APIs and front-ends differently than databases.
- Separate containers let you version and update versions in isolation.
- While you may use a container for the database locally, you may want to use a managed service for the database in production. You don't want to ship your database engine with your app then.
- Running multiple processes will require a process manager (the container only starts one process), which adds complexity to container startup/shutdown.

Remember that containers, by default, run in isolation and don't know anything about other processes or containers on the same machine. So, how do we allow one container to talk to another? The answer is networking.

```
docker network create todo-network
docker run -it --name ubuntu1 --network todo-network ubuntu
docker run -it --name ubuntu2 --network todo-network ubuntu
```

The Docker network *todo-network* would be created as a bridge network by default, there's like 4-5 other types as well which you can look up on your own.

The two containers would be created and connected to the same bridge network *todo-network* and so, the containers would be able to communicate with each other.

To confirm this, install `iputils-ping` on the two containers,

```
apt-get update && apt install iputils-ping
```

After this run,

```
# On ubuntu1
ping ubuntu2

# On ubuntu2
ping ubuntu1
```

You can check under the hood of these containers by running the `inspect` command on a container that's connected to a custom network and one to the default network (bridge). The IP Address changes for the custom network containers because the subnet gets changed.

Docker Compose

Docker Compose is a tool that was developed to help define and share multi-container applications. With Compose, we can create a YAML file to define the services and with a single command, can spin everything up or tear it all down.

The big advantage of using Compose is you can define your application stack in a file, keep it at the root of your project repo (it's now version controlled), and easily enable someone else to contribute to your project. Someone would only need to clone your repo and start the Compose app.

On the host machine, follow the instruction of this link

<https://docs.docker.com/compose/install/>

After installation, you should be able to run the following and see version information.
docker-compose version

Compose File

At the root of the app project, create a file named *docker-compose.yml*

You can look at the Compose file reference for Compose file syntax and features.
<https://docs.docker.com/compose/compose-file/>

Note: “version” parameter is not required in Docker Compose YAML files since version 1.27

Next, we’ll define the list of services (or containers) we want to run as part of our application.

Sample Compose YML File

This YML does the same thing as,
docker run ubuntu echo “Hello Ubuntu”

```
services:
  # ubuntu node = Name of Container
  ubuntu node:
    image: ubuntu
    command: echo "Hello Ubuntu"
```

How to run this YML file?

docker-compose -f docker-compose-v1.yml up

-f = Specify filename of the YML file, if not specified then it will assume the filename to be *docker-compose.yml*

It creates a new network by default even if we don’t specify it and attaches the created containers to it.

We can remove the containers and associated networks with a docker-compose YML file with the command,

docker-compose -f docker-compose-v1.yml down

Two Container Compose YML File

```
services:
  # Container 1
  ubuntu node:
    image: ubuntu
    # stdin = -i option, tty = -t option in docker run
```

```

    stdin_open: true
    tty: true
    # Setting environment variables
    environment:
        MY_VAR: "Hello"
    networks:
        # Assigning static IP address from the specified subnet
        # for net_ubuntu
        net_ubuntu:
            ipv4_address: 172.28.1.5

# Container 2 for Apache Web Server
httpdnode:
    image: httpd
    ports:
        - "8000:80"
    networks:
        net_ubuntu:
            ipv4_address: 172.28.1.6

# Attach both containers to the same network net_ubuntu
networks:
    net_ubuntu:
        # IP Address Management
        ipam:
            driver: default
            config:
                - subnet: 172.28.1.0/24

```

Three Container Docker Compose YML File

(includes networks, volumes, restart policy, dependencies and so much more)

```

services:
    # Container for React Frontend
    clientnode:
        image: <image_name>
        ports:
            - "3000:3000"
        networks:
            net_ubuntu:
                ipv4_address: 172.28.1.5

    # Container for SpringBoot Backend

```

```

servernode:
  image: <image_name>
  # Deployment specification
  deploy:
    # In case servernode container experiences failure
    # then restart the container with a delay of 10s and at max
    # do 5 such attempts with a gap of 120s between each
    restart_policy:
      condition: on-failure
      delay: 10s
      max_attempts: 5
      window: 120s
  ports:
    - "8085:8085"

  # The server node should not start until the DB node has started
  # and that is what the depends_on specification is used to do
  depends_on:
    - "dbnode"
  environment:
    DATABASE_HOST: "dbnode"
    DATABASE_PORT: "3306"
    MYSQL_DATABASE: "database_name"
    MYSQL_USER: "user_name"
    MYSQL_PASSWORD: "password"
  networks:
    net_ubuntu:
      ipv4_address: 172.28.1.6

dbnode:
  image: mysql
  ports:
    - "3306:3306"

  # Bind mount to connect db storage on host machine
  # and MySQL data on the dbnode container
  volumes:
    - /path/to/db:/var/lib/mysql

  # Environment variables for MySQL which you can check in its
  # DockerHub image
  # ROOT_PASSWORD is mandatory variable
  environment:

```



```
    MYSQL_ROOT_PASSWORD: "rootpassword"
    MYSQL_DATABASE: "database_name"
    MYSQL_USER: "user_name"
    MYSQL_PASSWORD: "password"
  networks:
    net_ubuntu:
      ipv4_address: 172.28.1.7

networks:
  net_ubuntu:
    ipam:
      driver: default
      config:
        - subnet: 172.28.1.0/24
```